REPUBLIQUE DU CAMEROUN
PAIX-TRAVAIL-PATRIE
****
UNIVERSITE DE DSCHANG
****
ECOLE DOCTORALE

REPUBLIC OF CAMEROON
PEACE-WORK-FATHERLAND
****
UNIVERSITY OF DSCHANG
****
POST GRADUATE SCHOOL

DSCHANG SCHOOL OFSCIENCES ANDTECHNOLOGY

Unité de recherche en informatique fondamentale, ingénieurie et applications(URIFIA)

THEME:

CONCEPTION D'UN ATELIER DE GÉNIE LOGICIEL POUR LA CRÉATION SYSTÉMATIQUE D'ASSISTANTS CONVERSATIONNELS PAR FINE-TUNING DES LLMs

Mémoire présenté et soutenu publiquement en vue de l'obtention du diplôme de Master of Science en Informatique

Option: **Intelligence Artificielle**

Présenté par :

BIODOKINE JEAN-BAPTISTE

Matricule : CM-UDS-20SCI0827
Licencié en Informatique Fondamentale

Sous la direction de:

TCHOUPE TCHENDJI Maurice

Maitre de conferences, Université de Dschang

Année Académique: 2025

Déclaration d'authenticité

Je soussigné, BIODOKINE Jean-Baptiste, étudiant au Département de Mathématiques et Informatique, Faculté des Sciences, Université de Dschang, certifie que ce mémoire de Master intitulé **conception d'un atelier de génie logiciel pour la création systématique d'assistants conversationnels par fine-tuning des llms**, réalisé au sein de l'unité de recherche en informatique fondamentale, ingénierie et applications (URIFIA) sous la supervision du Pr. TCHOUPE TCHENDJI Maurice, est original. Il n'a jamais été publié auparavant pour l'obtention d'un quelconque diplôme.

L'étudiant
BIODOKINE Jean-Baptiste

.....

Le directeur de mémoire
TCHOUPE TCHENDJI Maurice
Maitre de conferences, Université de Dschang

.....

Le chef de département

.....

Dédicace

À mes adorables parents BIODOKINE MARIUS HERVE et POH MARIE.

Remerciements

Avant toute chose, je rends grâce à **Dieu Tout-Puissant**, source de vie, de sagesse et de paix. C'est dans Sa présence que j'ai puisé la force et la patience nécessaires pour mener à bien ce travail. Sa grâce constante m'a soutenu dans les moments d'incertitude et guidé vers l'aboutissement de ce mémoire. À Lui soient la louange, l'honneur et la reconnaissance.

Ce mémoire représente l'aboutissement d'un parcours académique exigeant, mais aussi profondément enrichissant. Il n'aurait jamais été possible sans le soutien, les conseils et la présence de nombreuses personnes, que je tiens ici à remercier de tout cœur.

Je tiens à exprimer ma reconnaissance à :

— **Pr. TCHOUPE TCHENDJI Maurice**, mon directeur de mémoire, pour son accompagnement rigoureux, ses orientations précieuses et son exigence scientifique, qui ont donné à ce travail sa structure et sa cohérence ;

— **Dr. ZEKENG NDADJI Maxime M.**, pour sa bienveillance, sa pédagogie et ses conseils toujours pertinents à chaque étape de la recherche ;

— Les **enseignants du département d'Informatique de l'Université de Dschang**, pour la richesse de leurs enseignements et la qualité de la formation qui m'a été transmise ;

— Les **membres du jury**, pour le temps accordé à l'évaluation de ce travail, et pour les remarques constructives qui viendront enrichir mes réflexions futures ;

— Mes **Aînés académiques**, pour leurs encouragements et leur soutien dans les moments les plus exigeants ;

— Mes **parents**, **M. et Mme BIODOKINE**, pour leur amour inconditionnel et leurs prières silencieuses, qui m'ont donné la force d'avancer ;

— Mes **frères et sœurs**, **KASSI Simon**, **Rachelle**, **PIERRE Christian Atchom**, **Moise Victor**, **Marie De grace** et **TOBENG Onction**, pour leur affection et leurs mots d'encouragement ;

— Mes **camarades de promotion**, en particulier **TSAKENG TSAFACK Pavel J.**, **DJOU KENNE Bristol**, **KAMBA SAMGUE Durand** et **LEMOFOUET VALDINI Douglace**, pour les échanges constructifs, le soutien moral et les moments de partage qui ont rendu ce parcours plus humain ;

— **NEGUEM Prisca** pour son soutien inconditionnel.

Enfin, à tous ceux dont les noms n'apparaissent pas ici mais qui ont, d'une manière ou d'une autre, contribué à ce travail, je vous adresse toute ma gratitude. Votre aide, même silencieuse, m'a été précieuse.

Merci infiniment à chacun d'entre vous.

Table des matières

Dédicace ii
Remerciements iii
Table des matières iv
Liste des acronymes viii
Liste des tableaux ix
Liste des figures x
Résumé xii
Abstract xiii

INTRODUCTION GÉNÉRALE **1**
Contexte 1
Problématique 2
Objectifs 2
Contribution 3
Plan du document 3

Chapitre I ► **CADRE THÉORIQUE ET CONCEPTUEL** **5**
I.1 - Introduction 5
I.2 - Fondamentaux du génie logiciel 5
I.2.1 - Définition et rôle du génie logiciel 6
I.2.2 - Méthodes de développement logiciel 7
I.2.2.1 - Modèle en cascade 7
I.2.2.2 - Modèle en V 9
I.2.2.3 - Modèle incrémental 9
I.2.2.4 - Modèle en spirale 10
I.2.3 - Notion d'atelier de génie logiciel et d'environnement intégré 11
I.2.3.1 - Définition et principes 11
I.2.3.2 - Typologie des ateliers de génie logiciel 12
I.2.3.3 - Défis de la gestion des projets logiciels 13
I.3 - Concepts des assistants conversationnels 13
I.3.1 - Définition et fonctionnement 13

I.3.2 - Typologies des chatbots : chatbot à base de règles vs. basé sur IA . . . 15
I.3.2.1 - Chatbots basés sur des règles . . . 15
I.3.2.2 - Chatbots basés sur l'IA(Intelligence Artificielle) . . . 16
I.3.2.3 - Synthèse . . . 17
I.4 - Modèles de langage larges (LLMs) . . . 18
I.4.1 - Définition et caractéristiques . . . 18
I.4.2 - Fonctionnement et architecture des LLMs . . . 19
I.4.2.1 - L'encodeur . . . 21
I.4.2.2 - Le decodeur . . . 22
I.4.3 - Processus d'Apprentissage des LLMs . . . 24
I.4.3.1 - Le Pré-entraînement (Pre-training) . . . 24
I.4.3.2 - Le Fine tuning . . . 25
I.4.4 - Techniques d'adaptation et d'optimisation . . . 26
I.4.4.1 - Le prompt engineering . . . 26
I.4.4.2 - La génération augmentée par récupération (RAG) 27
I.4.5 - Contraintes et défis . . . 27
I.5 - Conclusion . . . 29

Chapitre II ▶ **REVUE DE LA LITTÉRATURE** **30**
II.1 - Introduction . . . 30
II.2 - Démarche de revue et Stratégie de recherche documentaire . . . 31
II.2.1 - Démarche de revue . . . 31
II.2.2 - Stratégie de recherche documentaire . . . 31
II.2.2.1 - Definition des Questions de recherche . . . 31
II.2.2.2 - Critères d'inclusion et d'exclusion . . . 33
II.2.2.3 - Moteurs de recherche utilisés . . . 34
II.3 - Analyse des travaux existants . . . 35
II.3.1 - Méthodologies actuelles de conception d'assistants . . . 35
II.3.2 - Plateformes et Outils existants pour le fine-tuning et la création d'assistants . . . 42
II.3.2.1 - Plateformes low-code / no-code : créer des chatbots sans coder . . . 42
II.3.2.2 - Outils existants pour le fine-tuning de LLMs . . . 43
II.3.3 - Intégration des LLMs dans ces méthodologies . . . 44
II.4 - Synthèse critique et positionnement de la recherche . . . 45
II.5 - Conclusion . . . 48

Chapitre III ▶ **ARCHITECTURE D'UN ATELIER DE CRÉATION D'ASSISTANTS CONVERSATIONNELS BASÉS SUR DES MODÈLES DE LANGAGE LARGES** **50**
III.1 - Introduction 50
III.2 - Objectifs de conception de l'atelier et vision systémique 51
III.2.1 - Public cible : Un atelier pour un concepteur conversationnel assisté 51
III.2.2 - Finalités fonctionnelles : faire du LLM un outil à portée de tous 52
III.2.3 - Contraintes non fonctionnelles : robustesse, modularité et évolutivité 53
III.2.4 - Démarche de conception adoptée 54
III.2.4.1 - Cycles itératifs et validation incrémentale . . . 54
III.3 - Architecture fonctionnelle proposée(Vue utilisateur) 54
III.3.1 - Vue d'ensemble du processus 55
III.3.2 - Description des Modules fonctionnels du workflow conversationnel 55
III.4 - L'architecture logicielle interne de l'atelier (Vue développeur) . 65
III.4.1 - Vue d'ensemble :une organisation en couches, claire et modulaire 65
III.4.2 - Composants logiciels fondamentaux 66
III.5 - Communication entre modules et persistance des données . . . 67
III.5.1 - Comment les modules communiquent entre eux 68
III.5.2 - Comment les données sont organisées et conservées . . . 68
III.6 - Validation théorique de l'architecture proposée 69
III.6.1 - Alignement entre exigences et composants 69
III.6.2 - Évaluation des critères de qualité logicielle 69
III.7 - Validation expérimentale simulée (preuve de concept) 70
III.7.1 - Présentation du cas d'usage choisi 70
III.7.2 - Parcours utilisateur dans l'atelier (via maquettes interactives) 71
III.7.3 - Résultats et analyse des tests 74
III.7.4 - Discussion générale 74
III.8 - Conclusion 76

Conclusion générale **77**
La problématique étudiée et les choix méthodologiques 77

Analyse et critique des résultats obtenus 77
Perspectives 78

Bibliographie **79**

Liste des acronymes

AES	Advanced Encryption Standard (Standard de chiffrement avancé)
AGL	Atelier de Génie Logiciel
API	Application Programming Interface (Interface de programmation applicative)
BERT	Bidirectional Encoder Representations from Transformers
CASE	Computer-Aided Software Engineering (Génie Logiciel Assisté par Ordinateur)
CI/CD	Continuous Integration / Continuous Deployment (Intégration et déploiement continus)
CTRL	Conditional Transformer Language Model
DSL	Domain Specific Language (Langage spécifique au domaine)
GPT	Generative Pre-trained Transformer (Transformeur génératif préentraîné)
GUI	Graphical User Interface (Interface Graphique Utilisateur)
GL	Génie Logiciel
IA	Intelligence Artificielle
IDE	Integrated Development Environment (Environnement de Développement Intégré)
JSON	JavaScript Object Notation
LLM	Large Language Model (Modèle de Langage de Grande Taille)
LoRA	Low-Rank Adaptation
MDD	Model-Driven Development (Développement dirigé par les modèles)
MLOps	Machine Learning Operations (Opérations pour le déploiement de modèles d'apprentissage)
NLG	Natural Language Generation (Génération de Langage Naturel)
NLP	Natural Language Processing (Traitement automatique du langage naturel)
NLU	Natural Language Understanding (Compréhension du langage naturel)
RAG	Retrieval-Augmented Generation (Génération augmentée par récupération)
REST	Representational State Transfer (Architecture de services web)
SDK	Software Development Kit (Kit de développement logiciel)
TALN	Traitement Automatique du Langage Naturel
T5	Text-To-Text Transfer Transformer (Transformeur de Transfert Texte-à-Texte)
UI	User Interface (Interface utilisateur)
YAML	Yet Another Markup Language

Liste des tableaux

I - Comparaison entre chatbots basés sur des règles et basés sur l'IA . 18

II - Questions de recherche et leurs motivations pour guider l'étude . . 32
III - Critères d'inclusion, d'exclusion et justifications 33
IV - Couverture des questions de recherche par les études analysées . . 46
V - Justifications des couvertures des questions de recherche par étude 47

Liste des figures

1 - Rapports au congrès américain sur les logiciels 6
2 - Place du génie logiciel dans le processus de développement d'un système [38] 7
3 - Architecture en cascade [33] 8
4 - Architecture en V [29] 9
5 - Architecture incrementale [22] 10
6 - Architecture en spirale [2] 10
7 - Architecture fonctionnelle d'un assistant conversationnel [12]. . . . 14
8 - Schéma typique d'un chatbot basé sur un script des règles, illustrant le parcours décisionnel conditionnel[37]. 15
9 - Diagramme de flux du processus de traitement d'un message par un Chatbot IA [31]. 17
10 - Architecture Transformer [40] 20
11 - L'encodeur 22
12 - Le decodeur 23
13 - Illustration des techniques de fine-tuning [30] 25
14 - Avantages du fine-tuning 26
15 - taxonomie des hallucinations dans la génération de code 28

16 - Processus de Sélection des Études pour la Revue Systématique . . 36
17 - Illustration du fonctionnement du MDD[36] 37
18 - Illustration des 03 Niveaux d'abstraction du MDD[26] 37
19 - Métamodèle Xatkit proposé pour la génération automatique de chatbots[1]. 39
20 - Présentation du framework Xatkit[6]. 40
21 - Architecture d'un cadre générique aux agents conversationnels [35]. 42
22 - Limites des approches existantes 48

23 - Workflow general de creation d'un assiststant conversationnel dans l'atelier 55
24 - Interface d'initialisation d'un nouveau projet : saisie des paramètres et validation en temps réel. 56

25 - Importation d'un fichier Swagger : téléversement et validation syntaxique. 58
26 - Configuration manuelle : saisie des endpoints et test de connexion. 58
27 - Interface de modélisation : saisie des intentions et validation des phrases. 60
28 - Interface d'expansion et de validation des données 61
29 - Interface de demarrage automatique du fine-tuning 62
30 - Interface de Choix du format du dataset et definition des hyper-paramètres 62
31 - Interface de test 63
32 - Prévisualisation de l'interface personnalisée avec choix de style et tonalité. 64
33 - Affichage du contenu du fichier API généré. 64
34 - Interface d'exportation : bouton d'export et sélection des versions. 65
35 - Vue en couches de l'architecture logicielle de l'atelier 66
36 - Schéma de communication entre les modules de l'atelier 68
37 - Page d'accueil 71
38 - Interface d'authentification 71
39 - Dashboard utilisateur 71
40 - Formulaire de creation d'un projet de chatbot 71
41 - Importation du fichier de spécification 72
42 - Definition des conversations 72
43 - Appercu du schema JSON genere 72
44 - Augmentation des données mappées à leurs actions 73
45 - Formatage du dataset d'entrainement 73
46 - Suivi de l'entrainement du modele 74
47 - Interface de test du modele et validation 74
48 - Interface d'affichage du Score de validation 74
49 - personnalisation et le déploiement du chatbot 75
50 - Fichier json produit 75
51 - Synthèse des points forts, limites et défis du workflow proposé . . 76

Résumé

Ces dernières années, les assistants conversationnels sont devenus des outils incontournables dans de nombreux secteurs. On les utilise pour répondre à des questions, accompagner des clients, ou même automatiser certaines tâches simples, les chatbots font désormais partie intégrante de nombreuses applications où ils interprètent les entrées en langage naturel et exécutent les actions correspondantes sur les systèmes backend. Ces assistants s'appuient de plus en plus sur des modèles de langage larges, capables de comprendre et de produire du texte en langage naturel.

Mais malgré leur potentiel, concevoir un assistant qui s'appuie sur ces modèles reste difficile. L'une des principales difficultés réside dans l'absence de stratégie standardisée pourla création de chatbots. Les développeurs se débrouillent souvent à la vavite, guidés par les capacités et les contraintes des modèles préentraînés existants, Le processus est souvent peu structuré et complexe à reproduire. Si ces modèles constituent une base solide, le caractère ad hoc de cette approche entraîne des incohérences, des inefficacités et l'omission de nombreuses bonnes pratiques en développement logiciel. Par conséquent, les chatbots créés selon ces méthodes peuvent manquer de robustesse, de maintenabilité et d'évolutivité, pourtant essentielles à un fonctionnement fiable dans des environnements complexes.

Ce travail propose une réponse à ce problème. Une architecture logicielle qui sert de base à un atelier complet pour guider, étape par étape, la création de ce type d'assistants. L'idée est de structurer tout le processus, depuis l'analyse des besoins jusqu'à l'intégration dans un système existant, en suivant une démarche inspirée des bonnes pratiques du génie logiciel. L'architecture proposée repose sur une approche modulaire. Elle permet de décomposer le processus en plusieurs composants réutilisables : collecte et préparation des données, spécialisation du modèle de langage (ou fine-tuning), des tests interactifs entre l'utilisateur et le chatbot via des dialogues, interfaçage avec des logiciels ou des services via des API, et supervision du comportement du modèle dans le temps.

Une expérimentation a été réalisée pour illustrer cette approche dans un cas concret : la conception d'un assistant capable de générer des factures à partir des commandes textuelles ou vocales. Cette étude a permis de valider la cohérence de l'architecture proposée et de montrer qu'il est possible de construire des assistants opérationnels en s'appuyant sur un cadre clair, sans nécessiter de compétences avancées en intelligence artificielle.

Ce travail contribue à poser les bases d'une nouvelle façon de concevoir des agents conversationnels : plus structurée, plus accessible, et mieux alignée sur les besoins des utilisateurs et des systèmes dans lesquels ils doivent s'intégrer.

Mots clés : modèles de langage larges, assistants conversationnels, génie logiciel, fine-tuning, architecture modulaire, traitement du langage naturel.

Abstract

In recent years, conversational agents have become essential tools across various sectors. They are used to answer questions, support users, or automate simple tasks. These chatbots are now integrated into many applications, where they interpret natural language inputs and execute corresponding actions on backend systems. Most of them rely on large language models (LLMs), which are capable of understanding and generating natural language text.

However, despite their potential, designing such assistants remains a challenging task. One of the main difficulties lies in the absence of a standardized strategy for chatbot development. Developers often rely on improvised solutions, guided more by the limits of pre-trained models than by structured methodologies. As a result, these processes are often unstructured, difficult to reproduce, and ignore key software engineering principles. This leads to assistants that may lack robustness, maintainability, and scalability—three essential qualities for any software operating in complex environments.

This thesis proposes a response to that problem : a software architecture that serves as the foundation for a complete development framework. The goal is to guide the creation of such assistants step by step, from requirement analysis to integration with existing systems, while following a structured approach based on established software engineering practices. The proposed architecture follows a modular design. It breaks down the process into several reusable components : data preparation, model fine-tuning, interactive testing via dialogue scenarios, connection to existing software systems through APIs, and monitoring of the assistant's behavior over time.

An experiment was conducted to demonstrate this approach in a practical case : the design of an assistant capable of generating invoices from textual or voice instructions. This study helped validate the coherence of the proposed architecture and showed that it is possible to develop operational assistants using a clear, guided process, without requiring advanced expertise in artificial intelligence.

This work lays the foundation for a new way of building conversational agents more structured, more accessible, and better aligned with the real needs of users and the systems in which they operate.

Key words : large language models, conversational assistants, software engineering, fine-tuning, modular architecture, natural language processing.

INTRODUCTION GÉNÉRALE

Sommaire

Contexte 1
Problématique 2
Objectifs 2
Contribution 3
Plan du document 3

Contexte

Depuis les débuts de l'informatique, les interactions entre l'humain et la machine ont connu de nombreuses transformations. Au départ, elles se faisaient uniquement par des lignes de commande. L'utilisateur devait taper des instructions précises, souvent difficiles à mémoriser. Avec le temps, les interfaces graphiques ont simplifié cette relation : il suffisait de cliquer sur des icônes ou de naviguer dans des menus. Cette évolution a permis une démocratisation de l'usage des outils informatiques, en les rendant plus visuels et intuitifs.

Une nouvelle étape a été franchie avec l'émergence des interfaces conversationnelles. Ces interfaces permettent aux utilisateurs de dialoguer avec un programme en langage naturel[24]. On parle ici d'agents conversationnels, parfois appelés assistants virtuels ou chatbots. Les premières générations de ces agents étaient généralement construites à partir de règles fixes, sous forme d'arbres de décision. Elles fonctionnaient correctement dans des cas simples, mais manquaient de souplesse pour des demandes complexes ou imprévues [16].

L'arrivée des techniques de traitement automatique du langage naturel (TALN), notamment avec l'apprentissage automatique, a permis de dépasser ces limites. Plus récemment, l'introduction des modèles de langage larges (Large Language Models, ou LLMs), comme BERT[8], GPT-3[5] ou T5[30], a marqué un tournant. Ces modèles sont préentraînés sur de grands volumes de textes. Ils peuvent interpréter des questions complexes, produire des réponses cohérentes, résumer des documents, ou encore générer du code source. Ils sont désormais intégrés dans des assistants capables d'interagir avec d'autres logiciels ou systèmes informatiques, par exemple via des interfaces de programmation (APIs). Cette capacité à comprendre et agir ouvre la voie à des applications concrètes dans des domaines comme la santé, l'administration, l'éducation ou le support client [43].

Cependant, même si les modèles de langage larges sont puissants, les démarches pour créer des assistants conversationnels[34] restent complexes. Le processus implique plusieurs étapes techniques : collecte des données, spécialisation du modèle (fine-tuning), gestion des dialogues, validation métier, et intégration logicielle[39].

Ce constat met en évidence un besoin réel : celui de concevoir une architecture claire et structuré, qui accompagne les utilisateurs dans la création d'assistants conversationnels personnalisés, tout en facilitant leur intégration dans des systèmes existants.

Problématique

Même si les modèles de langage larges permettent aujourd'hui de créer des agents conversationnels plus puissants et plus flexibles, il n'existe pas encore de cadre logiciel structuré pour guider efficacement leur mise en œuvre dans des contextes métiers.

Les outils existants sont souvent fragmentés, techniques, et peu adaptés à un usage collaboratif entre développeurs et experts métier[37]. Il est difficile d'adapter un modèle, de l'intégrer dans un système existant, ou d'en assurer la maintenance dans la durée.

Dès lors, une question centrale se pose : **Comment concevoir une architecture logicielle modulaire et accessible, qui structure toutes les étapes de création d'un assistant conversationnel fondé sur un modèle de langage large, tout en facilitant son intégration dans un système logiciel existant ?**

Objectifs

Ce travail porte sur la conception d'un atelier logiciel qui permet de créer des assistants conversationnels capables de comprendre des demandes formulées en langage naturel et d'agir dans un système informatique.

L'objectif principal est donc de **proposer une architecture claire et bien structurée**, qui guide les utilisateurs dans chaque étape de création d'un assistant, depuis l'analyse du besoin jusqu'à l'intégration technique. Pour atteindre cet objectif général, le travail s'appuie sur plusieurs objectifs spécifiques :

- **Analyser les travaux existants** (approche, outils, frameworks, plateformes de developpement des chatbots) et identifier ce qui manque pour en faire un cadre complet et réutilisable.
- **Analyser les besoins fonctionnels et non fonctionnels** liés à la création d'assistants conversationnels dans différents contextes d'usage.
- **Proposer une architecture logicielle claire et modulaire**, où chaque partie de l'atelier remplit une fonction bien précise : définition du besoin, traitement des données, adaptation du modèle, test des dialogues, etc.

— **Proposer un processus de travail simple et progressif**, inspiré des bonnes pratiques du génie logiciel, pour aider les utilisateurs à avancer étape par étape.
— **Tester l'approche à travers un exemple concret** afin de voir si l'atelier fonctionne bien et s'il peut vraiment aider à construire un assistant utile et cohérent.

Contribution

Ce travail propose une architecture logicielle conçue pour encadrer la création d'assistants conversationnels fondés sur des modèles de langage larges capable de piloter des systèmes existants. Cette architecture sert de base à un atelier logiciel qui accompagne les utilisateurs à chaque étape de conception, en suivant une démarche rigoureuse du genie logiciel.

Les contributions principales sont les suivantes :

— La conception d'une **architecture modulaire**, composée de blocs simples correspondant aux grandes étapes du développement : définition du besoin, préparation des données, spécialisation du modèle, test, intégration.
— La proposition d'un **processus de création progressif**, inspiré du génie logiciel, qui aide les utilisateurs à structurer leur travail, même sans expertise avancée en intelligence artificielle.
— L'adaptation de cette architecture à différents outils et fournisseurs de modèles, afin de garantir une certaine flexibilité dans son utilisation.
— L'illustration de l'approche par une étude de cas, pour montrer de manière concrète comment l'atelier peut être utilisé dans un projet réel.

Plan du document

Le présent mémoire est organisé en trois chapitres structurés autour de la problématique exposée.

Le **chapitre I intitulé Cadre théorique et conceptuel** développe le socle théorique et terminologique du sujet. Il présente les fondements du génie logiciel, les principes des architectures conversationnelles, et les caractéristiques des modèles de langage de grande taille.

Le **chapitre II intitulé Revue de littérature** dresse un *état de l'art systématique* des approches existantes. Il examine les approches, plateformes et outils liés à la création de chatbots, en mettant en lumière leurs limites méthodologiques, techniques et pratiques.

Le **chapitre III intitulé Architecture d'un atelier de création d'assistants conversationnels basés sur des modèles de langage larges** expose en détail la proposition architecturale de l'atelier. Il décrit le workflow utilisateur, modélise les modules logiciels, présente l'organisation interne du système et les choix

techniques opérés. Cette architecture est ensuite validée à travers une étude de cas réaliste : la conception d'un assistant conversationnel pour la gestion de facturation. Cette expérimentation permet d'évaluer l'efficacité de l'atelier, d'en identifier les limites actuelles et de dégager des pistes d'évolution.

Enfin, une **conclusion générale** synthétise les résultats obtenus et ouvre des perspectives pour l'industrialisation et la généralisation de la solution proposée.

Chapitre I

CADRE THÉORIQUE ET CONCEPTUEL

Sommaire

I.1 - Introduction 5
I.2 - Fondamentaux du génie logiciel 5
I.3 - Concepts des assistants conversationnels 13
I.4 - Modèles de langage larges (LLMs) 18
I.5 - Conclusion 29

I.1. Introduction

La conception d'assistants conversationnels intelligents repose sur des notions clés relevant à la fois du génie logiciel, de l'intelligence artificielle et du traitement du langage naturel. Pour bien situer notre travail, Ce chapitre s'intéresse donc à poser les fondations théoriques de ces trois piliers.

Nous commencerons par poser les bases du génie logiciel, en rappelant son rôle, ses méthodes classiques de développement, et en introduisant la notion d'atelier logiciel. Ensuite, nous expliciterons ce qu'est un assistant conversationnel, leur fonctionnement, leurs typologies, leurs usages, ainsi que leur architecture fonctionnelle générale. Enfin, nous décrirons les spécificités des modèles de langage de grande taille (LLMs), qui constituent le cœur des nouveaux assistants intelligents, en présentant leurs mécanismes d'apprentissage, leurs modalités de personnalisation, ainsi que les contraintes qui leur sont associées.

I.2. Fondamentaux du génie logiciel

I.2.1. Définition et rôle du génie logiciel

Le génie logiciel (« Software Engineering » en anglais), est une discipline de l'ingénierie informatique qui vise à encadrer de manière rigoureuse la conception, la réalisation et la maintenance des systèmes logiciels. Ce domaine est né dans les années 1970, à un moment où de nombreux projets informatiques prenaient du retard, coûtaient trop cher ou ne fonctionnaient tout simplement pas. C'est ce qu'on a appelé la "crise du logiciel".

À cette époque, les développeurs avaient de plus en plus de mal à maîtriser des systèmes devenus trop complexes. Plusieurs projets importants ont échoué, parfois après plusieurs années de travail et des millions investis. Par exemple, le projet TAURUS pour la Bourse de Londres a été abandonné après quatre ans de développement et une perte de 100 millions de livres. La fusée Ariane 5 a explosé en 1996 à cause d'une erreur logicielle. Au Cameroun, des erreurs dans les logiciels ont provoqué des retards de salaires ou des problèmes dans le calcul des moyennes universitaires. En 2000, on estime que les pannes logicielles ont coûté environ 175 milliards de dollars aux entreprises à travers le monde.

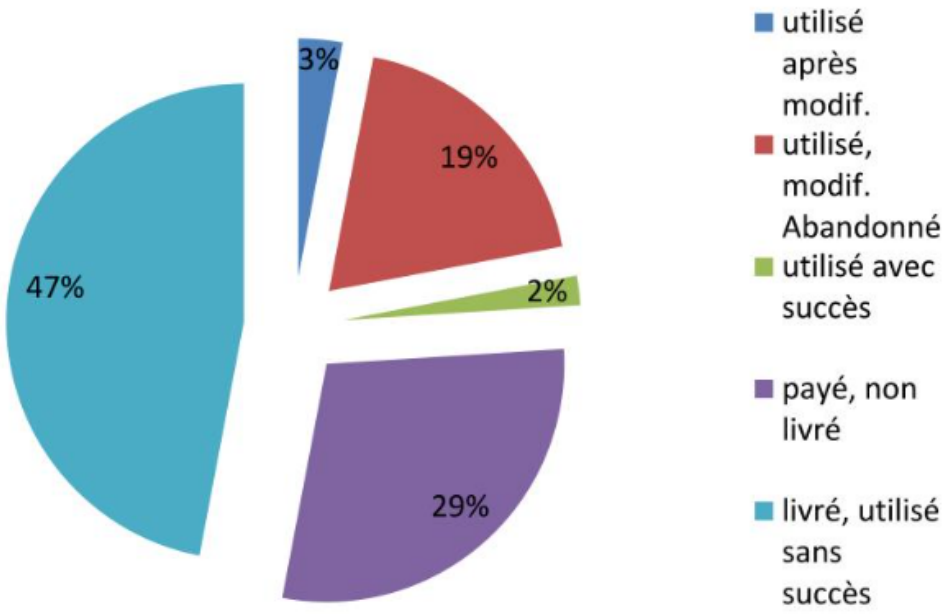


Figure 1 – *Rapports au congrès américain sur les logiciels*

Pour répondre à ces difficultés, on a commencé à aborder le développement logiciel comme une activité d'ingénierie à part entière, avec ses méthodes, ses étapes et ses outils. L'idée était de rendre la création de logiciels plus fiable, plus prévisible et plus structurée.

Le but du génie logiciel est donc de produire des logiciels qui fonctionnent bien, qui répondent aux attentes, et qui peuvent évoluer dans le temps. Pour cela, il s'appuie sur ce qu'on appelle souvent le carré "CQFD" : Coût, Qualité, Fonctionnalités, Délai. Un bon logiciel, c'est un logiciel qui fait ce qu'il doit faire, dans les temps, sans dépasser le budget, et avec une qualité suffisante pour durer.

Concrètement, le génie logiciel utilise des modèles pour représenter les systèmes, des langages pour décrire ce qu'il faut faire, une démarche claire pour organiser les étapes de développement, et des outils pour automatiser ou vérifier certaines tâches. Tous ces éléments sont organisés dans des méthodes, qui aident les équipes à construire un logiciel étape par étape, de manière plus sûre.

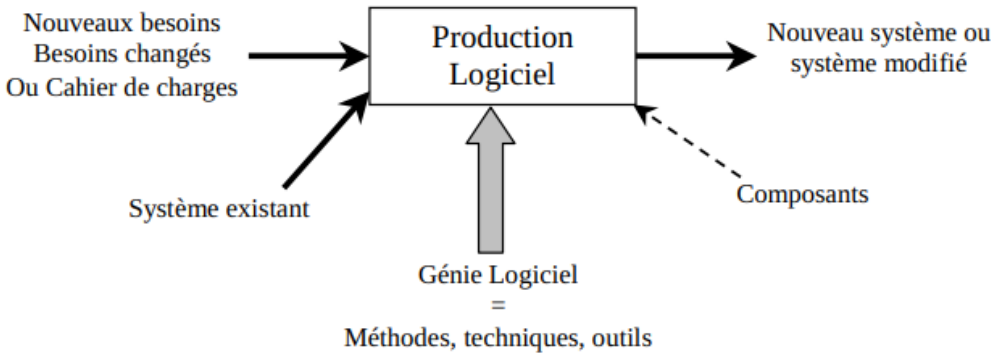


Figure 2 – *Place du génie logiciel dans le processus de développement d'un système [38]*

I.2.2. Méthodes de développement logiciel

Les méthodes de développement logiciel définissent les étapes à suivre pour construire un logiciel de manière structurée et maîtrisée. Elles précisent comment passer de l'expression des besoins à un produit fonctionnel, tout en encadrant les phases de conception, d'implémentation, de test et de maintenance. Ces méthodes permettent de réduire les risques d'échec, de contrôler la qualité et d'adapter le processus à la nature du projet.

I.2.2.1. Modèle en cascade

Le modèle en cascade (ou waterfall model) est l'un des premiers modèles formels proposés initialement par Winston W. Royce en 1970, qui repose sur une séquence rigide d'étapes allant de l'analyse des besoins jusqu'à la maintenance, sans retour en arrière formel. Ce modèle a longtemps été considéré comme la norme dans les projets logiciels industriels [33].

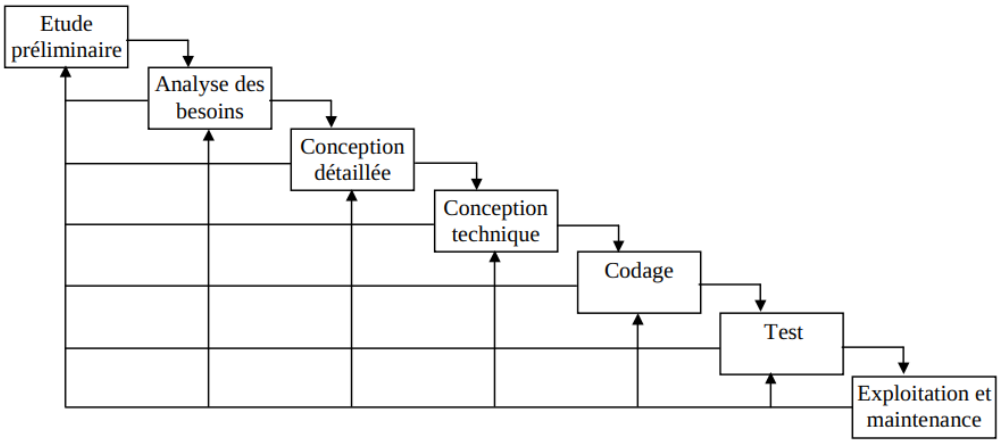


Figure 3 – *Architecture en cascade [33]*

Dans ce modèle, le processus est découpé en sept grandes étapes qui doivent être exécutées les unes après les autres, sans retour arrière formel :

1. **Étude préliminaire :** Cette première étape permet de déterminer si le développement d'un nouveau logiciel est nécessaire. On évalue la stratégie de l'entreprise et on décide s'il faut créer un produit sur mesure ou adopter un existant.
2. **Analyse des besoins :** L'objectif ici est d'identifier avec précision ce que l'utilisateur attend du système. On collecte les exigences fonctionnelles et non fonctionnelles pour produire un cahier des charges détaillé.
3. **Conception détaillée :** À partir des besoins recueillis, on organise les fonctionnalités dans des modules logiques. Chaque module est décrit techniquement afin de guider l'étape de codage à venir.
4. **Conception technique :** Cette phase consiste à choisir les outils, les technologies et l'environnement matériel ou logiciel nécessaires pour le développement du système.
5. **Codage :** Les développeurs écrivent le code source de chaque module.
6. **Tests d'intégration :** Une fois tous les modules codés, ils sont ensuite testés de manière isolée à travers des tests unitaires puis sont intégrés et testés ensemble. L'objectif est de vérifier que le système global répond bien aux exigences définies au début du projet.
7. **Exploitation et maintenance :** Une fois le logiciel livré, il est mis en service et maintenu dans le temps. Cette maintenance peut être corrective (en cas de bug), évolutive (ajout de fonctionnalités), ou adaptative (mise à jour de l'environnement technique).

Ce modèle présente l'avantage d'être simple à comprendre et à gérer, surtout pour des projets bien définis dès le départ. Toutefois, il souffre de plusieurs limites. Son principal inconvénient est son manque de flexibilité : une fois une phase validée, il est difficile de revenir en arrière. Par ailleurs, le client ne voit le produit qu'à la toute fin, ce qui peut générer des écarts importants entre ses attentes et le résultat obtenu.

Malgré cela, le modèle en cascade reste pertinent dans des contextes très formalisés, comme les systèmes critiques (défense, santé, aéronautique), où chaque étape doit être rigoureusement tracée et validée avant de passer à la suivante.

I.2.2.2. Modèle en V

Il est considéré comme une amélioration du modèle en cascade, auquel il ajoute une dimension forte de validation. Son nom vient de la représentation en forme de "V" où chaque phase de développement, située sur la branche descendante, correspond à une phase de test ou de validation, située sur la branche montante.

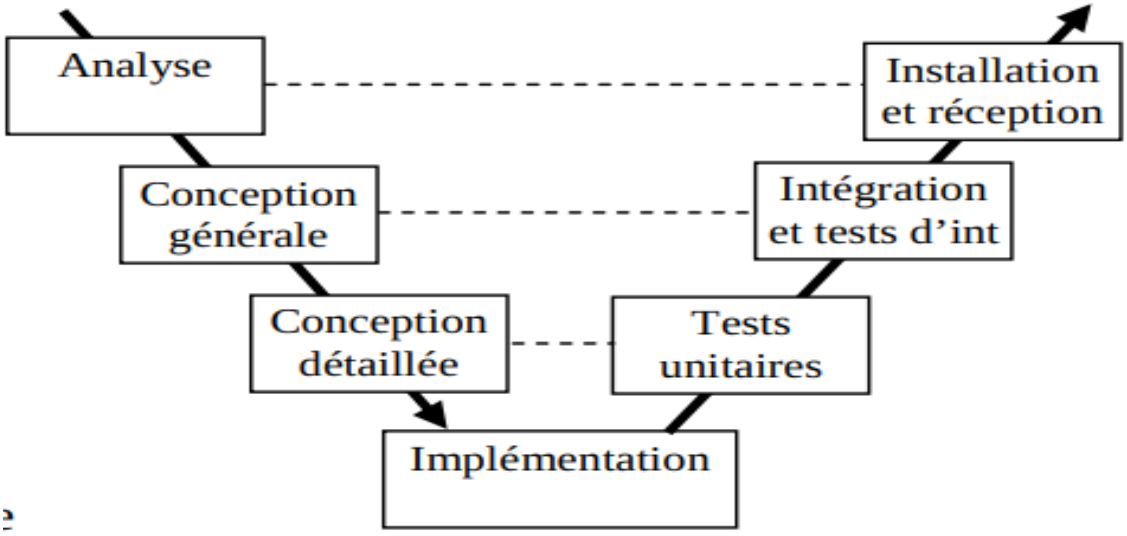


Figure 4 – *Architecture en V [29]*

Ce modèle convient surtout aux projets où l'on connaît très bien les besoins dès le départ, et où les erreurs sont difficiles à accepter, comme dans l'aéronautique, la santé ou les banques. Son principal avantage est qu'il permet de bien suivre chaque exigence du début à la fin. En effet, à chaque besoin défini au départ correspond un test prévu plus tard, ce qui permet une meilleure traçabilité entre les exigences initiales et la validation du système final [29]. En revanche, ce modèle est assez rigide. Il s'adapte mal aux projets où les besoins peuvent changer en cours de route.

I.2.2.3. Modèle incrémental

Le modèle incrémental repose sur une idée simple mais puissante : au lieu de livrer l'ensemble du logiciel en une seule fois, comme dans le modèle en cascade, on le construit progressivement, par blocs fonctionnels appelés incréments. Chaque incrément est une version partielle mais exécutable du logiciel, qui vient enrichir la version précédente par de nouvelles fonctionnalités. Cette approche permet de livrer rapidement des résultats exploitables tout en améliorant progressivement le produit [22].

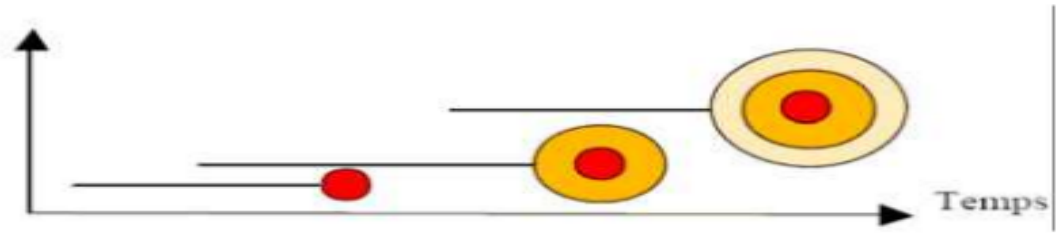


Figure 5 – *Architecture incrementale [22]*

Ce fonctionnement en couches successives permet une meilleure maîtrise du projet, car chaque incrément est une étape clairement définie. Cela facilite les tests, les retours utilisateurs et les ajustements en cours de développement. Par exemple, si une fonctionnalité posait problème, elle peut être retravaillée sans remettre en cause tout le système. Il est particulièrement adapté aux projets évolutifs ou innovants, où l'on ne connaît pas à l'avance tous les besoins avec précision. Il permet aussi de livrer rapidement une première version utilisable, même si elle n'est pas complète.

I.2.2.4. Modèle en spirale

Le modèle en spirale a été introduit par Barry Boehm en 1986 comme une réponse aux limites des approches séquentielles classiques, notamment dans les projets où les exigences sont complexes ou incomplètes dès le départ. Contrairement aux modèles linéaires comme la cascade, la spirale repose sur un enchaînement itératif de cycles, chacun permettant de définir les objectifs, analyser les risques, concevoir une solution, et l'évaluer avant de passer à la phase suivante. Chaque boucle de la spirale représente une évolution incrémentale du système. Ce qui distingue ce modèle, c'est l'accent mis sur la gestion proactive des risques, qui est intégrée comme étape centrale du processus. À chaque itération, les risques potentiels (techniques, humains, financiers, etc.) sont identifiés, analysés, puis atténués par des stratégies concrètes. Cela permet de réduire les échecs liés à des choix trop précoces ou mal évalués.

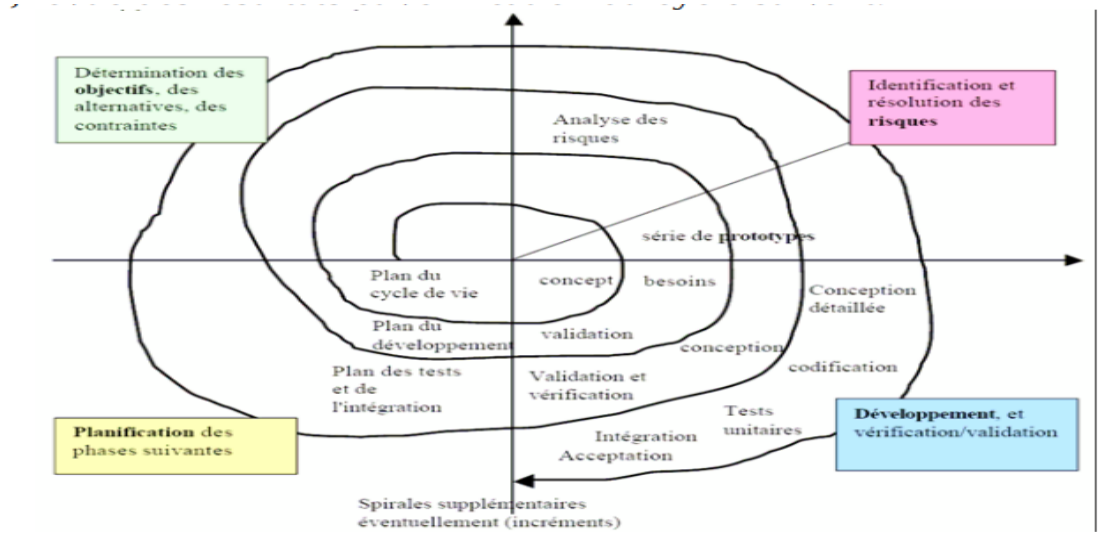


Figure 6 – *Architecture en spirale [2]*

Il est souvent recommandé pour les projets de recherche, les systèmes critiques ou les logiciels innovants, car il permet d'évoluer de façon maîtrisée vers une solution robuste, même quand toutes les informations ne sont pas connues à l'avance. Comme l'explique Boehm lui-même, cette approche « combine la nature itérative du prototypage avec les aspects contrôlés des modèles en cascade » [2].

I.2.3. Notion d'atelier de génie logiciel et d'environnement intégré

I.2.3.1. Définition et principes

Un atelier de génie logiciel, souvent appelé AGL, est un outil informatique qui aide les développeurs à construire des logiciels plus facilement, plus rapidement et avec plus de rigueur. On parle aussi parfois d'outils CASE (Computer Aided Software Engineering), car ils permettent d'automatiser certaines tâches du processus de développement, comme l'écriture du code, la génération de diagrammes, ou encore la vérification de la cohérence du système.

L'idée principale derrière un AGL est de ne pas repartir de zéro à chaque projet. L'atelier regroupe différents outils qui peuvent être utilisés ensemble : des éditeurs de code, des assistants à la modélisation, des outils de test, ou encore des systèmes de gestion de versions. En combinant tout cela, il devient possible de concevoir un logiciel de manière plus structurée, un peu comme dans une usine, où chaque étape de production est encadrée et assistée. Ces outils sont particulièrement utiles dans les projets complexes, où il est nécessaire de collaborer à plusieurs, de garder une trace des modifications, et d'assurer que le logiciel final réponde bien aux attentes. Comme le rappelle [42], un AGL désigne un ensemble de programmes conçus pour « produire des logiciels de manière industrielle ». Cela signifie que l'objectif est de rendre le développement plus professionnel, plus prévisible et plus fiable, même dans des environnements techniques exigeants.

Toujours dans la meme logique, on peut dire que les objectifs d'un atelier de génie logiciel sont multiples, mais tournent tous autour de trois idées centrales : faciliter, encadrer et améliorer la manière dont les logiciels sont conçus

- **Respect des normes et des exigences utilisateurs** : L'AGL aide à structurer le développement pour qu'il respecte les bonnes pratiques du domaine, les contraintes de qualité logicielle et les attentes fonctionnelles exprimées par les utilisateurs.
- **Amélioration de la collaboration** : Grâce à une centralisation des outils et des données du projet, l'AGL facilite le travail en équipe, la traçabilité des tâches, le partage de documents et la cohérence entre les différents intervenants.
- **Garantie de la qualité du produit** : L'automatisation des tests, des vérifications de conformité et des règles de codage contribue à produire des logiciels plus fiables, plus maintenables, et moins sujets à erreurs.
- **Optimisation du temps et des ressources** : En automatisant certaines étapes répétitives (génération de code, création de tests, documentation), l'AGL

permet de réduire les délais de livraison et d'utiliser les ressources humaines et techniques de façon plus efficace.

I.2.3.2. Typologie des ateliers de génie logiciel

Il existe plusieurs types d'ateliers de génie logiciel, chacun étant conçu pour répondre à des besoins spécifiques dans le cycle de développement d'un logiciel. Ces AGL sont souvent regroupés en grandes catégories selon la phase du projet qu'ils ciblent, allant de la conception à la maintenance.

Ateliers de conception (Design Workbenches) : Ils sont utilisés dans les premières étapes d'un projet pour modéliser l'architecture du système, définir les cas d'usage, les entités, et leurs relations. Ces outils permettent de représenter graphiquement le fonctionnement du logiciel avant même qu'une ligne de code ne soit écrite. Un exemple courant est l'utilisation d'UML (Unified Modeling Language), souvent supporté par des outils CASE(Computer-Aided Software Engineering). comme StarUML ou Enterprise Architect. Ces ateliers facilitent la communication entre les membres de l'équipe, en fournissant une base commune pour discuter des choix techniques dès les phases de planification.

Viennent ensuite les **Ateliers de développement (Development Workbenches)** : qui sont sans doute les plus connus. Ils regroupent les environnements de développement intégrés (IDE), comme Visual Studio, IntelliJ IDEA ou Eclipse. Ces plateformes offrent un espace centralisé pour rédiger, compiler, déboguer et organiser le code source. En plus de l'édition de code, ils intègrent souvent des fonctionnalités de complétion automatique, de gestion des dépendances, ou d'analyse statique, ce qui aide les développeurs à gagner en productivité et à limiter les erreurs.

Une autre catégorie importante concerne les **Ateliers de test (Testing Workbenches)** : Ces outils sont utilisés pour valider le bon fonctionnement du logiciel, détecter les bogues, et s'assurer que les résultats produits sont conformes aux attentes. Ils couvrent aussi bien les tests unitaires que les tests d'intégration, ou même les tests automatisés d'interfaces. Des outils comme JUnit (pour les tests Java), Selenium (pour les interfaces web) ou Postman (pour les API) sont aujourd'hui largement utilisés dans les projets modernes pour renforcer la fiabilité du logiciel avant sa mise en production.

Enfin, on trouve les **Ateliers de gestion de configuration (Configuration Management Workbenches)** qui interviennent dans le suivi de l'évolution du logiciel. Ils permettent de gérer les versions du code, de suivre les modifications, et de coordonner les contributions de plusieurs développeurs. Ces outils jouent un rôle clé dans les projets collaboratifs ou distribués. Git est aujourd'hui la solution de référence pour le versionnement, souvent accompagnée d'interfaces comme GitHub ou GitLab. D'autres outils comme Jenkins permettent d'automatiser certaines tâches liées à l'intégration continue et au déploiement.

Dans un atelier complet, ces différents types d'outils sont souvent combinés ou interconnectés pour offrir un environnement cohérent. Chacun joue un rôle précis dans la chaîne de production logicielle, et leur bonne articulation est essentielle pour assurer la fluidité, la qualité et la traçabilité du projet.

I.2.3.3. Défis de la gestion des projets logiciels

Malgré l'évolution des méthodes et des outils, la gestion de projets logiciels reste une activité complexe, exposée à de nombreux risques. Un premier défi réside dans la définition des besoins, souvent incomplète ou instable. Il n'est pas rare que les attentes des utilisateurs évoluent en cours de projet, ce qui rend difficile une planification rigide dès le départ. Cette incertitude initiale peut compromettre l'adéquation entre le produit final et les besoins réels, surtout dans les approches linéaires comme le cycle en V ou le modèle en cascade.

Un autre défi important est lié à la communication entre les parties prenantes. Développeurs, clients, utilisateurs finaux, gestionnaires de projet... tous n'ont pas les mêmes langages ni les mêmes priorités. Le risque de mauvaise interprétation est élevé, notamment lors des phases de spécification ou de validation. C'est pourquoi les méthodes agiles ont gagné en popularité : elles encouragent un dialogue plus régulier et plus itératif entre les acteurs du projet.

La gestion du temps et des ressources est également un point critique. Entre les estimations optimistes, les imprévus techniques, ou les changements de dernière minute, il devient difficile de tenir les délais et les budgets initiaux. Selon le rapport CHAOS du Standish Group[10], près de 30% des projets logiciels sont abandonnés avant leur terme, et la majorité dépasse les estimations de départ

Enfin, il existe un défi technique autour de la complexité croissante des systèmes logiciels. Avec l'intégration de composants externes, de modèles d'intelligence artificielle, de services cloud et d'API tierces, le développement logiciel moderne exige une grande capacité d'intégration, de test et de validation. Les erreurs peuvent apparaître à l'intersection de modules ou dans des situations peu anticipées, rendant les phases de test de plus en plus stratégiques.

Ces différents défis expliquent pourquoi le génie logiciel continue d'évoluer, en proposant des méthodes plus souples, des outils mieux intégrés, et des environnements comme les ateliers logiciels, capables de structurer l'ensemble du processus de manière progressive et maîtrisée.

I.3. Concepts des assistants conversationnels

I.3.1. Définition et fonctionnement

Les assistants conversationnels, ou chatbots, sont des systèmes logiciels capables de simuler une interaction en langage naturel avec un utilisateur humain. Leur fonction principale est de recevoir des requêtes textuelles ou vocales, de les analyser, puis de fournir des réponses pertinentes ou d'exécuter des actions spécifiques, de manière autonome et souvent en temps réel.

Selon Huang et al. (2020)[12], un assistant conversationnel est défini comme « un programme informatique capable d'interagir avec un utilisateur humain via un dialogue en langage naturel, souvent basé sur des techniques de traitement automatique du langage (NLP) et d'apprentissage automatique » .

Le fonctionnement général d'un assistant conversationnel repose sur une architecture modulaire articulée autour des composantes suivantes :

- **Compréhension du langage naturel (NLU)** : Elle permet au chatbot d'identifier les intentions de l'utilisateur et d'en extraire les informations importantes, comme des dates, des noms ou des quantités..
- **Gestion du dialogue (Dialogue Management)** : C'est-à-dire la capacité à suivre le fil d'une conversation. Un chatbot doit être capable de garder en mémoire le contexte d'un échange, de savoir ce qui a déjà été dit, et d'enchaîner correctement les réponses.Néamoins, cette gestion du contexte est cruciale dans les interactions complexes, comme celles où l'utilisateur fournit les informations étape par étape.
- **Génération de la réponse (NLG)** : Cela peut se faire soit à partir de modèles prédéfinis (réponses fixes), soit de façon plus dynamique, par exemple en construisant une commande structurée, une phrase personnalisée ou en interrogeant une base de données ou une API.
- **Exécution d'actions et accès aux données** : Dans certains cas, les agents conversationnels doivent aussi être capables d'interagir avec d'autres systèmes : base de données, logiciels internes, services web ou API. C'est ce qui leur permet, par exemple, de créer une commande, de creer une facture, mettre à jour un profil utilisateur ou déclencher un processus métier à la demande.

La Figure 21 présente une vue fonctionnelle standard du processus.

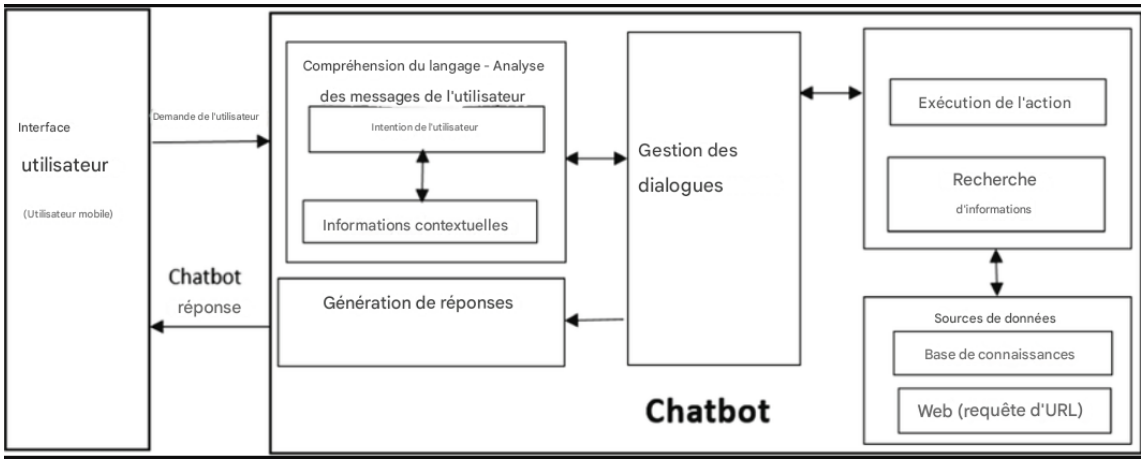


Figure 7 – *Architecture fonctionnelle d'un assistant conversationnel [12].*

Cette illustration met en évidence l'enchaînement logique des composants : une interface utilisateur capte la requête, qui est ensuite analysée pour en extraire les intentions et le contexte. La gestion du dialogue pilote le processus décisionnel en coordination avec un moteur d'exécution et un module d'accès aux sources de données (base de connaissances ou services web). Enfin, la réponse générée est renvoyée à l'utilisateur via l'interface initiale. Cette structuration garantit une interaction cohérente et pertinente entre le système et l'utilisateur.

I.3.2. Typologies des chatbots : chatbot à base de règles vs. basé sur IA

Les assistants conversationnels peuvent être classés en deux grandes catégories selon leur mode de fonctionnement : les chatbots basés sur des règles et les chatbots basés sur l'intelligence artificielle (IA).

I.3.2.1. Chatbots basés sur des règles

Également appelés « rule-based chatbots », fonctionnent selon des logiques conditionnelles explicites codées à l'avance. Leur comportement repose sur des arbres de décision, des structures de type `if-then` ou encore des expressions régulières associées à des scénarios bien precis. L'agent ne peut réagir qu'à des entrées textuelles précises, correspondant à des modèles définis manuellement par les concepteurs. Ce type de chatbot est souvent utilisé dans des contextes où les échanges sont courts, répétitifs, et où les intentions des utilisateurs sont fortement prévisibles. Par exemple, un chatbot conçu pour répondre aux questions fréquentes dans une banque (horaires, localisation des agences, taux d'intérêt) peut être entièrement basé sur un système de règles. Cela garantit une exécution rapide et un coût de développement relativement faible.

Toutefois, cette approche présente des limites importantes. La principale contrainte réside dans son manque de flexibilité : toute nouvelle intention utilisateur, toute variation linguistique non prévue, ou toute sortie de contexte risque de produire une réponse inappropriée ou une absence de réponse. En d'autres termes, le système ne « comprend » pas réellement le langage, mais agit comme un automate piloté par un script. Le maintien d'un chatbot à règles devient complexe au fur et à mesure que le nombre de scénarios augmente. Il nécessite une mise à jour manuelle régulière de ses structures, ce qui le rend peu adapté aux domaines évolutifs ou aux interactions nécessitant une interprétation plus libre du langage.

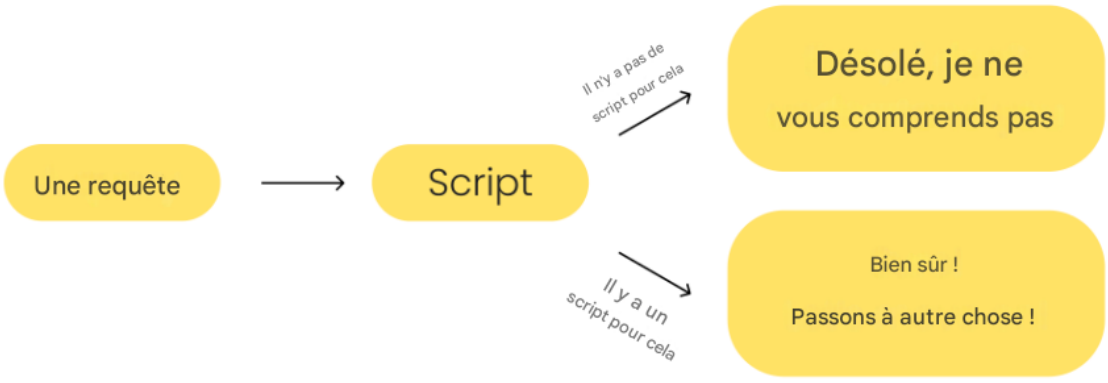


Figure 8 – *Schéma typique d'un chatbot basé sur un script des règles, illustrant le parcours décisionnel conditionnel[37].*

Ce type de chatbot peut être assimilé à un système expert simplifié, limité à la résolution de tâches bien définies. S'il reste pertinent pour des applications restreintes

et à faible complexité linguistique, il ne permet pas de modéliser des interactions plus ouvertes ou des contextes dynamiques.

I.3.2.2. Chatbots basés sur l'IA(Intelligence Artificielle)

Les chatbots basés sur l'intelligence artificielle, parfois appelés « data-driven chatbots » ou « NLP-based chatbots », s'appuient sur des modèles d'apprentissage automatique pour comprendre et générer du langage. Contrairement aux systèmes à règles, leur logique de réponse n'est pas écrite manuellement, mais apprise à partir de données. Ces chatbots sont généralement entraînés sur de grands corpus de dialogues, de textes ou d'intentions annotées, leur permettant de reconnaître des intentions variées, même formulées de manière inattendue.

Au cœur de cette approche, on retrouve des techniques de traitement automatique du langage naturel (TALN), telles que la classification d'intentions, l'extraction d'entités, ou plus récemment l'usage de modèles de langage pré-entraînés comme GPT, BERT ou T5. Ces modèles permettent au chatbot de produire des réponses plus naturelles, plus contextuelles, et souvent plus adaptées à l'intention réelle de l'utilisateur, même lorsque celle-ci est exprimée de manière ambiguë ou incomplète. L'un des principaux atouts de cette approche est sa capacité d'adaptation. Un chatbot intelligent n'a pas besoin que chaque phrase possible soit prévue à l'avance. Il peut généraliser à partir d'exemples et gérer des variantes linguistiques. Cela le rend particulièrement utile dans des contextes dynamiques, où les questions des utilisateurs peuvent varier en forme, en vocabulaire ou en ton.

Cependant, cette intelligence a un coût. D'abord, la création d'un chatbot basé sur l'IA nécessite un jeu de données d'entraînement pertinent, suffisamment riche pour représenter les cas d'usage visés. Ensuite, ces systèmes sont souvent perçus comme des boîtes noires, car leur raisonnement n'est pas toujours explicable de manière directe. Enfin, ils peuvent parfois générer des réponses incohérentes ou non souhaitées si les modèles ne sont pas bien ajustés ou si le contexte est mal interprété. Ce type de chatbot est donc plus adapté aux environnements complexes, où l'on cherche une plus grande souplesse d'interaction, une capacité à comprendre des formulations variées, et une personnalisation plus fine des échanges. Ils sont aujourd'hui largement utilisés dans les services clients automatisés, les assistants vocaux, ou encore les systèmes de recommandation intelligents.

Le diagramme ci-dessus illustre le cycle de vie d'une interaction utilisateur avec un chatbot basé sur l'IA. Chaque message de l'utilisateur est d'abord traité par deux composants clés de la Compréhension du Langage Naturel (NLU) :

`Classification d'Intention (Intent Classification)` : Ce module détermine l'objectif principal de la requête de l'utilisateur. Par exemple, pour la phrase "Je veux prendre un rendez-vous demain", l'intention identifiée pourrait être "PrendreRendezVous".

`Reconnaissance d'Entités (Entity Recognition)` : Simultanément, ce composant extrait les informations spécifiques et structurables du message. Dans l'exemple précédent, "demain" serait identifié comme une entité de type "Date". Ces informations, combinées au `Contexte` de la conversation (qui conserve la mémoire des échanges précédents), sont transmises au `Générateur de Réponses Candidates (Candidate Response Generator)`. Ce dernier accède à une base de `Réponses`

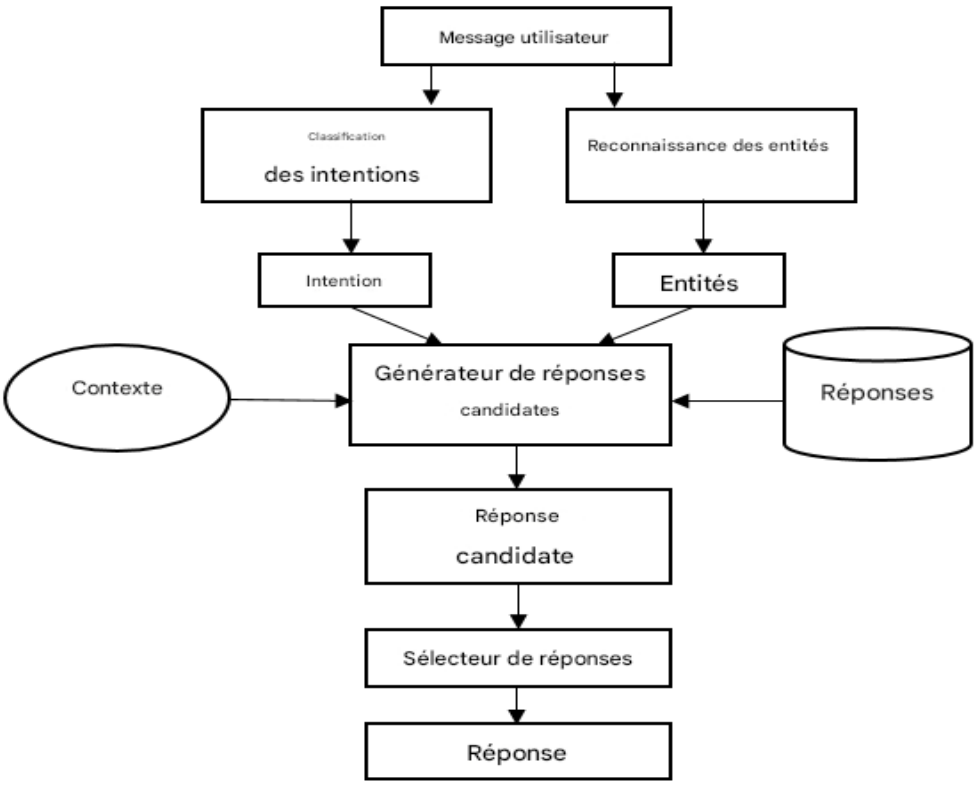


Figure 9 – *Diagramme de flux du processus de traitement d'un message par un Chatbot IA [31].*

(pouvant être des modèles, des données ou des logiques métier) pour formuler une ou plusieurs propositions de répliques. Enfin, le `Sélecteur de Réponse (Response Selector)` choisit la meilleure option parmi ces candidats pour constituer la `Réponse` finale qui sera présentée à l'utilisateur. Ce processus itératif permet au chatbot de maintenir un dialogue cohérent et pertinent.

Ce qui différencie fondamentalement les chatbots à base d'IA est leur capacité à s'affranchir d'un scénario rigide. Ils sont conçus pour interagir de manière plus humaine, en tenant compte non seulement du sens littéral, mais aussi du contexte et de l'historique de la conversation. Cette orientation vers une interaction naturelle et flexible les rend indispensables dans les solutions modernes de dialogue automatisé, transformant ainsi l'expérience utilisateur et ouvrant la voie à des applications de plus en plus sophistiquées.

I.3.2.3. Synthèse

Le tableau ci-dessous synthétise les différences clés entre ces deux approches :

Selon McTear (2020), cette évolution des chatbots reflète une transition du paradigme déterministe vers un paradigme probabiliste fondé sur l'apprentissage, où la conversation ne dépend plus uniquement d'un script préétabli mais d'un modèle entraîné sur des corpus variés [25].

En resumé, les chatbots peuvent être classés selon le type de logique qu'ils embarquent : des systèmes à base de règles simples, jusqu'aux agents intelligents ca-

Table I – *Comparaison entre chatbots basés sur des règles et basés sur l'IA*

Critères	Chatbots basés sur des règles	Chatbots basés sur l'IA
Complexité des requêtes	Idéal pour les requêtes simples et prévisibles	Excellents dans la gestion des requêtes complexes et ouvertes
Tâches ciblées	Efficace pour les tâches courantes et standard	Gère diverses tâches et interactions
Temps de développement et de formation	Rapide à développer et à mettre en œuvre	Nécessite plus de temps pour le développement et la formation
Maintenance et mises à jour	Nécessite des mises à jour manuelles fréquentes	Peuvent apprendre des données et s'améliorer, avec ajustements réguliers
Confidentialité et sécurité des données	Travaille avec peu de données, risques de confidentialité moindres	Nécessite une sécurité renforcée pour les ensembles de données volumineux
Budget et ressources	Moins coûteux à développer et maintenir	Coûts initiaux plus élevés, nécessité de ressources pour l'entraînement
Évolutivité	Évolutivité limitée pour les interactions complexes	Hautement évolutifs et adaptables à de nouveaux scénarios

pables de comprendre et de produire du langage naturel de manière contextuelle. Cette dernière catégorie, rendue possible par l'émergence des modèles de langage de grande taille, marque une véritable rupture technologique. Grâce à leur capacité d'analyse sémantique, de génération fluide, et d'adaptation au contexte conversationnel, Ils ouvrent ainsi la voie à une nouvelle génération d'assistants, plus fluides, plus intelligents, et surtout plus faciles à personnaliser pour des cas d'usage spécifiques.

I.4. Modèles de langage larges (LLMs)

I.4.1. Définition et caractéristiques

Les modèles de langage de grande taille, ou LLMs (*Large Language Models*), sont des systèmes d'intelligence artificielle conçus pour traiter le langage humain de manière avancée. Leur principal objectif est de comprendre, générer ou manipuler du texte en langage naturel, avec une précision et une fluidité qui les rendent utiles dans de nombreux domaines : rédaction automatisée, traduction, résumé, assistance vocale, ou encore génération de code.

Ce qui fait la spécificité des LLMs par rapport aux modèles traditionnels, c'est principalement leur **ampleur**. Ils sont entraînés sur d'immenses quantités de textes issus du web, de livres, d'articles, ou encore de dialogues. Pour cela, ils exploitent des milliards de paramètres, qui leur permettent de capter les régularités du langage, d'enrichir leur vocabulaire interne, et de modéliser des relations complexes entre les mots, les phrases et les idées. Les LLMs ne se contentent pas de traiter des mots isolés : ils sont capables de prendre en compte le *contexte global* d'une requête, ce qui leur permet de générer des réponses cohérentes, pertinentes et parfois même créatives. Grâce à cette capacité à apprendre les structures profondes du langage, ils peuvent s'adapter à une grande diversité de sujets, de tons et de formats. Cela explique leur large adoption dans les systèmes conversationnels modernes. Il est également important de noter que les performances des LLMs ne viennent pas seulement de leur volume de données ou de leur puissance de calcul, mais surtout de la manière dont ils sont construits. C'est cette structure interne qui leur donne la capacité de traiter efficacement le langage naturel, tout en s'adaptant à des tâches variées.

I.4.2. Fonctionnement et architecture des LLMs

Les modèles de langage de grande taille s'appuient sur une architecture particulière qui joue un rôle essentiel dans leur fonctionnement : le **Transformer**. Ce modèle, introduit en 2017, a profondément changé la façon dont les machines apprennent et traitent le langage. Grâce à lui, les LLMs peuvent analyser une phrase entière en tenant compte du sens global, et non plus mot par mot comme dans les anciennes approches.

L'architecture Transformer se compose en général de deux grandes parties : **l'encodeur** et **le décodeur**. L'encodeur lit le texte d'entrée (par exemple une question), l'analyse, et en construit une représentation que le modèle peut comprendre. Ensuite, le décodeur utilise cette information pour produire une réponse, mot après mot, de manière cohérente avec ce qui a été compris.

Tous les LLMs ne fonctionnent pas exactement de la même façon. Certains, comme *GPT*, n'utilisent que la partie décodeur, ce qui suffit pour générer du texte à partir d'une instruction. D'autres, comme *BERT*, se basent uniquement sur l'encodeur, lorsqu'il s'agit surtout de comprendre un texte. Chaque choix dépend de l'objectif : produire du texte, répondre à une question, détecter des intentions, etc.

En résumé, c'est cette architecture bien pensée — simple dans ses principes, mais puissante dans ses effets — qui permet aux LLMs de traiter le langage humain de manière efficace et adaptable à de nombreuses situations.

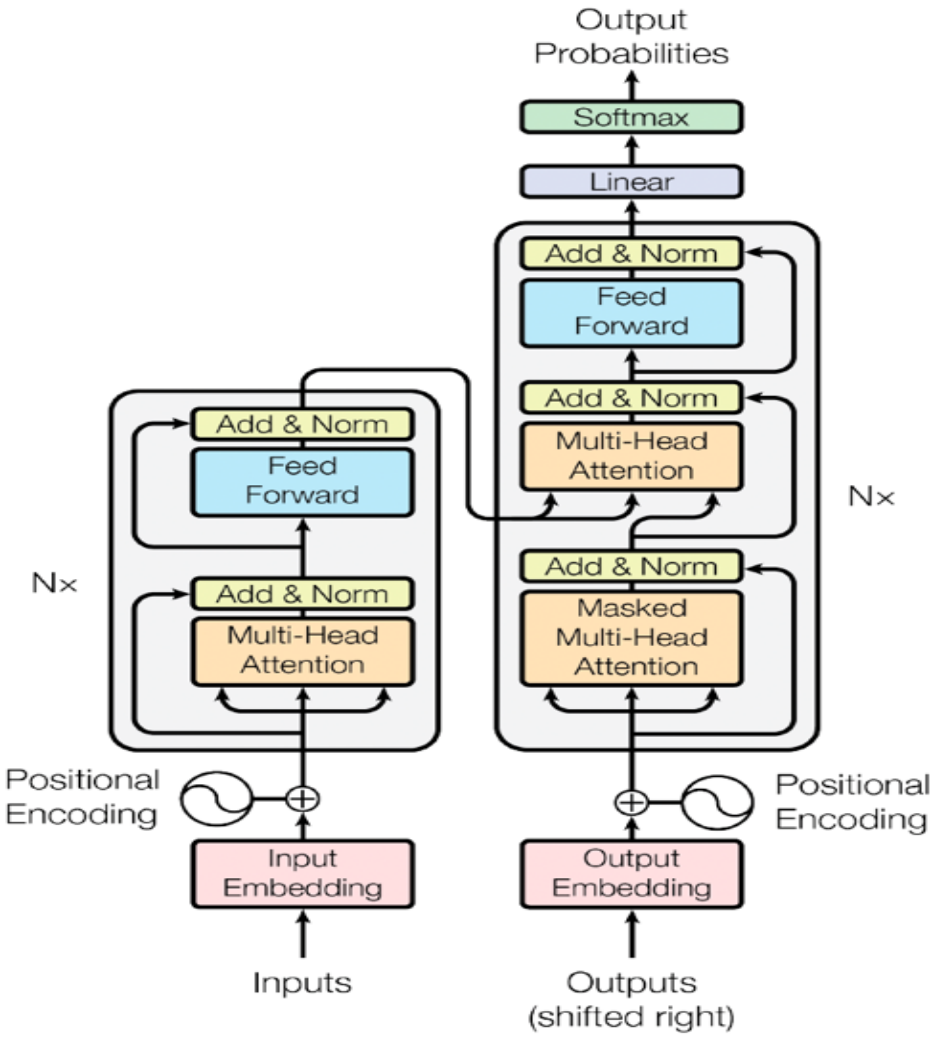


Figure 10 – *Architecture Transformer [40]*

Au cœur de cette architecture se trouve un principe simple mais puissant : **l'attention**. Ce mécanisme permet au modèle de repérer les mots les plus importants dans une phrase, même s'ils sont éloignés les uns des autres. Cela l'aide à mieux comprendre le message général et à produire des réponses plus justes et plus naturelles. L'attention est decrite par la formule suivante

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{I.1}$$

Où :

— Q est un vecteur qui représente l'élément pour lequel on cherche à extraire du contexte.

— K : ce sont les clés des autres éléments du texte : chacune est un vecteur K_j pour la position j. Elles contiennent l'essentiel de l'information sur chaque

mot pour que la requête Q puisse évaluer la pertinence de chaque mot j par rapport à i.

— $QK^\top$: On calcule le produit scalaire de Q_i avec chaque K_j, soit $Q_i \cdot K_j$. Ce score brut mesure la similitude ou la correspondance entre la requête et chaque clé : plus il est grand, plus le mot j est jugé pertinent pour i.

— $\sqrt{d_k}$: d_k est la dimension des vecteurs clés. Diviser par $\sqrt{d_k}$ permet de stabiliser les gradients en atténuant l'ampleur des produits scalaires lorsque la dimension est élevée, évitant ainsi que les softmax deviennent trop pointus.

— $\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)$ applique la fonction softmax sur les scores normalisés pour obtenir un vecteur de poids $\alpha = (\alpha_1, \ldots, \alpha_n)$ de somme 1. Chaque α_j indique la force d'attention accordée au mot j lorsque l'on produit le vecteur de sortie pour i.

— V sont les valeurs associées aux clés : chaque V_j contient l'information détaillée (un vecteur de même dimension) qu'on souhaite agréger. Plutôt que d'extraire directement les K_j, on prend la combinaison pondérée de V_j selon les poids α_j.

— Le produit $\text{softmax}(\cdots)V$ multiplie la matrice des poids α par la matrice V. Le résultat est un vecteur unique pour la position i : c'est le contexte appris, construit comme une moyenne pondérée des valeurs V_j, où chaque j contribue selon sa pertinence mesurée par α_j.

. Le Transformer utilise également une multi-head attention, c'est-à-dire qu'il calcule l'attention de plusieurs manières différentes en parallèle, puis concatène et projette ces différentes perspectives, permettant au modèle de capturer simultanément divers types de relations entre les mots. Ils disposent d'un encodage positionnel (positionnal encoding) pour conserver l'information sur l'ordre des mots dans la séquence, une donnée cruciale pour la compréhension du langage naturel. Cet encodage est ajouté aux vecteurs d'entrée afin que le modèle puisse déterminer la position relative des mot

I.4.2.1. L'encodeur

L'encodeur fonctionne comme un lecteur attentif. Il commence par transformer chaque mot de la phrase en un vecteur numérique à l'aide d'une couche d'« embeddings ». À ces vecteurs s'ajoutent des informations de position « positionnal encoding », ce qui permet au modèle de tenir compte de l'ordre des mots. Ces vecteurs enrichis passent ensuite à travers plusieurs blocs identiques, dans lesquels deux étapes essentielles se répètent : une première étape d'«attention multi-têtes», qui permet au modèle de détecter les liens entre les mots, suivie d'un petit réseau de neurones, appelé «feedforward», qui affine les représentations obtenues. Chaque bloc est accompagné de mécanismes de normalisation et de connexions dites résiduelles, qui facilitent l'apprentissage.

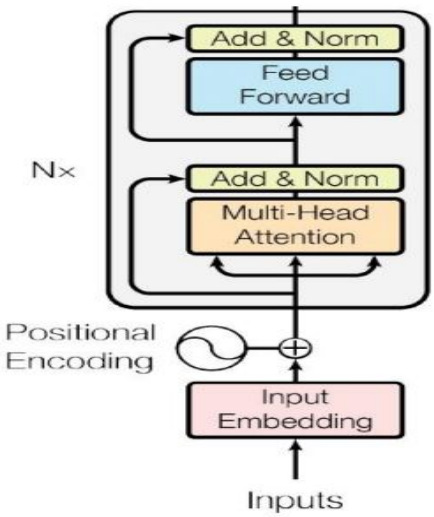


Figure 11 – *L'encodeur*

Sur cette figure, on observe que chaque mot passe d'abord par un encodage de position (positional encoding) avant d'entrer dans les couches d'attention. L'attention « auto-référentielle » (self-attention) permet de comparer chaque mot à tous les autres, ce qui donne au modèle une vue d'ensemble du contexte. Le réseau «feed-forward» vient ensuite transformer les informations obtenues, pour enrichir les représentations internes. Enfin, les connexions résiduelles et la normalisation assurent une bonne stabilité du réseau, ce qui est crucial lorsque des dizaines de blocs sont empilés.

Exemples de modèles encodeurs

Plusieurs modèles de type encodeur ont été développés dans le domaine du traitement du langage naturel. Voici quelques-uns des plus représentatifs :

— **BERT (Bidirectional Encoder Representations from Transformers)** : ce modèle développé par Google apprend à prédire des mots masqués dans une phrase, en tenant compte du contexte des deux côtés. Il est très utilisé pour des tâches comme la classification d'intention ou l'analyse de sentiment.

— **RoBERTa** : une version optimisée de BERT, avec un entraînement plus intensif et sans tâche de prédiction de phrase suivante.

— **DistilBERT** : une version plus légère de BERT, conçue pour être plus rapide tout en maintenant des performances comparables.

I.4.2.2. Le decodeur

Le décodeur joue maintenant le rôle de générateur. Contrairement à l'encodeur qui traite tous les mots d'un coup, le décodeur produit les mots un par un. À chaque étape, il prend en compte les mots déjà générés et interagit avec les informations produites par l'encodeur. Sa structure est semblable à celle de l'encodeur, mais elle contient un mécanisme d'attention supplémentaire. En plus de l'attention sur ses propres sorties passées, il inclut une attention croisée (appelée « cross-attention »)

qui lui permet de se connecter aux sorties de l'encodeur. Là encore, on retrouve des couches de feedforward, de normalisation et des connexions résiduelles.

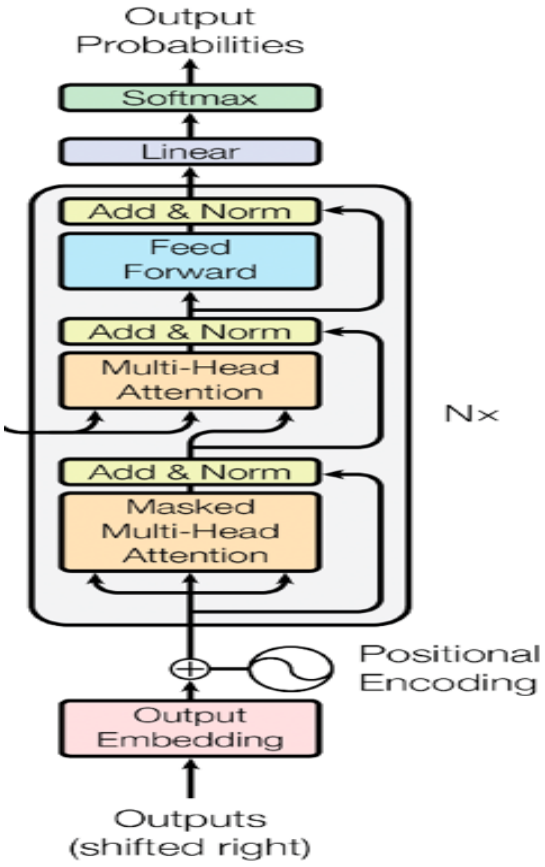


Figure 12 – *Le decodeur*

Dans cette figure, on peut voir que le premier bloc d'attention est masqué : le modèle ne peut pas voir les mots futurs qu'il n'a pas encore produits, ce qui est essentiel pour la génération mot à mot. Le second bloc d'attention reçoit les représentations issues de l'encodeur, ce qui permet de guider la génération à partir de l'entrée initiale. Enfin, comme dans l'encodeur, les blocs sont stabilisés par des connexions résiduelles et des normalisations.

Plusieurs modèles reposent uniquement sur l'architecture du décodeur, spécialement conçue pour la génération de texte. Voici quelques-uns des plus représentatifs :

— **GPT (Generative Pre-trained Transformer)** : ce modèle, développé par OpenAI, génère du texte mot par mot en utilisant uniquement des blocs de décodeur. Il repose sur l'attention masquée pour éviter de prédire des mots futurs, ce qui le rend particulièrement performant pour les tâches de génération de texte.

— **GPT-2 / GPT-3 / GPT-4** : ces versions successives de GPT augmentent progressivement la taille des modèles et la qualité du texte généré. Elles sont capables de comprendre et générer du texte dans des contextes très variés, allant de la traduction à la réponse à des questions complexes.

— **CTRL (Conditional Transformer Language Model)** : ce modèle permet de contrôler le style ou le sujet du texte généré à partir d'un préfixe conditionnel, tout en conservant l'architecture de décodeur pur.

Synthese

En combinant ces deux couches, encodeur et décodeur, le Transformer parvient à encoder l'information textuelle d'une manière riche et contextuelle, puis à la réutiliser pour produire une sortie cohérente et fluide. Cette architecture modulaire et empilable a permis le développement de modèles capables de traiter d'immenses volumes de données textuelles, d'apprendre des relations complexes entre les mots, et de s'adapter à des tâches variées telles que la génération de texte, la traduction, la réponse à des questions ou la synthèse de documents. Elle constitue aujourd'hui la base technique de presque tous les LLMs modernes.

I.4.3. Processus d'Apprentissage des LLMs

L'apprentissage des modèles de langage de grande taille repose sur un processus en deux grandes étapes : le pré-entraînement, qui constitue la phase d'apprentissage initiale à grande échelle, puis l'affinage (ou fine-tuning), qui permet d'adapter le modèle à des tâches spécifiques. Ce processus progressif permet aux LLMs d'acquérir d'abord des connaissances générales sur le langage, puis de les spécialiser selon les besoins.

I.4.3.1. Le Pré-entraînement (Pre-training)

Le pré-entraînement est une phase cruciale dans la formation des LLMs. Il consiste à exposer le modèle à d'immenses quantités de textes, parfois collectés à partir de sources variées comme des livres, des articles scientifiques, des forums ou des sites web. À ce stade, l'objectif n'est pas de résoudre une tâche précise, mais de familiariser le modèle avec les régularités du langage naturel. Il apprend, par exemple, à prédire le mot suivant dans une phrase, à remplir des trous dans un texte ou à générer des textes cohérents. Cette phase d'entraînement est dite non supervisée, car le modèle apprend sans étiquettes prédéfinies, simplement en observant les structures linguistiques.

Les principales étapes du pré-entraînement d'un modèle de langage sont les suivantes :

1. **Collecte des données** : Constitution d'un corpus textuel vaste, diversifié et représentatif.
2. **Nettoyage et prétraitement** : Suppression des bruits, normalisation des textes et mise en forme des données.
3. **Choix de l'architecture** : Sélection d'un modèle de type Transformer adapté aux objectifs (ex. : BERT, GPT).
4. **Entraînement du modèle** : Apprentissage des représentations linguistiques à partir des données via des algorithmes d'apprentissage profond.
5. **Évaluation** : Validation des performances sur des jeux de données spécifiques pour détecter les biais et ajuster les paramètres.

6. **Déploiement** : Utilisation du modèle pour des tâches réelles ou comme base pour un ajustement (fine-tuning).

Grâce à ce pré-entraînement, le modèle développe une compréhension fine de la syntaxe, de la grammaire, du vocabulaire, mais aussi des associations sémantiques entre les mots. Tout ceci lui permet ainsi de comprendre une large variété de phrases, même en dehors d'un domaine particulier.

I.4.3.2. Le Fine tuning

Après la phase de pré-entraînement, qui permet au modèle d'acquérir une compréhension générale du langage, le *fine-tuning* constitue une étape essentielle pour adapter ce modèle à une tâche précise. C'est cette phase qui permet de transformer un modèle générique en un assistant conversationnel performant, capable de dialoguer dans un contexte métier bien défini, comme celui de la prise de rendez-vous médical. Le fine-tuning consiste à réentraîner un modèle pré-entraîné sur un jeu de données spécifique, souvent de taille plus modeste mais bien préparé. Contrairement au pré-entraînement, qui est non supervisé, le fine-tuning est généralement réalisé en mode supervisé : chaque entrée est accompagnée d'une sortie cible (réponse, catégorie, action, etc.). Cela permet au modèle d'apprendre à associer les bonnes réponses aux bonnes formulations.

Concrètement, dans le cas d'un chatbot de génération de facture, le modèle peut être affiné à partir d'un ensemble de dialogues clients et produit. Il apprend alors à extraire des informations pertinentes telles que le nom du client, la quantite de produits commandés le nom et prix du produit, à partir de phrases formulées en langage naturel. Ce type de spécialisation permet de réduire les hallucinations du modèle, d'améliorer sa précision et sa cohérence.

L'intérêt du fine-tuning est largement documenté. Des travaux comme ceux de Raffel et al. [30] ont montré qu'un fine-tuning ciblé peut améliorer les performances sur une tâche spécifique de plus de 20 points de pourcentage par rapport à l'usage d'un modèle générique non affiné. De plus, même avec peu de données (quelques milliers d'exemples bien choisis), les résultats peuvent être significatifs, ce qui rend le fine-tuning accessible dans des contextes où l'annotation est coûteuse.

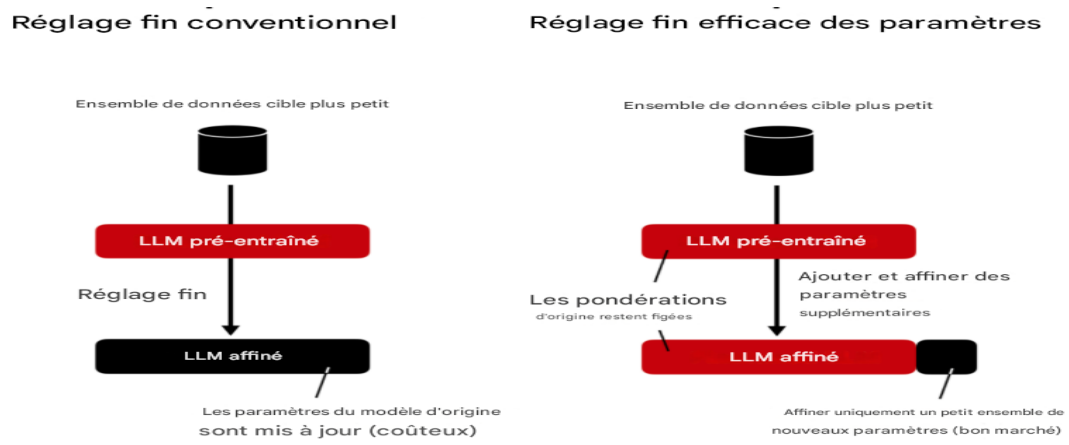


Figure 13 – *Illustration des techniques de fine-tuning [30]*

Le fine-tuning peut être total ou partiel. Dans le premier cas, tous les paramètres du modèle sont ajustés ce qui est couteux. Dans le second, seule une petite portion du modèle est modifiée, ce qui permet de réduire le coût en calcul et en mémoire. Cette approche, appelée « fine-tuning léger » (ou *parameter-efficient fine-tuning*), est de plus en plus utilisée, notamment avec des techniques comme LoRA (Low-Rank Adaptation) [11], qui rendent l'adaptation plus rapide et plus économique, tout en maintenant de bonnes performances. Il offre ainsi plusieurs avantages notables, comme l'illustre la Figure 14.



Figure 14 – *Avantages du fine-tuning*

Il permet d'atteindre une efficacité des données, car il ne nécessite qu'un volume modeste d'exemples spécialisés pour produire de bons résultats. Il améliore aussi l'efficacité temporelle, en capitalisant sur un modèle déjà formé. Grâce au transfert de connaissances issu du pré-entraînement, le modèle conserve une compréhension générale du langage tout en s'adaptant finement à des domaines spécifiques. Il favorise également la spécialisation des réponses et peut représenter une solution plus économique que l'entraînement d'un modèle complet depuis zéro.

Cette technique bien que presentant des avantages, presente aussi quelques limites évoquées précédemment, telles que la nécessité d'annoter des données, les ressources computationnelles ou les risques de surspécialisation. Il s'agit donc de trouver un compromis entre performance ciblée et maîtrise des coûts et des risques.

I.4.4. Techniques d'adaptation et d'optimisation

En complément du fine-tuning, il existe d'autres méthodes permettant d'adapter les LLMs à des tâches précises sans avoir à réentraîner entièrement le modèle. Ces approches, souvent plus légères, visent à exploiter au mieux les capacités d'un modèle pré-entraîné, soit en optimisant la manière de lui poser les questions, soit en l'enrichissant avec des informations externes. Les deux méthodes les plus utilisées aujourd'hui sont le *prompt engineering* et la *génération augmentée par récupération*, plus connue sous le nom de RAG.

I.4.4.1. Le prompt engineering

Le prompt engineering consiste à formuler soigneusement l'entrée fournie au modèle, c'est-à-dire le texte que l'on lui soumet, afin d'obtenir la sortie souhaitée. Le principe est simple, mais redoutablement efficace : un même modèle peut répondre de manière très différente selon la manière dont la question lui est posée.

Par exemple, demander *"Génère la facture du client numéro 254 pour le mois de juin"* ne produira pas forcément le même résultat que *"Établis un reçu pour le client 254 correspondant à ses achats de juin"*. Bien que les deux requêtes visent le même objectif, la formulation différente peut influencer la manière dont le modèle interprète et traite la demande, surtout s'il n'a pas été exposé à des variantes linguistiques similaires durant son pré-entraînement. Le prompt engineering cherche à tirer parti de ces variations, en structurant la requête de façon à guider le modèle vers une réponse précise, fiable et cohérente[41]. . Ce procédé est devenu central dans l'utilisation des LLMs, notamment dans les interfaces sans fine-tuning, car il ne nécessite aucune modification interne du modèle. Il est largement utilisé pour concevoir des assistants conversationnels dynamiques, capables de répondre selon des cadres pré-définis. Certaines techniques avancées incluent l'injection d'exemples, la formulation par étapes (chained prompts) ou encore la reformulation automatique des entrées.

I.4.4.2. La génération augmentée par récupération (RAG)

La méthode RAG proposée par Lewis et al. [23] est une autre stratégie qui permet de contourner une limite majeure des LLMs : leur connaissance est figée à la date de leur pré-entraînement. Ainsi, un modèle comme GPT-3, entraîné sur des données de 2021, n'a aucune connaissance des événements ou informations postérieures. Pour résoudre ce problème, le RAG consiste à connecter le modèle à une base documentaire externe, dans laquelle il peut rechercher les informations pertinentes avant de formuler sa réponse. Ce processus suit deux étapes principales. D'abord, une requête de l'utilisateur est utilisée pour interroger une base documentaire (articles, bases de données, FAQ, etc.). Ensuite, les documents les plus pertinents sont transmis au LLM, qui les utilise pour produire une réponse contextualisée. Cette approche garantit une réponse plus à jour, mais aussi plus fiable dans les domaines sensibles comme la médecine, le droit ou l'éducation.

RAG combine donc le meilleur des deux mondes : la puissance de compréhension du langage du LLM, et les données d'une source externe. Cela en fait une solution très utilisée pour les assistants conversationnels en entreprise.

I.4.5. Contraintes et défis

Malgré leurs performances impressionnantes, les modèles de langage de grande taille présentent plusieurs limites qui freinent leur adoption à grande échelle, notamment dans les contextes critiques comme la santé, le droit ou l'éducation. Ces contraintes ne sont pas seulement d'ordre technique, mais touchent aussi à des aspects économiques, éthiques et réglementaires.

L'un des défis majeurs reste leur **coût computationnel**. L'entraînement initial d'un LLM nécessite des ressources matérielles considérables, souvent hors de portée pour les institutions non industrielles. Par exemple, le modèle GPT-3, avec ses 175 milliards de paramètres, aurait nécessité plusieurs milliers de GPU et des semaines de calcul pour son pré-entraînement. Même après cette phase, le fine-tuning ou l'utilisation en production peut rester coûteux, tant en énergie qu'en mémoire vive.

Un autre enjeu important est celui des **hallucinations**. Il s'agit de situations où le modèle produit des réponses convaincantes sur la forme, mais factuellement fausses ou inventées. Cela pose un problème crucial dans les domaines à forte exigence de fiabilité, comme la médecine. Ce comportement découle du fait que les LLMs ne raisonnent pas au sens strict, mais génèrent du texte en se basant sur des régularités statistiques. Plusieurs études, comme celle de Ji et al. (2023) [15], mettent en lumière la fréquence de ces hallucinations, notamment dans les réponses à des questions précises ou techniques.

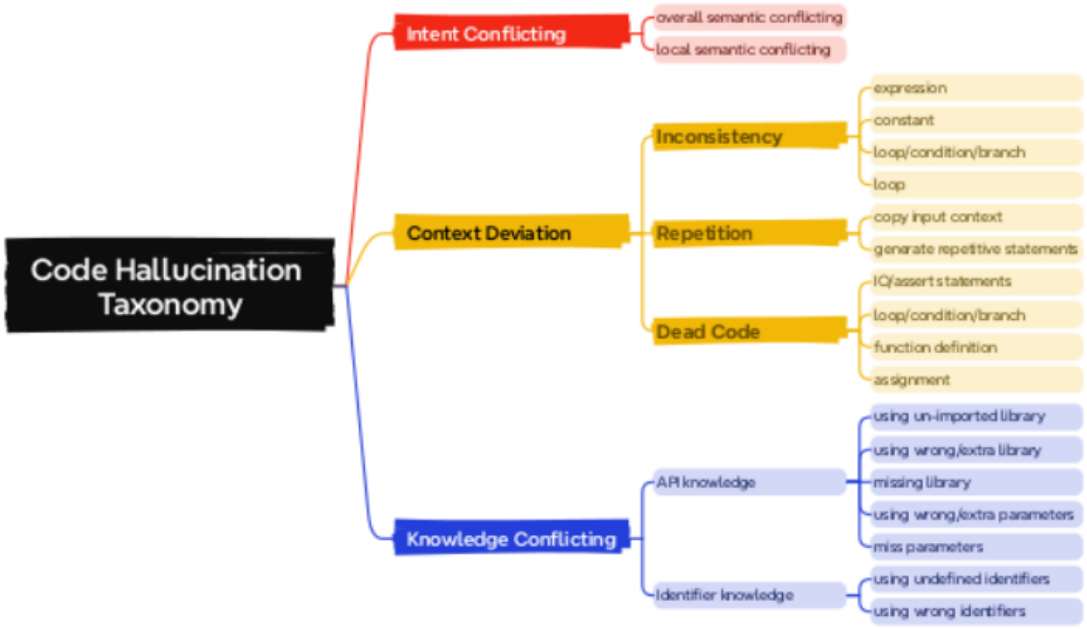


Figure 15 – *taxonomie des hallucinations dans la génération de code*

Par ailleurs, les LLMs posent des problèmes de **biais et discrimination**. Puisqu'ils apprennent à partir de textes disponibles sur Internet, ils reproduisent – parfois même amplifient – les stéréotypes présents dans ces données. Cela peut engendrer des comportements sexistes, racistes ou politiquement tendancieux, même dans des contextes supposés neutres. Des mécanismes d'atténuation existent, mais ils restent imparfaits.

Du point de vue de la **confidentialité**, les LLMs peuvent mémoriser des fragments de données sensibles présents dans leurs corpus d'entraînement. Cela soulève des inquiétudes sur la protection des données personnelles, en particulier lorsque les modèles sont utilisés dans des secteurs réglementés comme la santé, où la confidentialité est un impératif légal.

Enfin, l'intégration des LLMs dans des systèmes réels pose des défis en termes de **contrôle, supervision et explicabilité**. Il reste difficile de prédire exactement pourquoi un modèle génère une réponse donnée, ce qui complique la validation des résultats, notamment dans les scénarios critiques.

I.5. Conclusion

En somme, nous avons pu établir un cadre théorique structuré autour des piliers du génie logiciel, des mécanismes des assistants conversationnels et des fondements des modèles de langage de grande taille. Il a mis en évidence l'importance de principes méthodologiques rigoureux pour encadrer la création d'outils intelligents, tout en soulignant les apports récents de l'apprentissage profond dans le traitement automatique du langage. Ces éléments théoriques serviront de socle à l'analyse critique qui suit. Le chapitre II s'intéressera aux approches existantes pour la conception d'assistants conversationnels, en particulier celles qui mobilisent des techniques de fine-tuning sur les LLMs. Cette revue permettra d'identifier les limites actuelles, ainsi que les leviers d'innovation que notre proposition entend explorer.

Chapitre II

REVUE DE LA LITTÉRATURE

Sommaire

II.1 - Introduction 30
II.2 - Démarche de revue et Stratégie de recherche documentaire 31
II.3 - Analyse des travaux existants 35
II.4 - Synthèse critique et positionnement de la recherche 45
II.5 - Conclusion 48

II.1. Introduction

La conception d'assistants conversationnels repose aujourd'hui sur un large éventail d'approches, de cadres méthodologiques et de technologies. Toutefois, peu de travaux s'attachent à les analyser de manière structurée, en tenant compte des évolutions récentes liées à l'émergence des modèles de langage de grande taille. Pour mieux situer notre proposition dans ce contexte, une revue systématique des contributions existantes s'impose.

Ce chapitre présente cette revue, en s'appuyant sur une démarche rigoureuse permettant d'identifier les principales approches de conception, les outils disponibles et les rôles assignés aux LLMs dans les systèmes conversationnels.

II.2. Démarche de revue et Stratégie de recherche documentaire

II.2.1. Démarche de revue

Pour construire quelque chose de solide, il faut toujours commencer par bien regarder ce qui existe déjà. Avant de développer notre propre idée sur la création d'assistants conversationnels à l'aide des modèles de langage, nous avons pris le temps d'étudier avec soin les travaux déjà réalisés dans ce domaine. Cela ne s'est pas fait de manière hasardeuse. Nous avons suivi une méthode bien précise : la revue systématique de la littérature, telle que définie par Kitchenham [17]. Cette méthode nous a permis de faire une recherche claire, organisée et fiable, en sélectionnant uniquement les études les plus utiles pour notre sujet.

Nous avons d'abord décidé ce que nous voulions chercher, sur quelles bases nous allions choisir les documents à retenir, et quelles sources scientifiques nous allions consulter. Une fois cette préparation terminée, nous avons commencé à lire et analyser plusieurs publications qui expliquent comment des chercheurs ont utilisé des modèles de langage pour créer des assistants conversationnels.

Ensuite, nous avons regardé de près chaque étude choisie : quelles techniques ont été utilisées ? Quels résultats ont été obtenus ? Quelles limites ont été rencontrées ? Ce travail nous a permis de mieux comprendre les approches déjà proposées, de voir ce qui fonctionne bien et ce qui reste encore à améliorer.

Grâce à cette étape essentielle, nous avons pu poser les bases de notre propre réflexion, avec une vision claire de ce qui a été fait et de ce que nous pouvons apporter de nouveau.

II.2.2. Stratégie de recherche documentaire

Afin de garantir une analyse rigoureuse et exhaustive des recherches existantes, une revue systématique de la littérature (RSL) sera réalisée. Cette approche vise à identifier, évaluer et synthétiser de manière structurée les études pertinentes sur le sujet, offrant ainsi une vue d'ensemble complète des travaux antérieurs, l'identification des tendances actuelles et des lacunes dans la littérature. La méthodologie systématique adoptée repose sur un ensemble d'étapes clairement définies pour explorer en profondeur le sujet de recherche.

II.2.2.1. Definition des Questions de recherche

La première phase cruciale de planification d'une revue systématique consiste à formuler des questions de recherche claires et précises. Ces questions guident le processus de collecte, d'analyse et d'interprétation des données, ce qui délimite le champ d'investigation et assure la pertinence des réponses. *Ces questions de recherche serviront de fil conducteur pour structurer l'étude, guider l'analyse des outils existants et orienter la conception de la solution proposée.*

Les questions de recherche définies pour ce projet sont présentées dans le tableau suivant.

Table II – *Questions de recherche et leurs motivations pour guider l'étude*

ID	Questions de recherche	Motivation
QR1	Quelles sont les méthodologies et processus prédominants pour créer des assistants conversationnels intelligents ?	Identifier les approches existantes pour concevoir et implémenter des assistants conversationnels et analyser leur structure.
QR2	Ces méthodologies sont-elles alignées avec les étapes traditionnelles de l'ingénierie logicielle ? Si oui, lesquelles et dans quelle mesure ?	Évaluer si ces approches intègrent des principes d'ingénierie logicielle, tels que la collecte des besoins, la conception, l'implémentation, les tests et la maintenance.
QR3	Existe-t-il des outils ou plateformes dédiés qui facilitent la mise en œuvre de ces méthodologies ?	Déterminer la disponibilité d'outils (par exemple, ateliers d'ingénierie logicielle) qui aident les concepteurs à suivre des méthodologies structurées.
QR4	Quelles techniques d'IA sont généralement utilisées dans ces méthodologies (par exemple, fine-tuning de LLM, apprentissage par renforcement) ?	Comprendre le rôle et la diversité des techniques d'IA dans l'activation d'assistants conversationnels intelligents.
QR5	Comment ces approches soutiennent-elles la validation des assistants conversationnels par rapport aux spécifications initiales ?	Examiner si les méthodologies existantes incluent des mécanismes de validation garantissant l'alignement avec les exigences et objectifs.
QR6	Dans quelle mesure les tâches de ces approches sont-elles automatisées (par exemple, entraînement, validation ou déploiement) ?	Évaluer le niveau d'automatisation fourni par les méthodologies, ce qui pourrait influencer l'efficacité et la scalabilité dans la création d'assistants.
QR7	Quels sont les défis communs rencontrés dans le développement d'assistants conversationnels intelligents ?	Identifier les difficultés et obstacles récurrents dans le processus de création de systèmes conversationnels.
QR8	Quelles améliorations ou innovations sont fréquemment suggérées dans la littérature pour surmonter ces défis ?	Explorer les avancées potentielles ou optimisations pour améliorer les méthodologies existantes et surmonter les limites.
QR9	Comment ces méthodologies abordent-elles les préoccupations centrées sur l'utilisateur, telles que l'utilisabilité, l'accessibilité et la personnalisation ?	Évaluer si les facteurs orientés utilisateur sont intégrés dans la conception et la validation des assistants conversationnels.

II.2.2.2. Critères d'inclusion et d'exclusion

Pour assurer la rigueur et l'exhaustivité de la revue systématique, la phase de planification, essentielle à la cohérence et à la fiabilité de l'étude, inclut une définition précise des critères d'inclusion et d'exclusion des documents. Ces critères garantissent que seules les études les plus pertinentes sont retenues pour l'analyse, évitant ainsi des conclusions partielles ou erronées.

Table III – *Critères d'inclusion, d'exclusion et justifications*

Critère	**Critères d'inclusion**	**Critères d'exclusion**	**Justification**
Type de source	Articles de revues académiques, actes de conférences, chapitres de livres	Blogs, tutoriels, opinions, documents sans validation scientifique	Assurer la validité scientifique des sources exploitées
Langue	Documents publiés en anglais	Langues non maîtrisées (sauf traduction officielle)	Faciliter l'analyse sans barrière linguistique
Période	Articles publiés entre 2009 et 2024	Travaux antérieurs à 2015 (sauf papiers fondateurs)	Cibler l'évolution récente tout en intégrant les bases théoriques
Thématique	Fine-tuning des LLMs, ingénierie logicielle pour assistants, MLOps	Outils obsolètes, études trop spécifiques ou trop générales	Se concentrer sur des approches généralisables et actuelles
Méthodologie	Études proposant des cadres conceptuels ou des architectures	Articles descriptifs sans cadre reproductible ou résultats pratiques	Assurer la réutilisabilité et la valeur applicative des résultats
Type d'assistants	Assistants conversationnels basés sur des LLMs	Chatbots à règles ou systèmes sans LLM	Rester centré sur les technologies modernes ciblées par le projet
Accès au contenu	Texte intégral accessible	Publications inaccessibles en version complète	Permettre une lecture et une analyse approfondies des travaux

Concernant les **critères d'inclusion**, l'étude ciblera principalement les articles de revues académiques, les actes de conférences et les chapitres de livres. La période de publication est spécifiquement définie entre 2009 et 2024 afin de capturer les développements récents dans le domaine des grands modèles de langage (LLMs) et des assistants conversationnels. Tous les documents retenus devront être publiés en anglais pour des raisons de maîtrise linguistique. Sur le plan thématique, l'intérêt porte sur les travaux abordant la conception d'outils, d'ateliers ou de frameworks dédiés au fine-tuning des LLMs, les méthodologies d'ingénierie logicielle appliquées à

la création d'assistants conversationnels, ainsi que l'analyse des défis liés à la personnalisation des modèles de langage pour des cas pratiques. Le domaine d'application privilégié concerne les assistants conversationnels et leur création via des modèles pré-entraînés, incluant également les études sur l'intégration des LLMs dans des solutions concrètes telles que les pipelines automatisés ou le MLOps. Enfin, sur le plan méthodologique, les travaux devront proposer une architecture ou un cadre théorique pour des workflows logiciels, présenter des études qualitatives avec une méthodologie claire et reproductible, et apporter des résultats applicables à la conception d'outils logiciels, notamment en termes de réutilisabilité, de modularité et d'automatisation.

Quant aux **critères d'exclusion**, plusieurs types de documents ne seront pas pris en compte, tels que les blogs, les articles d'opinion, les publications sans accès au texte intégral qui limiteraient l'analyse, et les documents techniques dépourvus de validation scientifique, comme les tutoriels ou les guides non académiques. Les études centrées exclusivement sur des applications spécifiques, par exemple un chatbot pour une banque ou une application médicale, seront écartées si elles n'abordent pas des méthodologies générales applicables à d'autres contextes. De même, les articles jugés trop génériques, qui ne fournissent pas d'analyse approfondie ou de résultats directement applicables à la création systématique d'assistants conversationnels, seront exclus. Une attention particulière est portée à la période de publication, excluant les travaux antérieurs à 2015, à moins qu'ils ne soient reconnus comme des articles fondateurs (seminal papers). Les publications axées uniquement sur les algorithmes ou les architectures internes des modèles, se concentrant sur les détails techniques des LLMs plutôt que sur leur intégration dans un atelier, ne seront pas retenues. Les publications dans des langues non maîtrisées (comme le chinois, le russe, ou l'arabe) seront également exclues, sauf si une traduction officielle est disponible. Enfin, les articles abordant des technologies, outils ou approches largement dépassés dans le domaine du fine-tuning ou des assistants conversationnels (par exemple, l'utilisation de modèles de langage pré-LLM ou des outils non maintenus) seront mis de côté. Sont également exclus les thèmes non pertinents, tels que les études portant uniquement sur les chatbots basés sur des règles ou les assistants n'utilisant pas de LLMs, ainsi que les articles traitant de l'implémentation technique ou du code sans lien avec la conception conceptuelle d'un atelier.

II.2.2.3. Moteurs de recherche utilisés

La recherche a été menée sur plusieurs bases de données académiques majeures afin de garantir une couverture exhaustive de la littérature scientifique pertinente. Les principales plateformes consultées incluent IEEE Xplore, ACM Digital Library, ScienceDirect, SpringerLink et Google Scholar. Scopus a également été utilisé dans le cadre de cette recherche. Chaque requ^ete a ´et´e con¸cue pour couvrir des aspects cl´es de la recherche.

Les recherches seront effectuées à l'aide de mots-clés spécifiques combinés avec des opérateurs booléens, tels que "large language models" AND "fine-tuning" AND "conversational agents".

- Une**requête générale** a d'abord été élaborée pour couvrir les aspects fondamentaux du thème, combinant des termes relatifs aux agents conversationnels, aux méthodologies d'ingénierie logicielle, au fine-tuning des LLMs et

aux aspects de validation ou d'automatisation.Nous lavons defini comme suit ("AI chatbot" OR "conversational assistant" OR "intelligent assistant") AND ("software engineering methodology" OR "software development lifecycle" OR "engineering process") AND ("LLM fine-tuning" OR "machine learning" OR "natural language processing") AND ("validation" OR "testing" OR "tool support" OR "automation") apres son execution sur Scholar Google, nous avons obtenu 355 etudes.

- Sur **IEEE Xplore** nous avons propose la requete ("software engineering" OR "software development methodologies") AND ("fine-tuning" OR "transfer learning" OR "model adaptation") AND ("large language models" OR "GPT" OR "BERT" OR "T5") AND ("conversational AI" OR "chatbots") Apr‘es ex´ecution de la requete, nous avons obtenu 10 resultats parmi lesquels 10 Conf´erences.
- Sur SpringerLink, la requête proposee etait "software engineering" AND ("workshop" OR "framework" OR "approach" OR "training program") AND "fine-tuning" AND ("large language models" OR "GPT" OR "BERT" OR "transfer learning") AND ("conversational agents" OR "dialog systems") AND ("methodology" OR "design process" OR "development process" OR "system design") AND ("realworld applications" OR "practical applications" OR "case studies") Apr‘es ex´ecution de la requete, nous avons obtenu 06 resultats.
- Sur ACM Digital Library nous avons propose TITLE-ABS-KEY("large language models" AND "fine-tuning" AND ("chatbots" OR "customer support chatbots") AND "bias mitigation" AND ("real-world applications" OR "case study")) AND Keywords("workshop" OR "framework") AND Abstract("methodology" AND "design process" AND "practical approach") AND NOT Keywords("survey" OR "review") Apr‘es ex´ecution de la requete, nous avons obtenu 31 resultats.

II.3. Analyse des travaux existants

Dans le cadre de cette étude, 13 articles pertinents ont été analysés, traitant des méthodologies et approches d'ingénierie logicielle appliquées aux assistants conversationnels et au fine-tuning des LLMs. Ces travaux couvrent divers aspects tels que l'automatisation du développement, l'intégration des LLMs dans des workflows logiciels et la structuration d'un cadre modulaire pour la conception d'un atelier dédié. Voici une syntanalyse des études retenues et leur pertinence pour la recherche.

II.3.1. Méthodologies actuelles de conception d'assistants

La conception d'agents conversationnels repose sur différentes approches méthodologiques qui visent à structurer, automatiser et optimiser le processus de développement. Ces méthodes permettent d'adresser les défis liés à la complexité croissante des dialogues, à la diversité des besoins métiers, ainsi qu'à l'intégration de technologies avancées telles que les modèles de langage de grande taille (LLMs).

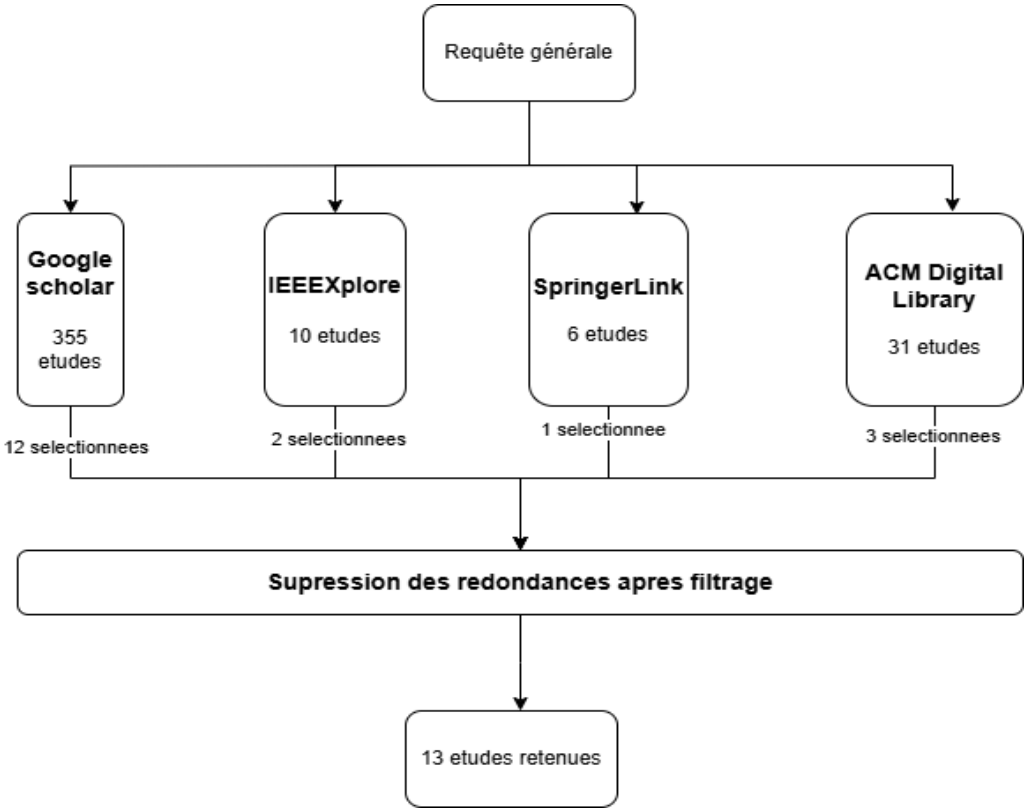


Figure 16 – *Processus de Sélection des Études pour la Revue Systématique*

Parmi les principales approches utilisées, on distingue notamment le développement dirigé par les modèles (Model-Driven Development, MDD), les architectures orientées microservices, ainsi que les plateformes low-code et no-code. Ces méthodes présentent des avantages et limites spécifiques, que nous détaillerons dans les sous-sections suivantes.

Le développement dirigé par les modèles (Model-Driven Development, MDD) est une approche méthodologique visant à privilégier la création de modèles abstraits pour concevoir un logiciel. Ces modèles représentent les composants, comportements et règles métier sous une forme compréhensible, souvent graphique ou déclarative. Le code source est ensuite automatiquement généré à partir de ces modèles.

L'objectif principal du MDD est d'augmenter la productivité, de réduire les erreurs humaines, et d'assurer une meilleure traçabilité entre les besoins métiers et la réalisation technique [36]. Le schéma ci-dessous illustre le principe fondamental du MDD :



Figure 17 – *Illustration du fonctionnement du MDD[36]*

Ce schéma synthétise le principe du développement dirigé par les modèles : à partir d'une représentation abstraite du système à concevoir, des transformations permettent d'obtenir automatiquement une application opérationnelle. Cette approche repose généralement sur trois niveaux d'abstraction formalisés par l'Object Management Group (OMG)[26] :

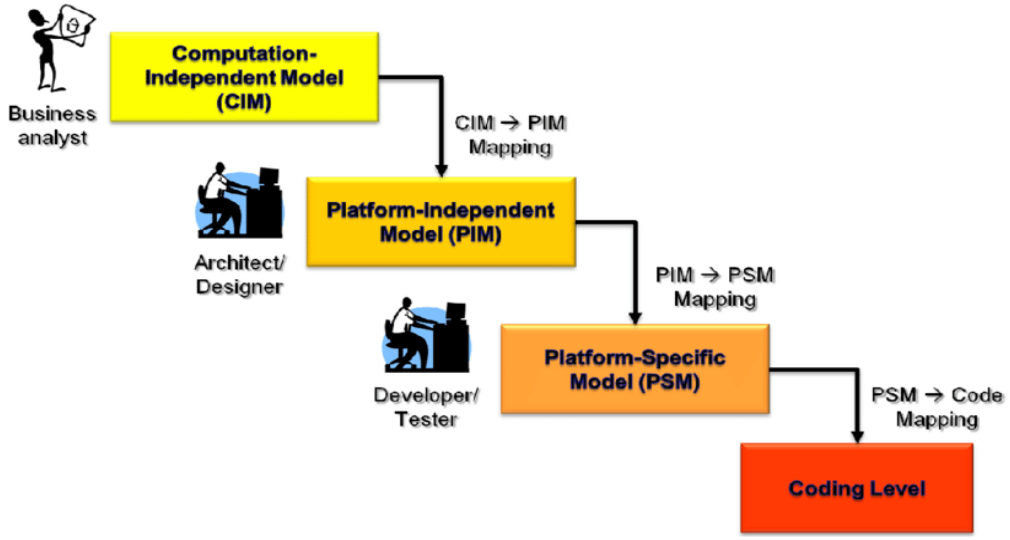


Figure 18 – *Illustration des 03 Niveaux d'abstraction du MDD[26]*

- **CIM (Computation Independent Model)** : il décrit les besoins métier et les objectifs sans se préoccuper des aspects techniques. C'est le niveau où l'on formalise ce que le système doit faire, indépendamment de la façon dont il sera implémenté.
- **PIM (Platform Independent Model)** : ce modèle représente l'architecture et les fonctionnalités du système sans faire référence à une technologie ou une plateforme donnée. Il constitue un pont entre les exigences métier et la solution technique.
- **PSM (Platform Specific Model)** : il traduit le PIM en une implémentation concrète, en intégrant les contraintes et les choix technologiques spécifiques (framework, langage, environnement d'exécution).

a. Application du MDD aux agents conversationnels

Dans le contexte des agents conversationnels, le MDD permet de modéliser les différents éléments du dialogue : intentions (intents), entités, règles de gestion, et enchaînements conversationnels (flux de dialogue). Cette modélisation peut être graphique (diagrammes, blocs) ou déclarative (fichiers de configuration, langages spécifiques).

Grâce au MDD, les experts métier peuvent formaliser les besoins sans connaissance approfondie en programmation, facilitant la collaboration avec les développeurs qui supervisent la génération automatique du chatbot. Cette méthode permet également d'assurer une meilleure cohérence et une maintenance simplifiée des agents conversationnels.

b. Travaux et études utilisant le MDD pour la creation des assistants

Plusieurs travaux ont mis en avant les bénéfices du MDD dans la conception d'agents conversationnels. Ces approches cherchent généralement à structurer le développement autour de modèles abstraits, facilitant ainsi l'automatisation, la maintenabilité et la réutilisabilité des composants conversationnels. Ci-dessous, nous présentons quelques études clés dans le contexte de notre recherche.

- Pérez-Medina et al. [1] présentent une revue systématique portant sur l'application des approches pilotées par les modèles (Model-Driven Development, MDD) dans la création d'agents conversationnels. À partir d'un corpus initial de 429 publications, les auteurs ont retenu 20 études primaires selon des critères d'inclusion précis, avec pour objectif d'identifier les méthodologies existantes, les bonnes pratiques, les outils et les cadres conceptuels utilisés dans ce domaine.

  Les résultats mettent en évidence la capacité du MDD à structurer le processus de conception conversationnelle (QR1), en distinguant clairement les intentions, les dialogues et les réponses. L'usage de langages spécifiques au domaine (DSL) et d'outils de génération de code témoigne d'un effort vers l'automatisation (QR3, QR6). La majorité des contributions identifiées s'orientent vers la définition de frameworks (55%), d'architectures (27%) et d'APIs (14%), ce qui montre une tendance à l'outillage structuré (QR2).

  Cependant, l'étude souligne plusieurs limites : faible prise en compte de la validation et de la personnalisation des assistants (QR5, QR9), manque de normes de développement, dépendance à des technologies propriétaires, et couverture limitée du langage naturel (QR4). Les enjeux liés à l'évaluation, à la qualité logicielle et à la maintenance sont également peu traités (QR7, QR8), ce qui restreint la pérennité des solutions proposées.
- L'article de Pérez-Soler et al. [28] intitule *Model-driven Chatbot Development* s'inscrit dans la continuité des travaux de Martinez garate et al. [1] proposant un cadre unifié pour industrialiser le développement et la migration des chatbots à travers une ingénierie dirigée par les modèles. Leur contribution repose sur trois piliers : un méta-modèle générique décrivant les composants essentiels d'un chatbot, un DSL appelé Conga pour modéliser indépendam-

ment de la plateforme cible, et un outil de recommandation pour guider le choix technologique selon des critères précis.

Cette approche répond directement à plusieurs enjeux identifiés : elle structure le processus de conception (QR1), renforce l'indépendance technologique (QR2, QR3), et automatise des tâches complexes comme la génération et la migration de chatbots (QR6). L'évaluation sur des cas réels de transfert entre Dialogflow et Rasa confirme la faisabilité du modèle proposé et son potentiel d'application concrète (QR5).

Cependant, des aspects clés restent peu explorés, notamment la prise en compte de la qualité des interactions, la gestion des erreurs utilisateurs (QR7) et l'interopérabilité avec des systèmes tiers (QR8). Par ailleurs, l'approche reste centrée sur des cas à domaine fermé, ce qui limite son adaptabilité dans des contextes conversationnels plus ouverts ou évolutifs (QR9).

- Dans cette même lancée, Martínez-Gárate et al.[1] approfondissent les limites des frameworks existants pour le développement d'agents conversationnels, en se concentrant sur Xatkit, et proposent une solution semi-automatisée orientée vers l'accessibilité, l'interopérabilité et l'intégration avec des plateformes populaires comme WhatsApp. Un apport particulièrement notable est l'intégration du modèle GPT-3, utilisé pour enrichir les réponses du bot dans les situations imprévues. Cela améliore la fluidité et la pertinence des interactions, en complétant les scénarios basés sur intentions avec des réponses génératives intelligentes.

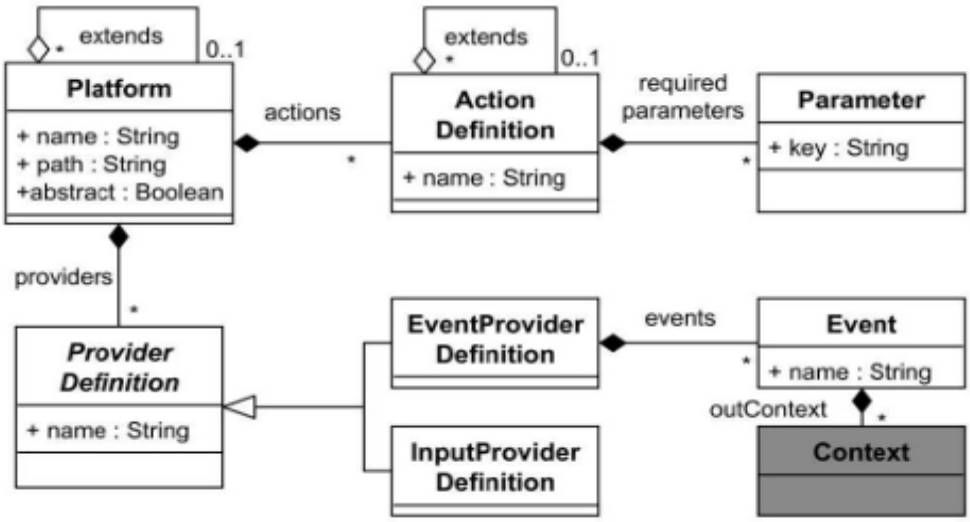


Figure 19 – *Métamodèle Xatkit proposé pour la génération automatique de chatbots[1].*

Leur approche repose sur un métamodèle étendu pour formaliser les composants du chatbot, un mécanisme de génération automatique de code Java via Acceleo, et l'intégration de Twilio pour permettre la communication via WhatsApp. L'originalité du travail réside aussi dans l'ajout de GPT-3 pour générer dynamiquement des réponses en cas de situations imprévues, augmentant ainsi la robustesse et la fluidité des interactions.

Cette contribution répond pleinement à QR1 (structuration de la conception), QR3 (outillage semi-automatisé) et QR6 (automatisation du passage modèle-code). L'ajout de GPT-3 touche également QR4, en intégrant une technique avancée d'IA pour enrichir les réponses. L'effort de simplification du développement, en s'appuyant sur une approche low-code, rend aussi la solution accessible à des profils non techniques (QR2, QR9).

Cependant, certaines limites subsistent. L'absence d'une interface graphique complète freine l'autonomie des utilisateurs non développeurs (QR9), l'interopérabilité reste limitée à certaines plateformes (QR8), et les aspects liés à l'évaluation, à la standardisation et à la qualité conversationnelle sont peu explorés (QR5, QR7).

- C'est dans la meme lancer que L'article de Jordi Cabot et al. [6] introduit Xatkit, un framework de développement low-code pour chatbots multimodaux, fondé sur les principes de l'ingénierie dirigée par les modèles (IDM) visant à simplifier et standardiser la création de chatbots multimodaux indépendants des plateformes.

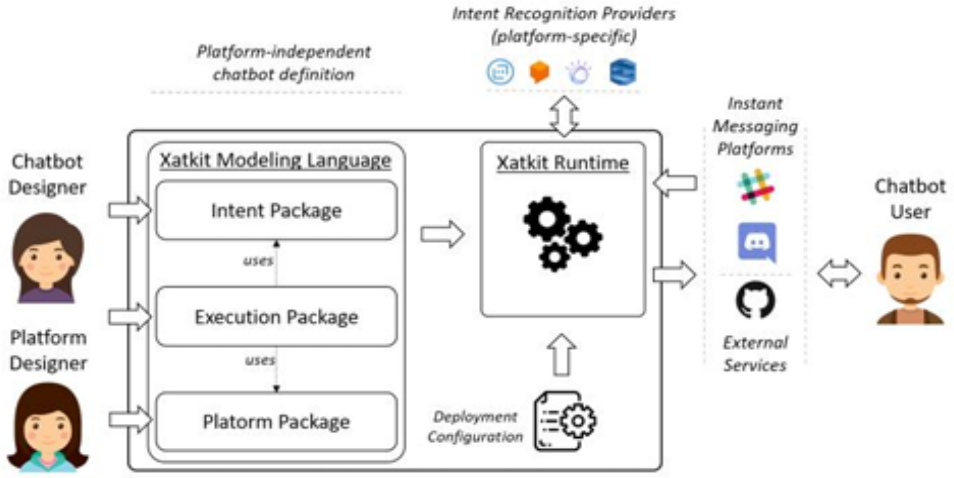


Figure 20 – *Présentation du framework Xatkit[6].*

L'approche repose sur un langage de modélisation dédié permettant de concevoir à la fois les intentions et la logique d'exécution, puis de générer automatiquement des bots déployables sur plusieurs canaux (Slack, Discord, GitHub...). Cette séparation claire entre conception et déploiement offre flexibilité et portabilité (QR1, QR2, QR3). Le moteur d'exécution assure l'automatisation du déploiement sans codage manuel (QR6), et l'étude de cas proposée montre la capacité du système à gérer des dialogues distribués et proactifs (QR5).

Cependant, l'absence d'interface graphique et le besoin de compétences techniques pour certaines extensions limitent l'accessibilité (QR9). Par ailleurs, les mécanismes de test, de validation et d'évaluation de la qualité restent peu développés (QR7, QR8), et la gestion des contextes ouverts n'est pas abordée en profondeur (QR4).

- Iyenghar et al.[14] proposent un chatbot assistant conçu pour faciliter l'usage des outils de modélisation dirigée par les modèles (MDE) dans le logiciel embarqué. Développé avec Rasa, cet assistant guide les utilisateurs à travers des tutoriels interactifs et adapte son aide selon le niveau de compétence, améliorant ainsi l'accessibilité et l'expérience utilisateur (QR1, QR9).

  Cette solution illustre une intégration directe de l'agent conversationnel dans l'outil de modélisation, favorisant la prise en main et la réduction des erreurs, tout en proposant une interaction naturelle multi-étapes (QR2, QR4). Néanmoins, cette approche soulève aussi plusieurs défis. La mise en place de tutoriels interactifs demande une configuration précise, surtout quand il s'agit de gérer plusieurs étapes ou des contextes variables. Le système reste encore difficile à faire évoluer de façon simple, notamment lorsqu'on veut ajouter ou modifier du contenu. Enfin, il n'existe pas encore de méthode claire pour évaluer l'efficacité de ce type d'assistant dans un usage réel, ce qui limite sa validation à grande échelle(QR5, QR7).

- Dans la même veine, [Koppanati[20] explore l'intégration de chatbots intelligents dans les pipelines CI/CD pour automatiser la surveillance et le reporting des tests logiciels en temps réel. L'étude met en avant l'utilisation de modèles de traitement automatique du langage naturel (TALN) pour faciliter la détection des erreurs, améliorer la communication entre développeurs et systèmes de test, et accélérer la prise de décision (QR1, QR6).

  Cette approche optimise le workflow de développement en réduisant les délais d'analyse, mais reste limitée au domaine du testing. Elle ne traite pas le fine-tuning des modèles de langage ni ne compare l'efficacité avec d'autres solutions existantes, ce qui restreint son champ d'application et la généralisation des résultats (QR4, QR7, QR8).

- Dans la continuité des travaux visant à automatiser la conception logicielle, [Kochbati[19] propose une approche hybride combinant traitement automatique du langage naturel (TALN) et apprentissage automatique pour transformer des exigences exprimées en langage naturel en modèles UML, dans un contexte de modélisation basée sur les modèles (MBSE).

  L'étude utilise des algorithmes comme Word2Vec et des heuristiques de TALN pour classifier et modéliser les spécifications, validant la méthode via un cas industriel. Cette démarche structure efficacement les exigences logicielles, apportant une automatisation prometteuse pour le passage des besoins vers l'architecture (QR1, QR4).

  Toutefois, cette recherche ne traite pas du fine-tuning des grands modèles de langage (LLMs) et se limite à un seul cas industriel, ce qui soulève des questions sur la généralisation de la méthode à d'autres domaines (QR7, QR8).

- Enfin, Les travaux de Sansonnet et al.[35] explorent une catégorie spécifique : les agents conversationnels d'assistance (ACA) destinés à accompagner les utilisateurs novices dans des environnements logiciels complexes.

  Ils proposent un cadre structuré autour de trois fonctions clés : la présence incarnée pour humaniser l'interaction, la compréhension des requêtes en langage naturel, et la compétence pour fournir des réponses adaptées via une

représentation symbolique du logiciel. Cette architecture repose sur un médiateur qui fait le lien entre l'agent et l'application, facilitant la généralisation de l'assistance (QR1, QR2).

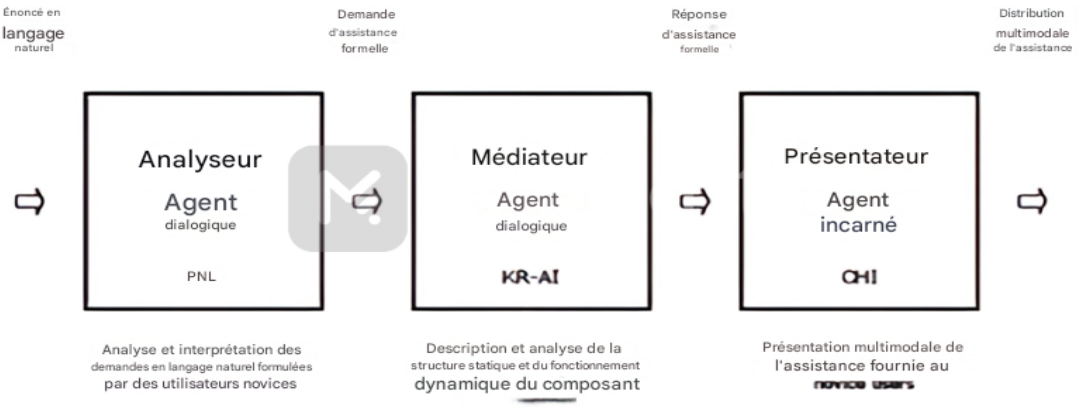


Figure 21 – *Architecture d'un cadre générique aux agents conversationnels [35].*

Trois stratégies d'implémentation sont identifiées : connecter l'agent à une application existante, développer simultanément l'agent et le logiciel, ou générer automatiquement les composants via des approches MDA. Cette démarche illustre une intégration poussée de l'IDM dans le développement des assistants (QR3, QR6).

Les limites portent principalement sur le coût élevé de développement, la maintenance des agents, et leur adaptation aux systèmes complexes, ce qui ouvre des pistes pour de futurs travaux sur la scalabilité et la robustesse (QR7, QR8, QR9).

II.3.2. Plateformes et Outils existants pour le fine-tuning et la création d'assistants

Aujourd'hui, créer un assistant conversationnel ne passe plus forcément par le développement manuel de bout en bout. Plusieurs outils ont été conçus pour rendre cette tâche plus rapide, plus accessible et plus adaptable, selon les besoins. On peut les regrouper en deux grandes familles : les plateformes low-code/no-code qui permettent de construire des bots avec peu ou pas de programmation, et les outils de fine-tuning qui permettent d'adapter des modèles de langage (LLMs) à des cas d'usage spécifiques.

II.3.2.1. Plateformes low-code / no-code : créer des chatbots sans coder

Ces plateformes sont conçues pour permettre à des utilisateurs même sans compétences en programmation — de concevoir des assistants en utilisant des interfaces visuelles. Elles reposent sur des blocs de construction simples : intentions, réponses, entités, logique de dialogue, et parfois des connexions avec des services externes (comme des APIs ou des bases de données).

- **Dialogflow (Google)** permet de créer des agents conversationnels à travers une interface très intuitive. On peut y définir des intentions (ex. : "réserver une table") et y associer des réponses ou actions. Dialogflow est souvent utilisé pour des assistants sur WhatsApp, Messenger ou des sites web. Il est aussi bien intégré à Google Cloud, ce qui facilite le déploiement [9].
- **Botpress** est une plateforme open source qui propose un éditeur visuel basé sur des flux et des événements. Contrairement à Dialogflow, il est plus personnalisable et peut être hébergé localement. Il permet de gérer des conversations complexes, d'intégrer des scripts, et de connecter facilement des services externes [3].
- **Rasa Studio**, intégré au framework Rasa, combine un éditeur graphique avec un moteur NLP open source. Il vise les développeurs qui veulent plus de contrôle sur la gestion du langage, tout en gardant un support visuel pour faciliter la conception des dialogues [32].

Ces plateformes permettent donc de créer rapidement des assistants pour des tâches précises, comme du service client, des bots de réservation, ou des FAQ interactives. Toutefois, elles ont des limites. Elles fonctionnent en général avec des intentions prédéfinies, et sont moins efficaces quand il s'agit de gérer des conversations ouvertes ou imprévues. Elles offrent peu de support pour l'évaluation systématique des performances des bots, et les scénarios complexes restent difficiles à maintenir sans expertise technique.

II.3.2.2. Outils existants pour le fine-tuning de LLMs

Les modèles de langage (comme GPT-3.5 ou GPT-4) sont très puissants, mais généralistes. Pour les rendre plus efficaces dans un domaine précis (par exemple la médecine, l'assurance, ou un service client), il est souvent nécessaire de les personnaliser, par un processus qu'on appelle fine-tuning.

- **OpenAI Fine-tuning API** permet d'ajuster GPT-3.5 ou GPT-4 sur ses propres données, en fournissant des paires d'exemples (ex. : question → réponse attendue). Ce processus permet d'obtenir des réponses plus adaptées à un cas métier, tout en gardant les capacités générales du modèle. Cela peut être utile pour des tâches où la formulation est spécifique ou où l'on veut éviter certaines erreurs [27].
- **Hugging Face Transformers + PEFT** : pour ceux qui travaillent en open source, la bibliothèque Transformers permet de charger et d'adapter de nombreux modèles de langage (LLaMA, Falcon, Mistral...). Grâce à des techniques comme LoRA (Low-Rank Adaptation), il est possible de faire un fine-tuning avec beaucoup moins de ressources, ce qui rend le processus plus accessible [13].

En somme, Les plateformes low-code/no-code et les outils de fine-tuning ont beaucoup fait avancer le domaine. Elles ont permis de rendre la création de bots plus rapide, plus accessible et parfois plus intelligente. Mais elles sont encore souvent isolées, c'est-à-dire qu'elles ne s'intègrent pas dans un processus complet allant de la modélisation à la maintenance. Elles ne proposent pas non plus de cadre structuré

pour évaluer, tester, ou faire évoluer un agent conversationnel comme le ferait une vraie démarche de génie logiciel. C'est justement sur ce point que se démarquent les approches MDD, et c'est aussi là que se positionne notre proposition.

II.3.3. Intégration des LLMs dans ces méthodologies

L'arrivée des modèles de langage de grande taille (LLMs), comme GPT-4, Claude, LLaMA ou Mistral, a profondément modifié la manière de concevoir des chatbots. Ces modèles, capables de comprendre des consignes complexes et de produire des réponses fluides en langage naturel, offrent une grande souplesse. Contrairement aux anciens systèmes qui reposaient sur des règles et des scénarios figés, les LLMs peuvent gérer des demandes imprévues ou mal formulées, ce qui change complètement la logique de développement d'un assistant.

Aujourd'hui, plusieurs façons d'utiliser les LLMs dans les assistants conversationnels existent. La plus courante consiste à les utiliser en complément d'un moteur classique, lorsque ce dernier ne comprend pas l'intention de l'utilisateur. On parle ici de fallback, c'est-à-dire que le LLM prend le relais lorsque les règles habituelles échouent. Des plateformes comme Botpress proposent déjà ce type d'intégration avec des modèles comme GPT-4 ou Claude[4].

Une autre façon de les utiliser est de placer le LLM au cœur du système, comme moteur principal. Dans ce cas, il génère directement les réponses à partir de consignes (ou prompts) construites à la volée. Des bibliothèques comme LangChain ou LlamaIndex permettent justement de structurer ce type de fonctionnement. Par exemple, on peut demander au modèle de chercher d'abord des documents dans une base, puis de formuler une réponse à partir de ces informations [21].

Enfin, certains développeurs utilisent les LLMs dans un mode appelé RAG (Retrieval-Augmented Generation). Cette méthode consiste à combiner un moteur de recherche interne avec le modèle. Concrètement, on récupère d'abord des contenus (par exemple dans une base de données), puis on les fournit au LLM dans le prompt. Cela permet d'obtenir des réponses précises sans avoir besoin de réentraîner le modèle.

Malgré tous ces avantages, les LLMs ne remplacent pas totalement les approches structurées comme le MDD. D'abord, ils restent peu contrôlables : ils peuvent produire des erreurs ou des réponses non adaptées au contexte. Ensuite, ils sont encore difficiles à intégrer dans une logique de conception complète : comment tester un chatbot si ses réponses varient à chaque exécution ? Comment modéliser un comportement basé sur des prédictions probabilistes ? Enfin, la majorité des outils MDD actuels ne sont pas pensés pour accueillir ce type de modèles dans leur cycle de vie.

En résumé, les LLMs permettent de créer des chatbots plus intelligents et plus ouverts, mais leur intégration dans des méthodologies robustes reste un défi. Très peu d'approches proposent aujourd'hui une manière claire et stable de combiner les avantages des LLMs avec la rigueur des modèles MDD. Ce constat confirme la nécessité d'explorer de nouveaux cadres hybrides, capables de tirer parti des deux mondes.

II.4. Synthèse critique et positionnement de la recherche

Les travaux étudiés plus haut montrent que les approches dirigées par les modèles (MDD) apportent une vraie valeur ajoutée dans la création de chatbots. Elles permettent de mieux structurer le processus de développement en séparant clairement la logique du bot (ce qu'il doit faire) du code technique (comment il le fait). Cette séparation rend le travail plus rapide, plus fiable, et plus facile à partager entre différents profils, même non développeurs.

L'article de Pérez-Medina et al. [1] offre une vue d'ensemble solide des pratiques actuelles. Il montre que les DSLs (langages spécifiques au domaine) sont largement utilisés pour modéliser les intentions, les dialogues et les réponses. Ces outils permettent d'automatiser une partie du développement, ce qui améliore la productivité. Mais l'étude met aussi en évidence des faiblesses importantes : peu d'outils pour tester les bots, très peu de normes partagées, et des difficultés à gérer le langage naturel de façon efficace. Tout cela freine l'adoption du MDD à grande échelle.

L'article de Pérez-Soler et al. [28] tente de dépasser ces blocages en proposant un métamodèle indépendant des plateformes. Cette idée est intéressante car elle permet de concevoir un chatbot une seule fois, puis de le déployer sur différents outils (comme Dialogflow, Rasa, etc.) sans tout refaire. Mais l'approche reste très théorique. Elle ne prend pas bien en charge les conversations complexes, ni les cas où l'utilisateur sort du scénario prévu.

L'étude de Martínez-Gárate et al. [1] apporte un exemple plus concret en enrichissant le framework Xatkit avec l'intégration de WhatsApp et de GPT-3. Cela permet aux bots d'être plus intelligents, plus réactifs et plus proches des besoins réels des utilisateurs. Mais là aussi, certaines limites reviennent : peu d'outils de test, pas d'interface complète pour les non-développeurs, et des modèles qui restent pensés pour des cas bien cadrés, peu évolutifs.

Enfin, l'article de Daniel et al. [7] présente une version plus avancée de Xatkit. Cette fois, le chatbot est entièrement modélisé, exécuté par un moteur indépendant, et capable de réagir à des événements extérieurs. Cela marque une vraie avancée dans l'automatisation et la modularité. Mais malgré cette souplesse, l'absence d'interface graphique conviviale et le manque d'outils pour évaluer la qualité des conversations restent des freins importants, surtout pour un usage professionnel.

Table IV – *Couverture des questions de recherche par les études analysées*

Étude / Article	**QR1**	**QR2**	**QR3**	**QR4**	**QR5**	**QR6**	**QR7**	**QR8**	**QR9**
Pérez-Medina et al.[1]	✓	✓	✓	✓					✓
Pérez-Soler et al.[28]	✓	✓	✓	✓					
Martínez-Gárate et al.[1]	✓	✓	✓	✓		✓			✓
Daniel et al.[6]	✓	✓	✓	✓		✓			
Iyenghar et al.[14]				✓					
Kochbati et al.[18]				✓					
Koppanati et al.[20]				✓		✓			
Sansonnet et al.[35]					✓		✓		✓

Ce tableau met en évidence la couverture des différentes questions de recherche par les études analysées. Afin d'expliciter les liens entre chaque étude et les questions qu'elle adresse, nous présentons ci-dessous un tableau de justifications. Celui-ci précise, pour chaque coche observée, les éléments techniques ou méthodologiques qui motivent l'association à une ou plusieurs questions de recherche. Cette démarche renforce la rigueur de notre revue systématique et permet de mieux positionner notre propre contribution par rapport à l'état de l'art.

Table V – *Justifications des couvertures des questions de recherche par étude*

Étude	QR	Justification
Pérez-Medina et al.[1]	QR1	Recense et synthétise les méthodologies utilisées pour développer des chatbots via MDD.
	QR2	Compare les phases MDD aux étapes classiques d'ingénierie logicielle.
	QR3	Analyse l'usage de DSLs et d'outils de génération de code.
	QR4	Mentionne l'intégration de techniques d'IA, notamment pour la génération.
	QR9	Aborde la séparation entre intentions, dialogues et réponses, améliorant l'adaptation utilisateur.
Pérez-Soler et al.[28]	QR1	Propose une démarche structurée pour modéliser et migrer les chatbots.
	QR2	Le métamodèle respecte les principes de conception logiciel.
	QR3	Introduit Conga, un DSL et un outil de recommandation de plateforme.
	QR4	Intègre des mécanismes de suggestion intelligente (IA symbolique + contraintes).
Martínez-Gárate et al.[1]	QR1	Étend Xatkit avec une architecture semi-automatisée pour créer des bots intelligents.
	QR2	Structure la chaîne depuis le modèle jusqu'au déploiement avec principes IDM.
	QR3	Utilise Acceleo pour générer du code à partir des modèles.
	QR4	Intègre GPT-3 pour des réponses génératives.
	QR6	Génération de code Java et déploiement WhatsApp automatisés.
	QR9	Améliore l'adaptabilité et la fluidité des réponses utilisateur.
Daniel et al.[6]	QR1	Définit une méthode modulaire pour construire des chatbots multi-plateformes.
	QR2	Respecte les étapes de conception, d'exécution et d'extension logicielle.
	QR3	Introduit un framework complet (Xatkit) low-code et extensible.
	QR4	Prise en charge de moteurs NLP (Dialogflow, Watson).
	QR6	Moteur d'exécution gérant automatiquement les plateformes cibles.
Iyenghar et al.[14]	QR4	Utilise le framework Rasa pour interpréter les intentions et adapter l'aide via NLP.
Kochbati et al.[18]	QR4	Applique Word2Vec et apprentissage automatique pour transformer des exigences textuelles.
Koppanati et al.	QR4	Système fondé sur NLP pour analyser les logs de tests.
	QR6	Automatisation du reporting dans les pipelines CI/CD.
Sansonnet et al.[35]	QR5	Conçoit un modèle agentiel pour aligner les réponses avec les actions prévues.

En résumé, ces travaux montrent tous que le MDD est une bonne base, mais qu'il ne suffit pas. On peut modéliser, générer du code, réutiliser des structures... mais il manque encore des briques essentielles : des outils de test fiables, des indicateurs de qualité, des moyens de personnaliser les bots selon le contexte métier, et surtout des solutions pour faire évoluer les agents dans le temps. Aujourd'hui, la majorité des approches restent adaptées à des bots simples, conçus pour des scénarios fermés.

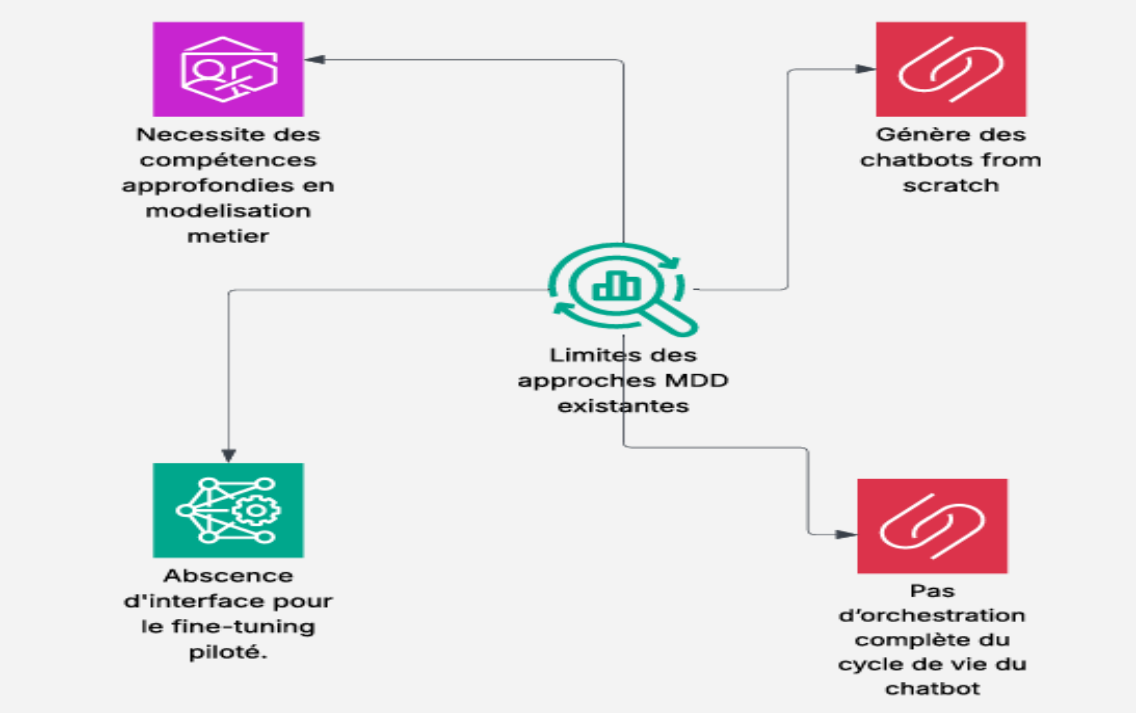


Figure 22 – *Limites des approches existantes*

C'est précisément sur ce vide que vient se positionner ce travail. L'objectif n'est pas de proposer une énième plateforme MDD, mais de concevoir un véritable atelier de développement, combinant les bonnes pratiques du génie logiciel avec les possibilités offertes par les grands modèles de langage (LLMs). Cet atelier vise à rendre la création de chatbots plus souple, plus modulaire, plus proche des besoins du terrain. Il intègre la modélisation, la génération, mais aussi les tests, la personnalisation, et la connexion à des services réels via des API. Ce projet se veut donc une réponse complète, pensée non seulement pour concevoir des agents conversationnels, mais aussi pour les faire vivre, évoluer, et s'adapter durablement à leur environnement.

II.5. Conclusion

À la lumière des travaux analysés, une conviction s'impose avec force : les approches dirigées par les modèles (MDD) représentent aujourd'hui l'une des tentatives les plus sérieuses pour professionnaliser le développement de chatbots. Elles facilitent la modélisation, réduisent la charge technique, et permettent une meilleure réutilisabilité.

Cependant, elles ne couvrent pas encore tous les besoins réels du terrain. Les limites sont nettes : faible prise en compte du langage naturel, absence d'outils de test et d'évaluation, personnalisation rigide, et peu de solutions pensées pour la maintenance ou l'évolution des bots dans le temps. Ces constats révèlent un manque d'outillage global pour accompagner le cycle de vie complet des chatbots.

C'est sur ce constat que se fonde notre contribution. Le chapitre suivant présente une architecture complète pour un atelier de développement d'agents conversationnels, pensée comme une extension des approches MDD. Elle intègre les bonnes pratiques du génie logiciel, les capacités des modèles de langage actuels (LLMs), et les outils nécessaires pour concevoir, tester et faire évoluer les chatbots dans des environnements réels.

Chapitre III

ARCHITECTURE D'UN ATELIER DE CRÉATION D'ASSISTANTS CONVERSATIONNELS BASÉS SUR DES MODÈLES DE LANGAGE LARGES

Sommaire

III.1 - Introduction 50
III.2 - Objectifs de conception de l'atelier et vision systémique 51
III.3 - Architecture fonctionnelle proposée(Vue utilisateur) 54
III.4 - L'architecture logicielle interne de l'atelier (Vue développeur) 65
III.5 - Communication entre modules et persistance des données 67
III.6 - Validation théorique de l'architecture proposée 69
III.7 - Validation expérimentale simulée (preuve de concept) 70
III.8 - Conclusion 76

III.1. Introduction

Tout système qui prétend maîtriser la complexité doit d'abord en révéler la structure. Concevoir un assistant conversationnel intelligent n'est pas un simple enchaînement de scripts ou de modèles : c'est une démarche d'ingénierie. Elle exige une vision claire, une méthode structurée, et des fondations techniques solides.

Ce chapitre présente l'architecture d'un atelier de genie logiciel pensé pour encadrer la création systématique d'agents conversationnels par fine-tuning de LLMs. Notre démarche repose sur deux piliers complémentaires.

D'un côté, un workflow fonctionnel, guidé étape par étape, organise l'expérience utilisateur en modules clairs de la définition des intentions jusqu'à la génération du

modèle prêt à l'emploi. De l'autre, une architecture logicielle interne, fondée sur les principes du génie logiciel, alimente cette fluidité apparente en assurant la robustesse, la cohérence et la traçabilité de chaque projet.

En dévoilant les couches visibles et invisibles de cet atelier, ce chapitre affirme une conviction forte : le dialogue intelligent ne peut émerger que d'une construction rigoureuse, où chaque composant logiciel sert une finalité claire. Ici, l'architecture n'est pas un détail technique elle est la condition de l'intelligence.

III.2. Objectifs de conception de l'atelier et vision systémique

L'atelier que nous proposons vise à rendre accessible la création d'assistants conversationnels intelligents, en s'appuyant sur le fine-tuning des modèles de langage de grande taille (LLMs). Ce système ne cherche pas seulement à centraliser des outils techniques, mais à structurer un espace complet, pensé pour guider pas à pas les utilisateurs, quel que soit leur niveau, dans la conception d'agents conversationnels adaptés à leurs besoins métiers. Ainsi, nous présenterons ci-dessous le public cible, les objectifs fondamentaux qui ont guidé la conception de notre atelier, tant sur le plan fonctionnel que non fonctionnel et également la méthodologie adoptée, structurée selon une approche rigoureuse, progressive et centrée sur l'utilisateur.

III.2.1. Public cible : Un atelier pour un concepteur conversationnel assisté

L'atelier que nous proposons a été conçu pour un profil d'utilisateur bien précis, que nous appelons le concepteur conversationnel assisté. Il ne s'agit pas ici d'un expert en intelligence artificielle, ni d'un ingénieur spécialisé dans les modèles de langage, mais plutôt d'un utilisateur – technique ou non – qui souhaite créer un assistant conversationnel intelligent sans se perdre dans des détails complexes.

Ce concepteur peut être un développeur généraliste, un expert métier, un enseignant, un étudiant, ou même un passionné d'automatisation. Ce qui compte, ce n'est pas son profil technique, mais sa volonté de concevoir un agent capable de comprendre le langage naturel et d'interagir avec un système existant.

L'une des idées fondatrices de l'atelier est justement de rendre ce processus universel. Tous les utilisateurs suivent le même cheminement : un workflow progressif, balisé étape par étape, qui ne demande aucune connaissance préalable en IA. Le concepteur n'a pas à comprendre les rouages du fine-tuning, ni les structures internes des modèles ; il interagit avec une interface pensée pour le guider intuitivement, en s'appuyant sur des formulaires, des suggestions automatiques, et une organisation logique du travail.

Autrement dit, l'atelier transforme ce qui était autrefois réservé à une élite technique en une expérience de conception accessible et structurée, où chacun peut bâtir son assistant à son image, avec ses propres objectifs métier, sans dépendre d'une équipe de data scientists.

III.2.2. Finalités fonctionnelles : faire du LLM un outil à portée de tous

La principale finalité fonctionnelle de notre atelier est de permettre à un utilisateur, quel que soit son niveau technique, de créer un chatbot personnalisé capable de comprendre le langage naturel, de produire des réponses structurées et de communiquer avec un système informatique existant. Pour atteindre cet objectif, l'atelier repose sur une chaîne d'actions cohérente, pensée comme un parcours progressif.

- **Initialisation du projet** : l'utilisateur peut créer un nouveau projet en renseignant un nom, une description, le modèle LLM à utiliser (ex. GPT-3.5), ainsi que les informations de connexion nécessaires (clé API, fournisseur, etc.). Un environnement de travail dédié est automatiquement généré.
- **Définition du système métier cible** : l'atelier permet d'importer un fichier Swagger/OpenAPI décrivant les entités et les actions disponibles dans un système externe (ex. système de gestion de facturation). L'utilisateur peut aussi saisir ces informations manuellement via une interface guidée.
- **Modélisation des intentions conversationnelles** : l'utilisateur indique les types de requêtes que le chatbot devra comprendre (ex. créer une facture, rechercher un client), et les associe aux actions réelles du système métier.
- **Génération d'exemples de conversation** : à partir des intentions définies, l'atelier propose automatiquement des formulations variées en langage naturel. L'utilisateur peut les valider, les modifier ou les enrichir.
- **Mappage entre phrases et actions** : chaque exemple de phrase est associé à une commande structurée (au format JSON, par exemple), correspondant à une action que le chatbot doit déclencher sur le système cible.
- **Préparation des données d'entraînement** : les paires validées (phrase → commande) sont formatées automatiquement dans un format compatible avec le modèle sélectionné (JSONL, CSV, etc.).
- **Entraînement du modèle** : l'utilisateur peut configurer les paramètres d'apprentissage (nombre d'époques, taille de lot, température, etc.) et lancer le fine-tuning du LLM directement via l'atelier.
- **Test du chatbot entraîné** : une interface permet de dialoguer avec l'agent conversationnel et d'observer les actions générées en réponse. Cela facilite l'évaluation de la compréhension et la détection des erreurs.
- **Amélioration continue** : l'utilisateur peut corriger manuellement les erreurs du chatbot, ajouter de nouveaux exemples, et relancer un entraînement à partir d'un corpus enrichi.
- **Déploiement du chatbot** : l'assistant peut être déployé sous forme d'API REST, de microservice Dockerisé, ou intégré dans une interface web.
- **Exportation et versioning du projet** : l'utilisateur peut sauvegarder l'intégralité du projet (modèle entraîné, données, configuration) sous forme d'archive, avec un identifiant de version, une documentation générée automatiquement, et un hash d'intégrité.

Chaque fonctionnalité est conçue pour réduire la complexité, guider sans enfermer, et favoriser une appropriation progressive des capacités offertes par les modèles de langage.

III.2.3. Contraintes non fonctionnelles : robustesse, modularité et évolutivité

Au-delà des fonctions visibles, l'atelier a été conçu pour répondre à des exigences de qualité logicielle.

La première de ces exigences est la modularité. Chaque composant du système a été conçu comme une entité indépendante, ce qui permet d'en modifier ou d'en améliorer un sans risquer de perturber le fonctionnement des autres. Cette séparation des responsabilités favorise non seulement la maintenance, mais aussi la réutilisabilité des modules dans d'autres contextes.

À cette modularité s'ajoute la robustesse, qui désigne la capacité du système à fonctionner de manière stable, même lorsqu'il est confronté à des entrées incomplètes, incohérentes ou inattendues. L'atelier doit être capable de gérer ces situations sans interruption, en produisant des messages d'erreur explicites, informatifs et non bloquants. Une attention particulière a été portée à la gestion des exceptions et à la validation des données, afin de limiter les comportements indésirables.

Un autre principe central est l'accessibilité. L'atelier s'adresse à un public large, qui n'a pas nécessairement une expertise technique poussée. Son interface a donc été pensée pour être intuitive et fluide. Toutes les opérations proposées, qu'il s'agisse du paramétrage initial, de la génération de données ou du déploiement final, doivent pouvoir être réalisées sans écrire une seule ligne de code. Ce choix démocratise l'usage des modèles de langage et rend leur exploitation possible dans des environnements éducatifs, institutionnels ou professionnels peu techniques.

L'interopérabilité constitue également une exigence forte. Le système est conçu pour s'adapter à plusieurs fournisseurs de modèles LLM, tels que OpenAI, HuggingFace ou Cohere. Il doit également être capable de manipuler différents formats de données standards, comme JSON, JSONL, CSV, ou Swagger/OpenAPI, et de s'intégrer sans friction dans divers environnements de développement.

Par ailleurs, l'atelier doit répondre à une exigence de scalabilité. Cela signifie qu'il doit pouvoir gérer efficacement plusieurs projets à la fois, avec des volumes de données croissants, sans que les performances ne se dégradent. Que l'utilisateur travaille sur un assistant simple ou sur une dizaine de projets complexes, la fluidité de l'expérience doit rester constante.

Enfin, la question de la traçabilité et du versioning est au cœur de l'architecture du système. Chaque projet, chaque entraînement de modèle, et chaque jeu de données est enregistré avec des métadonnées complètes : identifiant unique, date de création, description, paramètres utilisés et résultats obtenus. Cette organisation permet de revenir facilement à une version précédente, de comparer les performances entre différentes itérations, et de garantir la reproductibilité des résultats.

III.2.4. Démarche de conception adoptée

III.2.4.1. Cycles itératifs et validation incrémentale

Dans une approche traditionnelle, on développe souvent un système dans son ensemble, puis on attend la fin du processus pour le tester. Cette méthode, bien qu'efficace dans certains contextes, présente de nombreux risques dans un projet comme le nôtre, qui touche à la fois des technologies complexes (LLMs) et des profils utilisateurs très variés (développeurs, experts métiers, pédagogues, etc.).

Pour pallier ces risques, nous avons adopté une stratégie de validation incrémentale reposant sur des cycles courts, itératifs, et orientés vers l'utilisateur. Cette méthode consiste à développer chaque fonctionnalité de manière isolée, à la tester de façon ciblée, puis à l'intégrer dans le système global une fois validée. Le processus suit quatre étapes principales :

1. **Développement séparé des fonctionnalités** Chaque partie importante de l'atelier (par exemple : création automatique de réponses, connexion à une API, ou mise en ligne du chatbot) est d'abord conçue et testée seule, dans un petit environnement de test. Cela permet de mieux comprendre ce que fait chaque partie sans être gêné par le reste, corriger plus facilement les erreurs et de vérifier que ça fonctionne bien avant de tout assembler.

   Pendant cette étape, on peut aussi se concentrer sur un seul objectif à la fois : rendre la fonction plus simple à utiliser, plus claire ou plus utile.

2. **Ajout progressif dans l'atelier complet** Quand une fonctionnalité est corrigée et validée, on l'intègre dans l'atelier général.

   Cependant, cette intégration ne se fait que si elle est stable (pas de bugs), elle fonctionne bien avec les autres parties déjà en place et elle reste facile à utiliser pour l'utilisateur final.

   Ainsi, l'atelier se construit petit à petit, avec des éléments solides qui fonctionnent bien ensemble.

Cette méthode de travail présente plusieurs avantages importants pour la conception de l'atelier. Tout d'abord, elle permet de garantir une stabilité du système, car chaque fonctionnalité est ajoutée une par une, ce qui réduit les risques de conflits ou de dysfonctionnements entre les différentes parties du projet. Ensuite, elle assure une cohérence fonctionnelle, puisque chaque module est testé dans un contexte réel d'utilisation (métier ou pédagogique), ce qui permet de confirmer sa pertinence et son utilité concrète. Enfin, cette approche contribue à offrir une expérience utilisateur fluide, car l'atelier peut évoluer progressivement sans devenir complexe. Chaque ajout est pensé pour s'intégrer naturellement aux fonctions existantes, en respectant la logique et la simplicité globale de l'outil.

III.3. Architecture fonctionnelle proposée(Vue utilisateur)

Concevoir un agent conversationnel basé sur des modèles de langage de grande taille (LLMs) est une tâche qui, en général, nécessite des compétences spécialisées en

traitement du langage naturel et en génie logiciel. L'ambition de notre atelier est de rendre cette démarche accessible à un public plus large, en proposant un processus guidé, progressif et modulaire. Ce workflow se matérialise sous la forme d'un enchaînement d'étapes successives, chacune correspondant à une fonction précise dans le cycle de vie du chatbot.

Ce parcours utilisateur ne reflète pas directement l'architecture technique du système, mais plutôt la logique fonctionnelle vécue par un utilisateur qui souhaite créer un assistant intelligent adapté à un cas d'usage métier concret. Chaque étape est pensée pour réduire la complexité, faciliter les prises de décision, et favoriser la production d'un agent opérationnel, personnalisable et déployable.

III.3.1. Vue d'ensemble du processus

Le processus de création guidée dans l'atelier se compose de huit modules successifs. Ensemble, ils couvrent toutes les phases classiques du développement logiciel, de l'expression des besoins au déploiement final

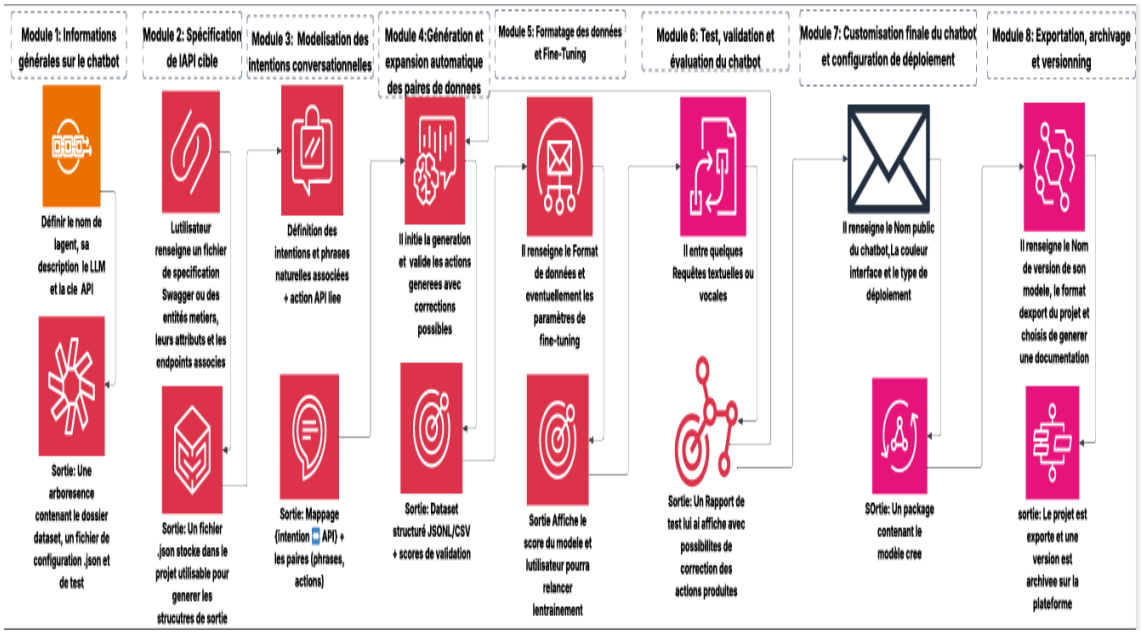


Figure 23 – *Workflow general de creation d'un assiststant conversationnel dans l'atelier*

III.3.2. Description des Modules fonctionnels du workflow conversationnel

1. Initialisation du projet

L'initialisation du projet constitue la première phase fondamentale du processus de création d'un assistant conversationnel au sein de l'Atelier de Génie Logiciel (AGL), une plateforme conçue pour un public diversifié, incluant développeurs, ingénieurs et utilisateurs novices. Cette étape transforme les intentions initiales en un environnement structuré, préparant le terrain pour le fine-tuning d'un modèle

de langage (LLM) et l'intégration avec un système existant. Chaque mécanisme est minutieusement défini pour garantir une progression logique et une traçabilité optimale, alignée avec les principes du Model-Driven Development (MDD). Voici une description exhaustive des composantes et des processus impliqués, détaillant comment chaque action est réalisée et validée.

Le processus s'amorce avec l'entrée des paramètres de base via une interface graphique dédiée(voir figure 24), où l'utilisateur saisit un nom unique pour le projet dans un champ texte. Cette action déclenche une validation immédiate orchestrée par un module de contrôle interne, qui interroge un fichier de registre, tel que `projects_index.json`, pour confirmer l'absence de doublons parmi les projets existants. En cas de conflit, une alerte textuelle s'affiche, invitant à opter pour une alternative, tandis qu'une zone de texte libre permet d'ajouter une description, un élément optionnel mais crucial pour documenter l'objectif fonctionnel. Une assistance contextuelle, intégrée sous forme d'outils d'aide en ligne, propose des exemples génériques, garantissant une accessibilité universelle sans exiger de compétences techniques préalables.

De cette fondation s'élève la sélection d'un modèle de langage, accessible via une liste déroulante, qui active une consultation d'une base de données interne, `llm_specs.json`. Cette ressource associe chaque modèle à ses caractéristiques, telles que le fournisseur et le format de données attendu (par exemple, "jsonl" pour les modèles OpenAI), une information extraite et stockée pour guider les étapes de préparation des données d'entraînement. Simultanément, l'utilisateur fournit une clé API dans un champ sécurisé, masquée par une icône de masquage visuel. La validation de cette clé s'appuie sur un framework web léger, Flask, configuré pour tester la connexion en temps réel. Flask exécute une requête HTTP GET vers l'endpoint du fournisseur du LLM sélectionné (par exemple, `/models` pour OpenAI), avec un délai maximal de cinq secondes. Une réponse HTTP 200 valide la clé, tandis qu'un code 401 ou 403, ou un timeout, génère une erreur spécifique affichée à l'écran, accompagnée d'une invitation à corriger l'entrée. Une fois validée, la clé est chiffrée à l'aide de la bibliothèque `cryptography` avec l'algorithme AES-256, avant d'être enregistrée dans un fichier de configuration, assurant la protection des données sensibles.

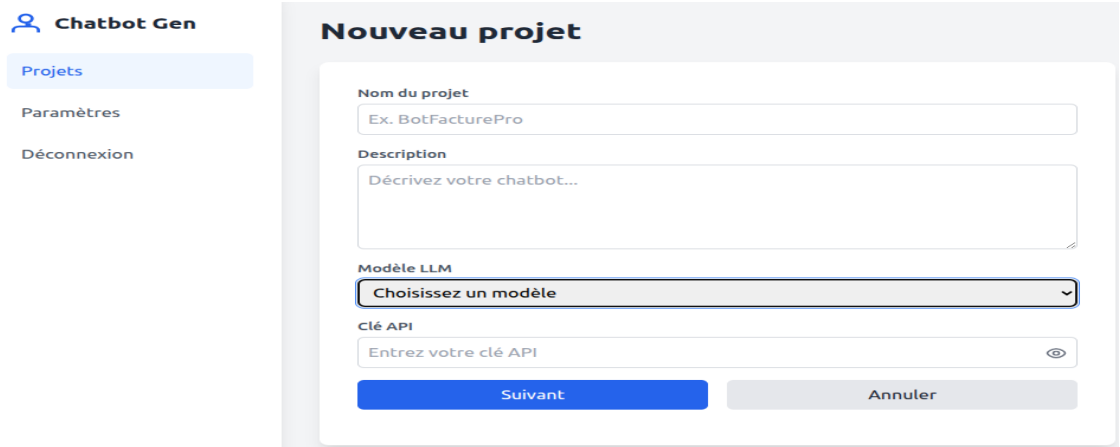


Figure 24 – *Interface d'initialisation d'un nouveau projet : saisie des paramètres et validation en temps réel.*

Sur cette base validée s'érige la création automatique d'un espace de travail structuré, un processus piloté par un gabarit défini dans un fichier `project_template.yaml`. Ce modèle spécifie la hiérarchie des dossiers et fichiers à générer, comme suit :

```yaml
project_root: /projets/nom_du_projet
files:
  - name: meta.yaml
    content:
      name: nom_du_projet
      description: description_saisie
      llm: modele_choisi
      date: date_actuelle
  - name: llm_config.json
    content:
      api_key: cle_chiffree
      format: format_modele
      endpoint: point_acces
  - name: data/training
  - name: data/test
  - name: data/validation
  - name: logs/ validation_log.txt
  - name: docs/README.md
```

Un générateur d'artefacts, intégré au moteur MDD, remplace dynamiquement les placeholders (comme `nom_du_projet` ou `modele_choisi`) par les valeurs fournies, puis crée les dossiers et initialise les fichiers avec leurs contenus respectifs. Par exemple, le fichier `meta.yaml` intègre les métadonnées exactes saisies, tandis qu'un fichier `README.md` émerge dans le dossier `docs/` pour documenter le projet dès son lancement, incluant des détails comme le nom, le modèle choisi et la date de création. Cette structure modulaire, générée de manière automatisée, soutient la gestion et l'évolutivité tout au long du cycle de développement.

La finalisation de cette phase s'accompagne d'une journalisation rigoureuse, où chaque action est consignée dans un fichier situé dans le dossier `logs/`, avec un horodatage précis (par exemple, "2025-07-01 03 :08 WAT - Projet initialisé"). Cette traçabilité permet de suivre l'historique des opérations et de diagnostiquer d'éventuels problèmes. Le statut du projet est mis à jour dans `meta.yaml` comme "initialisé", signalant que l'environnement est prêt. L'utilisateur est alors redirigé vers la phase suivante, où la configuration de l'intégration avec un système existant pourra être entreprise, assurant une transition fluide vers les étapes de modélisation et de fine-tuning

Cette phase initiale illustre la capacité de l'AGL à structurer un projet de manière systématique, en s'appuyant sur des outils comme Flask pour la validation et des gabarits MDD pour la génération. Les mécanismes de validation multicouches et la création automatisée d'une documentation initiale renforcent la robustesse du processus. Toutefois, une dépendance à la connectivité réseau pour la validation de la clé API pourrait constituer une limite. Une solution alternative, comme une

simulation hors ligne en cas d'échec, pourrait être envisagée pour pallier ce risque, garantissant ainsi une flexibilité accrue dans divers contextes d'utilisation.

2. Déclaration du système cible

La spécification de l'API constitue une étape pivot dans le cadre de l'Atelier de Génie Logiciel (AGL)t. Cette phase traduit les exigences d'interaction avec une API métier en un schéma structuré, essentiel pour guider le fine-tuning d'un modèle de langage (LLM) et assurer des dialogues cohérents avec des actions concrètes.

L'opération s'engage avec la définition des spécifications via une interface duale, offrant deux voies distinctes. La première option repose sur l'importation d'un fichier de spécification(voir figure 25) au format Swagger ou OpenAPI, téléversé à travers un champ dédié d'upload de fichiers. Cette action active un parseur intégré, basé sur la bibliothèque `pyyaml` pour les fichiers `.yaml` ou `json_ schema` pour les `.json`, qui analyse la syntaxe et extrait les entités, endpoints et méthodes définis. Une validation initiale vérifie la conformité du fichier avec le schéma OpenAPI (version 2.0 ou 3.0)Si le LLM GPT a ete selectionne ; à l'aide de `jsonschema`, signalant toute erreur (par exemple, un endpoint manquant) via une alerte à l'écran. La seconde option(voir figure 26), la configuration manuelle, invite l'utilisateur à saisir des informations dans des champs spécifiques : un nom d'entité (ex. : "Facture"), une liste d'attributs (ex. : "id", "montant") avec des types de données (chaîne, numérique), un endpoint (ex. : "/factures"), une méthode HTTP (GET, POST, etc.), et un payload au format JSON (ex. : `{"client_id": "1", "montant": 100}`). Chaque entrée est validée en temps réel : les attributs doivent inclure au moins un champ obligatoire, et l'endpoint est testé pour sa syntaxe (ex. : début par "/").



Figure 25 – *Importation d'un fichier Swagger : téléversement et validation syntaxique.*

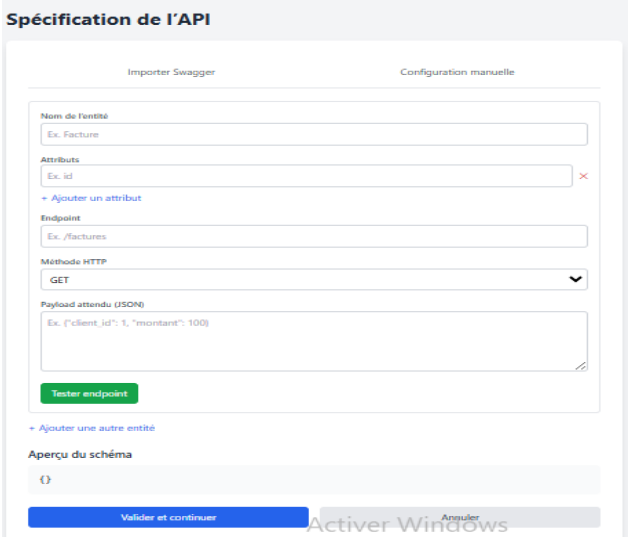


Figure 26 – *Configuration manuelle : saisie des endpoints et test de connexion.*

À partir de ces données, l'atelier génère un schéma structuré au format JSON, une opération orchestrée par un moteur de transformation MDD. Ce schéma, stocké

dans un fichier `api_schema.json`, mappe les entités aux endpoints et méthodes, par exemple :

```
{
  "entities": {
    "Facture": {
      "attributes": {"id": "string", "montant": "number"},
      "endpoint": "/factures",
      "method": "GET",
      "payload": {"client_id": "string", "montant": "number"}
    }
  }
}
```

La génération s'accompagne d'une validation croisée : un outil comme `requests` exécute une requête HTTP sur l'endpoint spécifié (ex. : GET `/factures`) avec la méthode choisie, vérifiant un code de réponse 200 ou 404, et logue les résultats dans un fichier `validation_log.txt`. Toute incohérence (endpoint inaccessible, payload mal formé) déclenche une notification, permettant une correction immédiate. Ce schéma sert de référence pour associer les intentions conversationnelles (ex. : "consulter une facture") aux appels API, préparant ainsi les données d'entraînement pour le fine-tuning.

L'étape se conclut par l'intégration de ce schéma dans l'espace de travail, où il est lié aux métadonnées du projet dans `meta.yaml`. Un système de journalisation enregistre chaque action avec un horodatage (ex. : "2025-07-01 03 :48 WAT Schéma API généré"), assurant une traçabilité complète. Cette structure standardisée permet la modélisation conversationnelle, où les interactions avec le système cible seront définies avec précision.

3. Modélisation des intentions conversationnelles

Cette phase établit un pont entre les expressions humaines et les actions définies dans l'API du module précédent, préparant ainsi le fine-tuning d'un modèle de langage (LLM). Le processus débute par la saisie des intentions via une interface graphique dédiée (voir figure 27), où l'utilisateur entre une liste d'intentions principales dans des champs texte. Une validation en temps réel, assurée par un module de contrôle, vérifie l'unicité des entrées en les comparant à une liste interne temporaire, signalant tout doublon par une alerte. Pour chaque intention, des exemples de phrases sont ajoutés dans une zone multiligne, soumis à un filtre basé sur des expressions régulières pour exiger au moins trois mots et exclure les caractères spéciaux invalides.

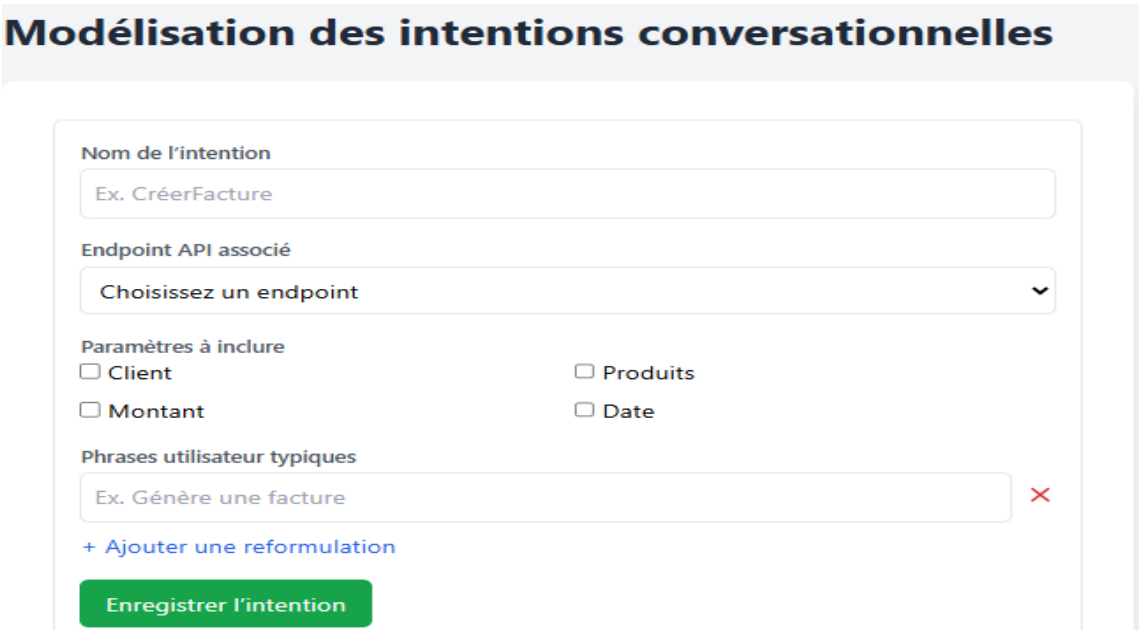


Figure 27 – *Interface de modélisation : saisie des intentions et validation des phrases.*

Ces données alimentent ensuite un mapping intentions-actions, orchestré par un moteur de transformation MDD. Ce mapping lie chaque intention aux actions de l'API, en s'appuyant sur le schéma JSON précédent (ex. : `api_schema.json`), avec une validation automatique comparant les entités et méthodes disponibles. Le résultat est sauvegardé dans un fichier JSON, généré via la bibliothèque `json`, illustrant une structure typique comme :

```
{
  "intentions": {
    "intention1": {
      "examples": ["phrase exemple 1", "phrase exemple 2"],
      "action": {
        "endpoint": "/chemin",
        "method": "GET",
        "payload": {}
      }
    }
  }
}
```

Une validation croisée, effectuée avec `requests`, teste chaque endpoint associé via une requête HTTP, vérifiant les codes de réponse (200 ou 400), et enregistre les résultats dans `validation_intentions.log`. Toute erreur (endpoint inaccessible, payload incorrect) déclenche une notification pour correction. Ce mapping prépare les données d'entraînement du LLM en annotant les phrases pour l'apprentissage.

L'étape s'achève par l'intégration du fichier dans l'espace de travail, référencé dans `meta.yaml`, avec une journalisation des actions horodatées (ex. : "2025-07-01 04 :50 WAT Mapping intentions généré"), assurant une traçabilité complète. Cette structure pave la voie pour la phase suivante, où les dialogues seront peaufinés.

4. Expansion et Génération du corpus conversationnel

L'expansion et la génération du corpus conversationnel s'affirment comme une étape cruciale dans l'Atelier de Génie Logiciel (AGL), visant à enrichir un assistant conversationnel avec des données variées. Cette phase transforme les intentions en un ensemble prêt pour le fine-tuning d'un modèle de langage (LLM), reliant les expressions naturelles à des actions structurées. Le processus (voir figure 28)débute par la création automatique de variantes à partir des intentions, réalisée par un modèle de paraphrase utilisant la bibliothèque Hugging Face Transformers. Ce modèle génère plusieurs reformulations de chaque phrase, en suivant des règles comme la préservation du sens et une longueur minimale de mots. Ces suggestions apparaissent dans une interface graphique, où l'utilisateur peut les ajuster, les supprimer ou en ajouter dans des champs prévus. Une vérification en temps réel, basée sur des expressions régulières, contrôle l'absence de caractères spéciaux invalides et la cohérence syntaxique, affichant une alerte en cas d'erreur.



Figure 28 – *Interface d'expansion et de validation des données*

Ces variantes sont ensuite liées à des commandes structurées, une tâche gérée par un moteur MDD. Chaque phrase est associée à une action tirée de l'API précédente, validée par une comparaison avec le schéma JSON existant. Le résultat est converti en un fichier d'entraînement au format .jsonl si GPT est le LLM, produit avec la bibliothèque `jsonlines`, illustrant une structure typique comme :

```
{"text": "phrase reformul e", "action": {"endpoint":
   "/chemin", "method": "GET", "payload": {}}}
```

Cette conversion inclut une validation : `requests` teste les endpoints avec un payload fictif, vérifiant les codes de réponse, et enregistre les résultats dans le fichier `training_validation.log`. Toute erreur (format incorrect, endpoint invalide) déclenche une notification pour correction. Ce corpus prépare les données pour le fine-tuning du LLM.

L'étape se termine par l'ajout du fichier dans l'espace de travail, référencé dans `meta.yaml`, avec une journalisation des actions horodatées (ex. : "2025-07-01 05 :06

WAT Corpus généré"). Cette structure ouvre la porte à la phase suivante de modélisation.

Formatage des données et entraînement du modèle

Le formatage des données et l'entraînement du modèle constituent une étape centrale dans l'Atelier de Génie Logiciel (AGL). Cette phase adapte le corpus conversationnel au fine-tuning d'un modèle de langage (LLM) et lance l'apprentissage, reliant les données préparées à un modèle fonctionnel. Le processus débute par le formatage automatique des données, ajusté selon le fournisseur du LLM choisi (comme OpenAI ou Hugging Face). À partir du fichier généré précédemment, un moteur de transformation utilise la bibliothèque `json` pour adapter la structure aux exigences spécifiques : par exemple, un format ligne par ligne pour OpenAI ou une conversion en dataset pour Hugging Face via `datasets`. Cette adaptation inclut une validation initiale, vérifiant la cohérence des paires texte-commande avec un schéma prédéfini, et signale toute erreur (ex. : champ manquant) via une alerte.



Figure 29 – *Interface de demarrage automatique du fine-tuning*

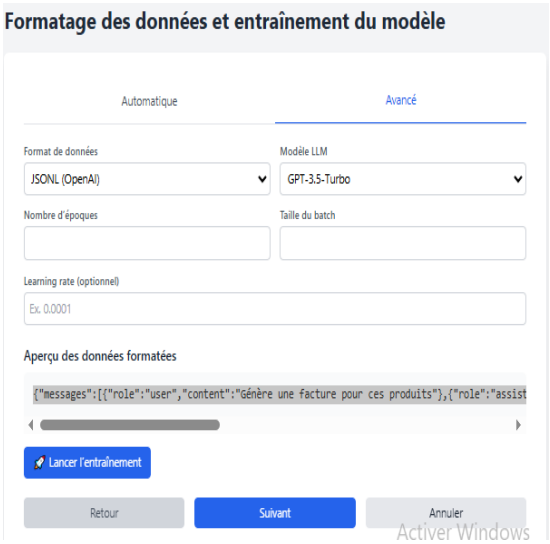


Figure 30 – *Interface de Choix du format du dataset et definition des hyperparamètres*

L'entraînement suit deux approches. Dans le mode automatique(voir figure 29), conçu pour les débutants, le système fixe des paramètres par défaut : une taille de lot de 4, 3 epochs, et un taux d'apprentissage de 0,0001, sélectionnés en fonction des recommandations du fournisseur LLM. L'utilisateur valide simplement le lancement via l'interface graphique. Une fois les paramètres définis, l'apprentissage démarre, supervisé par un outil comme TensorBoard, qui affiche en temps réel des métriques telles que la perte (loss) et l'exactitude. L'utilisateur peut suivre chaque étape (initialisation, itérations, convergence) et interrompre si nécessaire, les progrès étant sauvegardés à intervalles réguliers.

Pour le mode manuel, l'utilisateur configure les hyperparamètres du modele. Il entre une taille de lot (ex. : 4 ou 8), un nombre d'epochs (ex. : 3 à 5), et un taux d'apprentissage (ex. : 0,0001 à 0,001) dans des champs dédiés. Ces valeurs

sont validées en temps réel : la taille de lot doit être un multiple de 2, et le taux d'apprentissage doit rester dans une plage acceptable, avec une alerte en cas d'erreur. Une fois validés, ces paramètres sont appliqués, et l'apprentissage démarre, supervisé par TensorBoard qui affiche en temps réel des métriques comme la perte (loss) et l'exactitude.

L'étape se conclut par la création d'un modèle entraîné, intégré dans l'espace de travail avec un fichier de référence dans `meta.yaml`. Une journalisation enregistre les détails (ex. : "2025-07-01 05 :30 WAT - Modèle entraîné"), assurant une traçabilité complète. Ce modèle préparé ouvre la voie à la phase de test et d'optimisation.

Validation et tests fonctionnels

La validation et les tests fonctionnels forment une étape importante dans l'Atelier de Génie Logiciel (AGL). Cette phase teste le modèle après l'entraînement et prépare les ajustements ou le passage à la suite. Le processus démarre avec des tests du chatbot sur une interface graphique. L'utilisateur tape des requêtes en langage naturel dans un champ texte ou les dit à voix haute(voir figure 31), grâce à la bibliothèque `speech_recognition` pour la voix. Le chatbot analyse chaque demande, reconnaît l'intention, et crée une commande pour le système cible (ex. : un appel API). Chaque test donne un retour clair : un score de précision en pourcentage (calculé avec le F1-score), des logs d'exécution enregistrés dans `test_logs.txt` avec une date (ex. : "2025-07-04 05 :49 WAT - Test effectué"), et une liste des erreurs (ex. : intentions mal comprises).

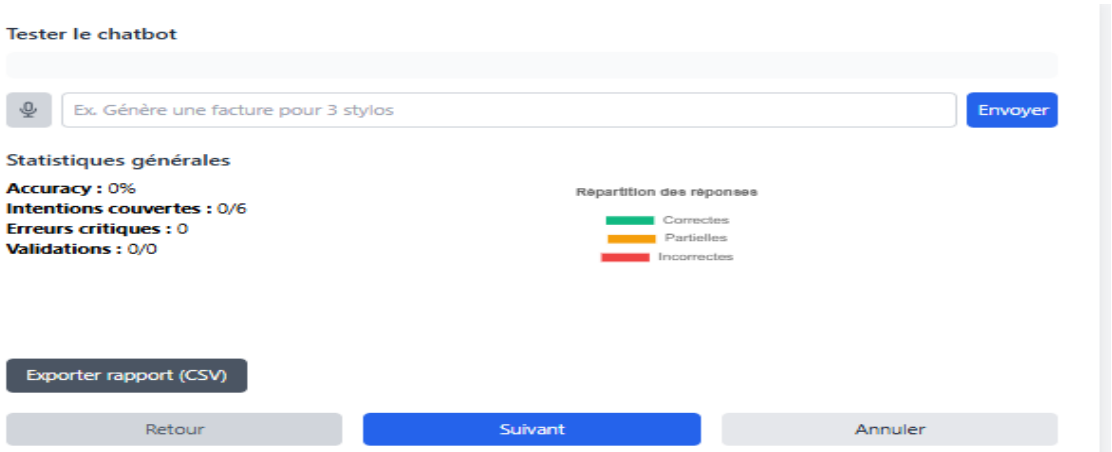


Figure 31 – *Interface de test*

Après les tests, l'utilisateur voit des statistiques générales à l'écran, comme la précision moyenne et le nombre d'erreurs. Si les résultats sont faibles (par exemple, précision sous 80%), il peut revenir au module 3. Là, il ajoute de nouvelles phrases dans l'interface, relance le formatage et l'entraînement avec un bouton dédié. S'il juge les résultats bons (précision au-dessus de 80%), la phase de personnalisation et déploiement commence.

L'étape se termine en sauvegardant les logs et statistiques dans l'espace de travail, notés dans `meta.yaml`, avec une trace des actions (ex. : "2025-07-04 05 :54 WAT - Tests terminés"). Cela prépare le modèle pour la suite.

Personnalisation et configuration de déploiement

La personnalisation et la configuration de déploiement marquent l'étape finale dans l'Atelier de Génie Logiciel (AGL). Cette phase ajuste le modèle validé et le prépare pour une utilisation réelle. Chaque action est organisée simplement pour garantir un résultat prêt. Le processus commence avec la personnalisation sur une interface graphique(voir figure 32. L'utilisateur choisit un nom unique pour le chatbot dans un champ texte, vérifié par un contrôle pour éviter les doublons. Il sélectionne un style d'interface (ex. : clair, sombre) et une tonalité (formelle, amicale, technique) via des menus déroulants. Ensuite, il configure les paramètres de déploiement : un port d'écoute (ex. : 5000), une URL d'accès (ex. : localhost), et des options de sécurité (ex. : HTTPS activé) ou mode (API avec Flask, microservice). Chaque choix est validé en temps réel, signalant les erreurs (ex. : port occupé).

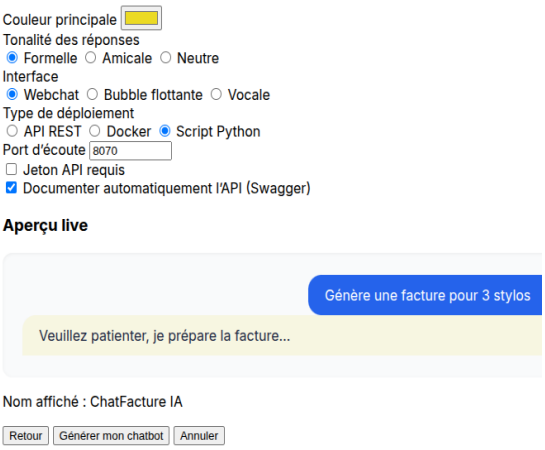


Figure 32 – *Prévisualisation de l'interface personnalisée avec choix de style et tonalité.*

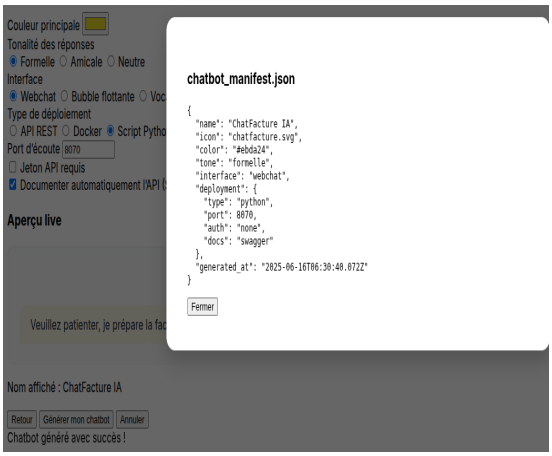


Figure 33 – *Affichage du contenu du fichier API généré.*

L'étape se termine quand le chatbot est prêt. Les fichiers générés sont notés dans `meta.yaml`, et une trace des actions est ajoutée (ex. : "2025-07-04 06 :15 WAT Déploiement finalisé"). Le modèle est alors disponible pour une utilisation dans un environnement réel, via un serveur local ou autre.

Exportation, archivage, versionnement et documentation

L'exportation, l'archivage, la versionnement, et la documentation marquent la dernière étape dans l'Atelier de Génie Logiciel (AGL). Cette phase regroupe tous les éléments en une archive prête à l'emploi et documentée. Chaque action est organisée clairement pour un résultat fiable, voyons comment cela se passe. Le processus commence avec l'exportation sur une interface graphique. L'utilisateur clique sur un bouton "Exporter" pour créer une archive .zip contenant le modèle entraîné, le code source, l'interface graphique, les fichiers de configuration, et un rapport PDF. Cette archive est générée avec la bibliothèque Python zipfile, vérifiant l'intégrité des fichiers

avant compression. Le rapport PDF, créé avec pdflatex, inclut des sections sur les choix effectués (ex. : tonalité), les performances (ex. : précision), et l'architecture (ex. : structure API).

Ensuite, la versionnement est gérée automatiquement. Chaque export reçoit un numéro (ex. : v1.0, v1.1) basé sur la date et l'heure (ex. : "2025-07-04_07 :11_WAT"), stocké dans un dossier versions dans l'espace de travail. L'utilisateur peut sélectionner une version précédente via un menu déroulant pour la restaurer ou l'analyser, avec une comparaison des performances affichée.

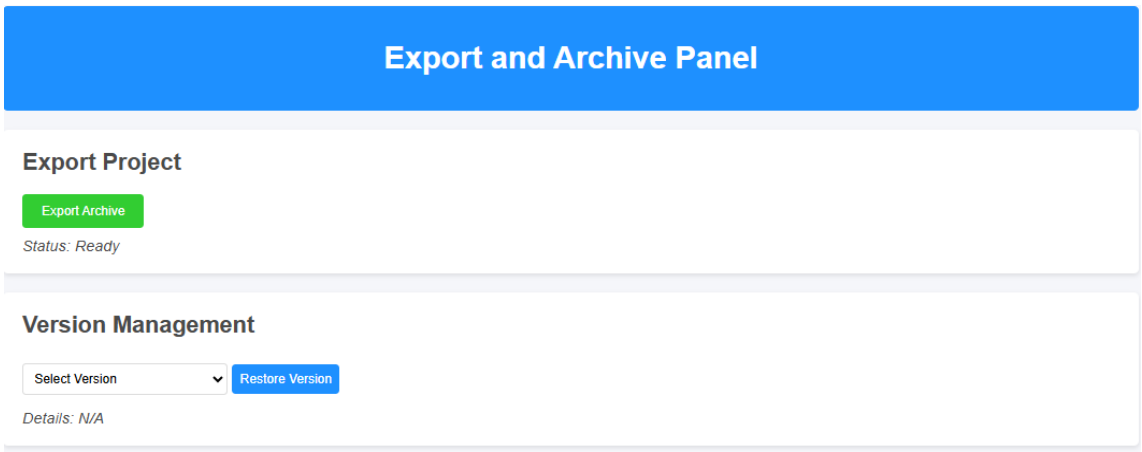


Figure 34 – *Interface d'exportation : bouton d'export et sélection des versions.*

L'étape se termine par la sauvegarde de l'archive et des versions dans l'espace de travail, notées dans `meta.yaml`, avec une trace des actions (ex. : "2025-07-04 07 :20 WAT - Archive exportée"). Cette structure permet de partager le projet ou de le conserver pour de futures mises à jour.

III.4. L'architecture logicielle interne de l'atelier (Vue développeur)

Jusqu'ici, nous avons vu comment un utilisateur pouvait créer un chatbot pas à pas grâce à un parcours fluide, structuré en modules clairs. Mais pour que cette simplicité apparente soit possible, une organisation technique solide doit exister en arrière-plan.

Ce que nous décrivons ici, c'est l'ossature logicielle de l'atelier : ses couches internes, ses composants fondamentaux, et les échanges entre eux. Autrement dit, comment le système est construit pour faire fonctionner tout ce que l'utilisateur voit et utilise. C'est une vue de l'intérieur, celle du développeur ou de l'architecte logiciel.

III.4.1. Vue d'ensemble :une organisation en couches, claire et modulaire

L'architecture de notre atelier est organisée en plusieurs couches, chacune ayant un rôle bien précis. Cette approche permet de séparer les responsabilités, de faci-

liter la maintenance, et surtout, de faire évoluer certaines parties sans devoir tout reconstruire.

Voici les principales couches qui composent notre système :

1. **La couche présentation** C'est la couche visible par l'utilisateur. Elle comprend l'interface web du logiciel, les écrans de configuration, les formulaires guidés et les tableaux de bord. Elle peut aussi proposer une API REST pour les utilisateurs qui veulent intégrer le système à leurs propres outils.
2. **La couche logique métier** C'est ici que se trouve le cerveau du système. Elle coordonne les actions entre les différents modules : création de projets, vérification des entrées, validation des étapes, gestion du parcours utilisateur. On y trouve l'orchestrateur principal, qui veille à ce que chaque action suive la bonne logique.
3. **La couche services LLM** C'est elle qui s'occupe des interactions avec les modèles de langage. Elle transforme les données d'entraînement au bon format, déclenche le fine-tuning via des appels API (vers OpenAI, HuggingFace ou autres), et gère les retours. Elle assure aussi l'inférence, c'est-à-dire la capacité du modèle à répondre aux requêtes.
4. textbfLa couche de données Toutes les informations du projet y sont stockées : les phrases utilisateur, les commandes associées, les fichiers de configuration, les logs d'entraînement et de test. C'est la mémoire du système. Elle utilise des formats simples comme JSON, CSV ou des bases de données légères selon les cas.
5. **La couche outils** Enfin, cette couche contient des outils techniques utiles pour faciliter le travail : des convertisseurs de fichiers, des analyseurs de texte, des générateurs d'exemples, ou des modules capables de lire des fichiers Swagger. Ces outils travaillent souvent en arrière-plan, mais sont indispensables au bon fonctionnement général.

Figure 35 – *Vue en couches de l'architecture logicielle de l'atelier*

III.4.2. Composants logiciels fondamentaux

À l'intérieur de ces couches, on trouve plusieurs composants logiciels : des blocs indépendants, chacun chargé d'une tâche précise. Ces composants interagissent entre eux pour que le système reste fluide, stable et réactif. Voici une description simple et précise des composants clés.

Le service de gestion des projets

C'est lui qui permet de créer un nouveau chatbot, de le sauvegarder, de le rouvrir plus tard, et de garder son historique. Il crée automatiquement les répertoires, initialise les fichiers de configuration, et conserve toutes les données du projet.

Le moteur de génération de données

Ce module transforme les intentions conversationnelles en exemples d'entrées utilisateur. Il propose des variantes de phrases, en s'appuyant sur des modèles prédéfinis ou directement sur un LLM. Il construit aussi les commandes JSON correspondantes, prêtes à être utilisées pour l'entraînement.

L'orchestrateur de fine-tuning

C'est le composant qui lance et supervise l'entraînement du modèle. Il prépare les données au bon format, choisit le modèle cible, communique avec les API externes (OpenAI, Hugging Face, etc.), et affiche l'évolution du processus pour l'utilisateur.

Le service d'interprétation des intentions

Ce module permet de tester le chatbot. Il reçoit une requête utilisateur, l'envoie au modèle, récupère la commande générée, puis affiche le résultat de manière lisible. Il permet aussi de noter les réponses et d'enregistrer les erreurs pour améliorer le système.

Le gestionnaire d'API métiers

Il est chargé de lire et interpréter les fichiers Swagger ou les configurations manuelles fournies par l'utilisateur. Il extrait les entités, les endpoints et les paramètres, pour permettre au chatbot de comprendre les actions possibles sur le système cible.

L'exportateur / versionneur de modèles

Une fois le chatbot validé, ce module assemble tous les éléments nécessaires dans une archive : modèle entraîné, code source, fichiers de configuration, interface, etc. Il attribue aussi un identifiant de version et génère un rapport automatique en PDF.

L'interface utilisateur guidée

Construite avec des technologies modernes (comme React ou Vue.js), elle permet à l'utilisateur d'interagir facilement avec le système. Elle masque toute la complexité technique, sans jamais brider les possibilités.

Le serveur de test interactif

Il permet d'évaluer le comportement du chatbot dans une vraie interface. L'utilisateur peut dialoguer avec l'agent, observer les résultats, et obtenir des rapports en direct. Ce composant est aussi utilisé pour simuler des scénarios métiers complexes.

III.5. Communication entre modules et persistance des données

Une fois que l'on connaît les composants internes de l'atelier, il est naturel de se poser deux questions essentielles : comment ces composants échangent des informa-

tions, et comment les données produites tout au long du projet sont stockées, organisées et retrouvées. AInsi, nous apporterons une réponse claire à ces deux aspects : la communication interne (entre les modules) et la gestion des données (modèle de persistance).

III.5.1. Comment les modules communiquent entre eux

Dans un atelier bien conçu, chaque module a un rôle bien précis. Mais pour que l'ensemble fonctionne harmonieusement, il faut que ces modules puissent échanger des informations sans confusion ni perte.

Nous avons choisi une communication basée sur des appels internes d'API. Autrement dit, chaque module dispose d'un petit service interne auquel les autres peuvent envoyer des messages. Ces messages sont envoyés au format JSON, un format simple, lisible, et standard, qui permet de transmettre des objets structurés (comme des projets, des jeux de données, des réponses, etc.). Ce mode de communication présente plusieurs avantages :

— Il permet à l'atelier d'être distribué sur plusieurs machines, si nécessaire.

— Il permet de séparer les modules : chacun peut être développé, testé et amélioré indépendamment.

— Il facilite les mises à jour : si un module évolue, il suffit d'adapter son API, sans toucher au reste.

Pour les actions plus complexes, comme le déclenchement de l'entraînement d'un modèle ou l'évaluation d'un chatbot, l'orchestrateur central agit comme chef de trafic : il reçoit la commande d'un module, la transmet au bon composant, récupère la réponse, puis l'envoie au module demandeur.

Figure 36 – *Schéma de communication entre les modules de l'atelier*

III.5.2. Comment les données sont organisées et conservées

Tout au long du projet, l'utilisateur produit et manipule une grande quantité de données : noms de projets, intentions conversationnelles, phrases d'exemple, commandes JSON, jeux d'entraînement, modèles entraînés, logs de test, etc.

Pour éviter de se perdre dans cette masse d'informations, l'atelier adopte une organisation claire et structurée, basée sur des dossiers de projet.

```
project_root: /projets/nom_du_projet
files:
  - name: meta.yaml
    content:
      name: nom_du_projet
      description: description_saisie
      llm: modele_choisi
      date: date_actuelle
  - name: llm_config.json
    content:
```

```
11        api_key: cle_chiffree
12        format: format_modele
13        endpoint: point_acces
14    - name: data/training
15    - name: data/test
16    - name: data/validation
17    - name: logs/ validation_log.txt
18    - name: docs/README.md
```

III.6. Validation théorique de l'architecture proposée

Dans une démarche de conception logicielle, la validation théorique constitue une étape essentielle pour évaluer la solidité, la cohérence et la pertinence d'une architecture avant même sa mise en œuvre. Il ne s'agit pas ici de tester du code ou de mesurer des performances, mais d'examiner si les choix faits lors de la conception répondent de manière structurée aux exigences initiales et respectent les principes fondamentaux du génie logiciel.Ainsi, nous analyserons l'architecture proposée selon trois axes : l'adéquation entre les besoins et les composants, la qualité intrinsèque de la structure logicielle, et la pertinence du cheminement utilisateur dans le cadre du workflow défini.

III.6.1. Alignement entre exigences et composants

L'atelier proposé a été conçu pour répondre à un ensemble clair d'exigences identifiées à la suite de l'analyse de la littérature et de l'observation des limites des outils existants. Parmi ces exigences figurent notamment la simplification du processus de fine-tuning, l'encadrement pas à pas de l'utilisateur, la réutilisabilité des modules et l'accessibilité à des profils non techniques. Chaque module fonctionnel de l'atelier a été pensé pour répondre à un besoin précis. Par exemple, le module de chargement des données vise à rendre l'étape de préparation du dataset plus intuitive, souvent considérée comme un frein majeur dans les solutions actuelles. Le module de configuration du modèle permet quant à lui de guider l'utilisateur dans la sélection du LLM et l'ajustement des paramètres sans exiger de compétences avancées en apprentissage automatique. Le module de génération automatique du format de fine-tuning offre une abstraction du format JSONL, généralement source d'erreurs pour les débutants.

Un tableau de correspondance entre exigences et modules, accompagné d'un schéma, peut utilement être intégré ici pour illustrer cette traçabilité.

III.6.2. Évaluation des critères de qualité logicielle

La qualité d'une architecture logicielle se mesure aussi à sa capacité à évoluer, être comprise, réutilisée et maintenue sans engendrer de complexité excessive. L'atelier proposé a été structuré autour de principes simples et éprouvés du génie logiciel.

- **Modularité** Chaque fonctionnalité de l'atelier est isolée dans un module distinct, selon le principe de séparation des responsabilités. Cela permet de développer, tester ou modifier un module sans impact direct sur les autres.
- **Réutilisabilité** Les composants sont conçus pour être indépendants du cas d'usage. Par exemple, le module de génération du jeu de données d'entraînement peut être utilisé aussi bien pour un chatbot de support client que pour un agent éducatif.
- **Extensibilité** L'architecture est pensée pour accueillir de futurs ajouts, comme l'intégration de modèles vocaux ou l'ajout de modules de post-traitement. L'usage de points d'extension définis facilite ces évolutions sans devoir refondre l'ensemble.
- **Accessibilité** L'un des objectifs majeurs de l'atelier est de rendre la création d'assistants conversationnels accessible à des non-experts. Le choix d'une interface guidée, illustrée par des maquettes claires, va dans ce sens. L'utilisateur n'a pas à manipuler directement du code ou des fichiers techniques complexes.

III.7. Validation expérimentale simulée (preuve de concept)

Ici, on présentera une simulation guidée du fonctionnement de l'atelier à travers un cas d'usage concret. L'objectif est d'évaluer la pertinence fonctionnelle de l'approche proposée à partir des maquettes interactives développées pour représenter l'ensemble du workflow utilisateur.

III.7.1. Présentation du cas d'usage choisi

Dans le cadre de la validation de notre atelier, nous avons mis en œuvre un cas d'usage représentatif et concret : la conception d'un assistant conversationnel destiné à simplifier la gestion des factures dans un système de commerce électronique. Ce domaine a été choisi en raison de sa fréquence d'usage, de la clarté de ses processus, et de la richesse des interactions qu'il implique. L'expérimentation a consisté à parcourir l'ensemble du workflow proposé par l'atelier, depuis l'initialisation du projet jusqu'au déploiement final du chatbot. Le cas a été construit à partir d'une API métier décrivant des entités courantes telles que `Facture`, `Commande` et `Client`, et expose plusieurs actions standards via Swagger.

Pour ce scénario, plusieurs intentions ont été définies, correspondant à des besoins récurrents dans le cadre de la facturation en ligne :

— **afficher_facture** : afficher les détails d'une facture spécifique,

— **creer_facture** : récupérer une facture au format PDF,

— **facture_commande_spécifique** : obtenir une facture associée à une commande précise,

— **supprimer_facture** : Supprimer la facture d'un client via son identifiant.

III.7.2. Parcours utilisateur dans l'atelier (via maquettes interactives)

L'utilisateur a été guidé à travers toutes les étapes du processus :

Page d'Accueil et Authentification Avant d'accéder aux fonctionnalités de l'atelier, l'utilisateur arrive sur une page d'accueil accueillante. Après s'être authentifié (via un système de connexion sécurisé par identifiant et mot de passe.



Figure 37 – *Page d'accueil*

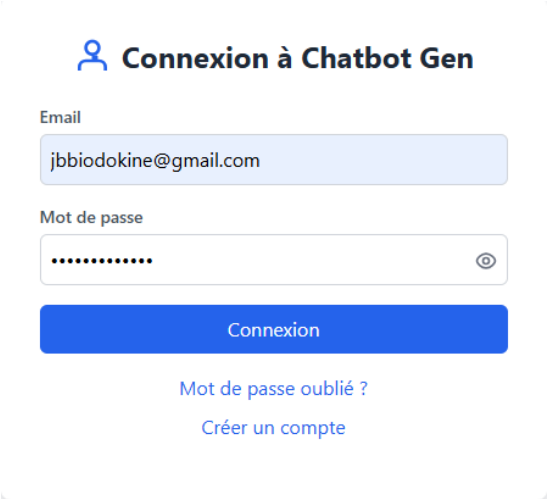


Figure 38 – *Interface d'authentification*

1. **Accès à l'espace personnel et création dun nouveau projet** Une fois authentifié, il accède au tableau de bord principal et crée un nouveau projet dans l'atelier.

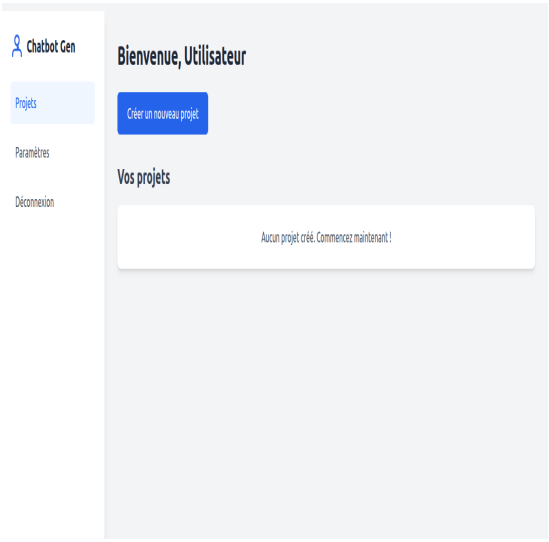


Figure 39 – *Dashboard utilisateur*

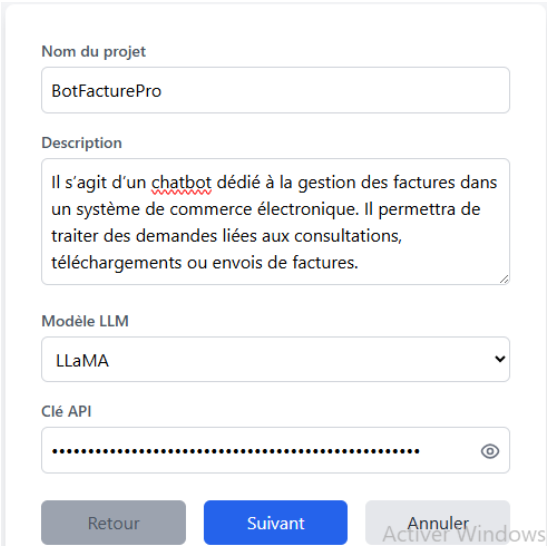


Figure 40 – *Formulaire de creation d'un projet de chatbot*

Apres avoir renseigné tous les champs du formulaire et configurer la clé API nécessaire pour la connexion au modèle de LLM choisi, L'atelier génère automatiquement un environnement de travail dédié, incluant une structure de répertoires, un fichier de configuration (project.json) et un identifiant unique pour le chatbot de facturation. Une vérification est effectuée pour s'assurer de la validité de la clé API et de la connexion au LLM. Le projet "Assistant Facturation" est désormais actif.

2. **Déclaration du Système Cible** L'assistant de facturation doit interagir avec le système métier de gestion de factures existant (par exemple, une base de données de clients ou un ERP). Nous allons donc importer un fichier Swagger/OpenAPI décrivant l'API de gestion des factures (contenant des endpoints comme /factures/creer, /commandes/creer, /Commandes/afficher)



Figure 41 – *Importation du fichier de spécification*

Ainsi, L'atelier analyse les spécifications fournies et génère un schéma structuré (en JSON) de tous les endpoints et actions disponibles. Cette "carte" est essentielle pour que le chatbot puisse associer les demandes utilisateur à des opérations concrètes sur le système existant de facturation.

3. **Modélisation des Intentions Conversationnelles** Apres avoir specifie l'API du systeme cible, l'utilisateur spécifie les requêtes que l'assistant de facturation devra comprendre. Il définit des intentions clés comme "CréerFacture", "ModifierFacture", "RechercherClient", "ListerProduits". Pour chaque intention, il fournit des exemples de phrases très spécifiques qu'un client pourrait formuler. Par exemple, pour "CréerFacture" : "Fais-moi une facture pour [Client] avec [Produit] et [Quantité]", ou "Je voudrais facturer [Article] pour [Nom_Client]". Ces phrases sont ensuite associées aux actions de l'API de facturation déclarées précédemment.

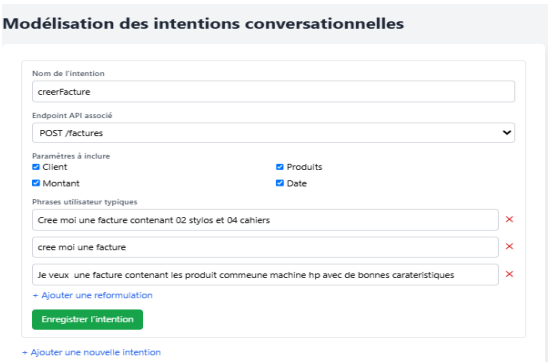


Figure 42 – *Definition des conversations*



Figure 43 – *Appercu du schema JSON genere*

Ensuite, Le système enregistre ce mappage entre les intentions des utilisateurs et les actions de l'API de facturation (généralement en YAML ou JSON), posant la première pierre de l'intelligence conversationnelle du chatbot.

4. **Expansion et Génération du Corpus Conversationnel** Ensuite, L'atelier aide à construire un corpus de données riche pour le fine-tuning. À partir des intentions et phrases d'exemple fournies, il génère automatiquement de multiples variantes de ces phrases (ex : "Établis une facture pour [Client] concernant [Produit] en [Quantité]", "Il faut éditer une note de frais pour [Client] et [Produit]"). L'utilisateur peut valider, modifier ou ajouter de nouvelles phrases pour enrichir la diversité linguistique et couvrir plus de cas. Chaque variante est associée à une commande structurée au format JSON, qui représente l'action concrète à déclencher sur le système de facturation.



Figure 44 – *Augmentation des données mappées à leurs actions*

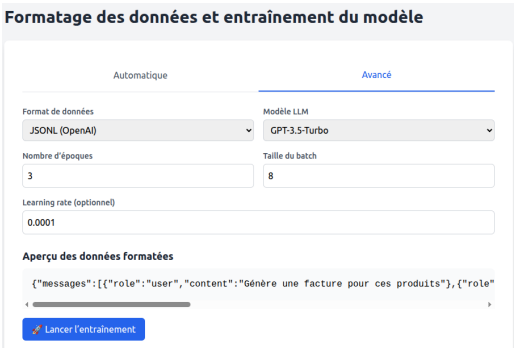


Figure 45 – *Formatage du dataset d'entrainement*

Une fois validées par l'utilisateur, toutes ces paires "phrase naturelle + commande JSON" sont transformées en un fichier d'entraînement (par exemple au format .jsonl dans notre cas) prêt à être utilisé pour l'adaptation du LLM. et l'on definit immediatement les hyperparametres pour lancer l'entrainement.

5. **Démarrage du fine-tuning, tests fonctionnels et validation des actions generées par le modele** Une fois les paramètres définis, le fine-tuning est lancé et L'utilisateur suit en temps réel l'évolution du fine-tuning, jusqu'à l'obtention d'un modèle conversationnel personnalisé, capable de comprendre les requêtes de facturation et de générer les commandes API correspondantes enfin, Le chatbot de facturation nouvellement entraîné peut être testé. L'utilisateur interagit avec lui via une interface simulant une conversation, en saisissant ou dictant des requêtes

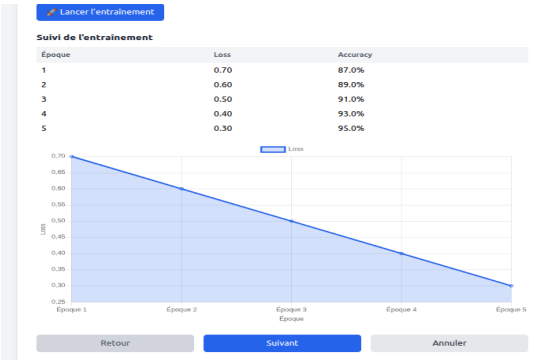


Figure 46 – *Suivi de l'entrainement du modele*



Figure 47 – *Interface de test du modele et validation*

Si les résultats ne sont pas satisfaisants (par exemple, le chatbot ne reconnaît pas une intention de facturation ou extrait mal une entité), l'utilisateur peut revenir aux modules précédents pour enrichir le corpus de données ou ajuster les intentions. Cela permet d'affiner progressivement le modèle de facturation.

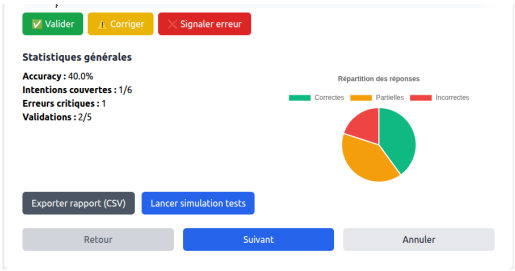


Figure 48 – *Interface d'affichage du Score de validation*

6. **Personnalisation, Configuration de Déploiement, Exportation, Archivage, Versionnement et Documentation** Une fois le chatbot de facturation validé, l'utilisateur lui donne sa touche finale. Il choisit un nom public, un style d'interface pour son déploiement web, et des paramètres de déploiement (port d'écoute, URL d'accès API REST, ou configuration pour un microservice Dockerisé). L'atelier génère automatiquement tous les éléments techniques nécessaires pour intégrer l'assistant de facturation dans un environnement réel, comme un fichier d'interface ou un point d'entrée API.

   Enfin, Pour finaliser le projet de l'assistant de facturation, l'utilisateur peut exporter une archive complète du projet. Cette archive contient le modèle entraîné, le code source de l'agent, l'interface graphique, les fichiers de configuration, et un rapport PDF détaillé résumant les choix effectués, les performances obtenues et l'architecture du chatbot de facturation

III.7.3. Résultats et analyse des tests

III.7.4. Discussion générale

À l'issue de la présentation des modules fonctionnels ci dessus qui composent le parcours utilisateur de notre atelier, il est essentiel de prendre du recul pour examiner, de manière critique mais constructive, les forces réelles de notre approche, ses limites actuelles, ainsi que les enjeux majeurs qui se dessinent pour les futurs usages.

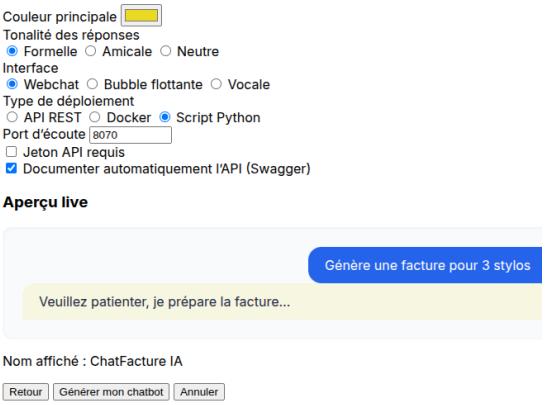


Figure 49 – *personnalisation et le déploiement du chatbot*



Figure 50 – *Fichier json produit*

Forces de l'approche

Notre atelier se distingue d'abord par sa capacité à intégrer de manière fluide les modèles LLM dans un environnement structuré de conception logicielle. Contrairement aux outils traditionnels, souvent techniques ou fragmentés, ici, l'utilisateur est guidé de bout en bout à travers un workflow clair, intuitif et progressif.

L'interaction proposée est simple, humanisée, sans sacrifier la rigueur des processus de développement. L'atelier exploite également les principes du Model-Driven Development (MDD), permettant de traduire des intentions métiers en actions techniques sans rupture de continuité.

Tout cela contribue à une expérience fluide, rassurante, et surtout, accessible à un public plus large que les seuls ingénieurs spécialisés.

Limites observées

Toutefois, plusieurs limites doivent être reconnues — et assumées — comme des opportunités de raffinement.

D'abord, une forme de rigidité du système peut apparaître dans certaines situations, notamment lorsque l'utilisateur sort du cadre standard prévu par le workflow. Par ailleurs, les contraintes propres au fine-tuning (coût, temps d'entraînement, dépendance au format des données) posent des défis techniques non négligeables.

De plus, l'intégration avec des systèmes tiers, bien que prévue, reste partiellement asynchrone ou non temps réel dans sa version actuelle. Cela peut limiter certaines applications critiques (ex. : systèmes industriels, soins médicaux).

Enjeux et défis à venir

Les enjeux futurs sont à la fois techniques, éthiques et stratégiques.

En premier lieu, la confidentialité des données constitue un impératif absolu, notamment dans les contextes réglementés. Il s'agira d'ajouter des couches de chiffrement, des mécanismes d'anonymisation, et des audits systématiques des logs.

Ensuite, l'atelier devra évoluer pour s'adapter à des domaines spécifiques, comme la santé ou l'éducation, où les exigences métiers sont complexes et critiques. Cela

suppose la possibilité d'intégrer des référentiels métiers, des validations humaines, et des contraintes réglementaires (RGPD, HDS...).

Enfin, la question de la scalabilité doit être posée avec sérieux. Si l'atelier est aujourd'hui performant sur des cas unitaires, son extension à une échelle multi-projets, multi-équipes ou multi-utilisateurs nécessite une réflexion poussée sur les infrastructures (conteneurs, bases distribuées, gestion centralisée des modèles...).

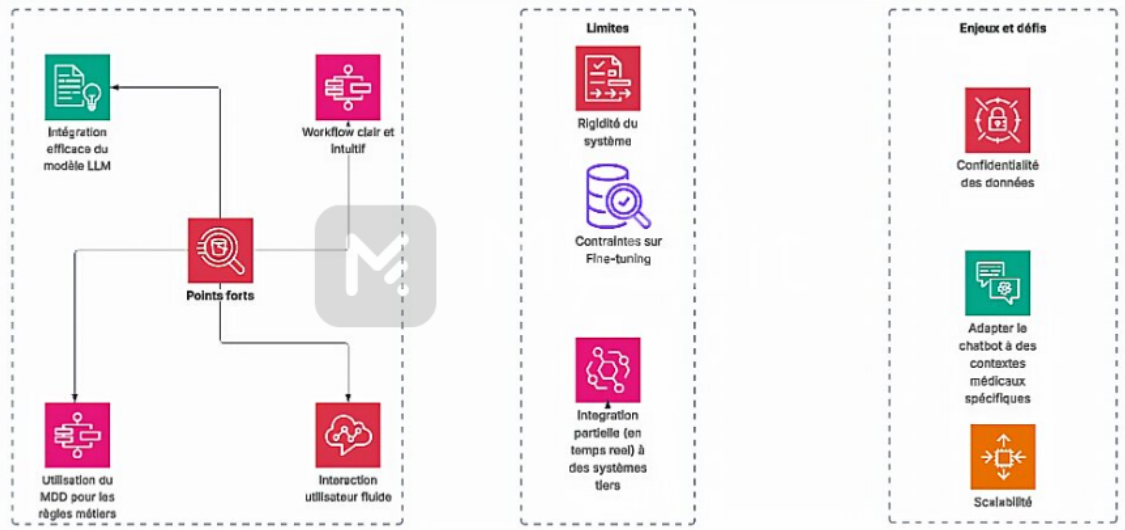


Figure 51 – *Synthèse des points forts, limites et défis du workflow proposé*

III.8. Conclusion

L'atelier présenté dans ce chapitre articule de manière cohérente une architecture logicielle robuste et un processus de conception guidé pour la création d'agents conversationnels fondés sur des LLMs. Chaque module logiciel, chaque étape du workflow a été défini pour répondre à des besoins réels, tout en respectant les principes d'accessibilité, de modularité et de réutilisabilité.

La validation théorique a permis de confirmer l'adéquation de l'architecture avec les exigences formulées dans les chapitres précédents. Quant à la validation expérimentale, elle a montré, à travers un scénario réaliste de gestion de factures, que l'atelier permet à un utilisateur non expert de produire un assistant conversationnel fonctionnel, en suivant un parcours progressif et encadré.

Ces résultats confirment la pertinence de la démarche adoptée et ouvrent des perspectives concrètes : généralisation à d'autres contextes applicatifs, intégration avec différents fournisseurs de LLMs, enrichissement des modules d'assistance et d'évaluation, voire déploiement dans des environnements industriels. L'approche proposée constitue ainsi une contribution utile à la démocratisation des technologies conversationnelles fondées sur les LLMs.

Conclusion Générale

Sommaire

La problématique étudiée et les choix méthodologiques 77
Analyse et critique des résultats obtenus 77
Perspectives 78

En somme, ce travail s'est inscrit dans une volonté de rendre plus simple, plus claire et plus accessible la création d'assistants conversationnels intelligents, en s'appuyant sur la puissance des modèles de langage de grande taille. L'objectif n'était pas seulement de concevoir un prototype, mais de poser les bases d'un véritable outil logiciel, structuré et réutilisable, pensé pour des utilisateurs de différents niveaux, même sans expertise en intelligence artificielle.

La problématique étudiée et les choix méthodologiques

La problématique principale de ce travail part d'un constat simple : il existe peu de solutions logicielles qui permettent de créer facilement des assistants conversationnels personnalisés à partir de LLMs, sans devoir passer par des processus complexes, souvent réservés à des experts. C'est face à ce besoin que l'idée de construire un atelier logiciel, structuré et modulaire, a émergé.

Pour y répondre, nous avons suivi une approche inspirée du génie logiciel. Nous avons analysé les outils existants, puis proposé une architecture organisée en modules clairs, chacun correspondant à une étape du cycle de vie d'un chatbot : de la définition des besoins jusqu'au déploiement. Ce choix permet à l'utilisateur d'avancer étape par étape, avec un accompagnement guidé à chaque niveau.

Analyse et critique des résultats obtenus

Les résultats obtenus montrent que l'atelier proposé permet de guider efficacement un utilisateur dans la création d'un assistant conversationnel personnalisé. Grâce à une interface simple et des modules intégrés, il est possible de générer des exemples de dialogue, d'adapter un modèle de langage existant, de tester les interactions et de déployer un assistant opérationnel. L'étude de cas centrée sur la

génération de factures a permis de valider l'approche. Même sans expertise technique avancée, un utilisateur peut suivre le processus et obtenir un résultat fonctionnel.

Cependant, certaines limites sont apparues, et méritent d'être analysées.

Tout d'abord, l'atelier repose encore fortement sur la qualité des données fournies. Dans les cas où les exemples sont peu nombreux ou mal structurés, le fine-tuning devient imprécis, ce qui réduit la qualité des réponses du chatbot. Cette dépendance pose problème dans les domaines où les données sont sensibles, rares ou difficiles à formuler.

Ensuite, l'atelier reste généraliste. Il est conçu pour s'adapter à plusieurs contextes, mais cette flexibilité a aussi un coût : il devient difficile de l'appliquer directement à des métiers très spécifiques, où les logiques de dialogue et les règles métiers sont complexes ou très spécialisées.

Enfin, l'absence d'un mécanisme intégré de retour utilisateur limite la capacité du système à s'améliorer dans le temps. Aujourd'hui, si le chatbot donne une mauvaise réponse, aucune boucle de rétroaction automatique ne permet de corriger ou d'enrichir son comportement.

Ces éléments montrent que, même si l'atelier constitue une base solide, plusieurs pistes d'amélioration sont possibles : renforcer les outils de génération et de validation des données, ajouter des modules d'adaptation métier plus ciblés, et intégrer un système de suivi des performances et des retours utilisateurs.

Ce travail ouvre donc la voie à des évolutions futures, tant sur le plan technique que méthodologique, pour construire des assistants plus robustes, plus évolutifs, et encore mieux intégrés dans leur environnement.

Perspectives

Plusieurs pistes d'amélioration sont possibles pour les suites de ce travail. L'une d'elles serait d'ajouter un système d'auto-évaluation, pour que le chatbot puisse ajuster ses réponses selon ses performances. Une autre serait de permettre au chatbot d'apprendre progressivement à partir des interactions réelles avec les utilisateurs.

Il serait également intéressant d'ouvrir notre atelier à d'autres types de modèles, et à plusieurs plateformes d'entraînement, pour offrir plus de choix et plus de flexibilité. Enfin, une validation dans plusieurs contextes professionnels permettrait de mieux évaluer son impact, de l'adapter à différents usages, et de l'améliorer encore.

À long terme, l'ambition est de faire de cet atelier une solution simple, ouverte et fiable, qui permet à toute organisation de créer ses propres assistants conversationnels, sans avoir besoin de connaissances techniques avancées.

Bibliographie

[1] José Alfonso Aguilar-Calderón, Carolina Tripp-Barba, and Martínez-Gárate. Model-Driven Approaches for Conversational Agent Development : A Systematic Mapping Study. *IEEE Access*, 2023. Le volume et les numéros de page spécifiques ne sont pas explicitement indiqués dans l'extrait fourni [1].

[2] Barry W. Boehm. A spiral model of software development and enhancement. *ACM SIGSOFT Software Engineering Notes*, 11(4) :14–24, 1986.

[3] Botpress Team. Botpress - the open-source conversational ai platform. https://botpress.com, 2023. Consulté en juin 2025.

[4] Botpress Team. Botpress + llm : Enhancing bots with generative ai. https://botpress.com/docs/guides/llm-integration, 2023. Consulté en juin 2025.

[5] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33 :1877–1901, 2020.

[6] Gwendal Daniel, Jordi Cabot, Laurent Deruelle, and Mustapha Derras. Xatkit : A multimodal low-code chatbot development framework. *IEEE Access*, 8 :15332–15350, 2020.

[7] Mustapha Derras et al.

[8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert : Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, 1(1) :4171–4186, 2019.

[9] Google Cloud. Dialogflow documentation. https://cloud.google.com/dialogflow/docs, 2024. Consulté en juin 2025.

[10] Standish Group. Chaos report 2020. Technical report, The Standish Group International, 2020. https://www.standishgroup.com/sample_research_files/CHAOSReport2020.pdf.

[11] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Lu Wang, and Weizhu Chen. Lora : Low-rank adaptation of large language models. *arXiv preprint arXiv :2106.09685*, 2022.

[12] Ming Huang, Zhiyuan Zhou, and Dan Yang. Conversational ai : Dialogue systems, conversational agents, and chatbots. *Foundations and Trends® in Information Retrieval*, 14(1) :1–122, 2020.

[13] Hugging Face. Parameter-efficient fine-tuning (peft) library. https://huggingface.co/docs/peft, 2023. Consulté en juin 2025.

[14] Padma Iyenghar, Friedrich Otte, and Elke Pulvermueller. AI-guided Model-Driven Embedded Software Engineering. In *Proceedings of the 10th International Conference on Model-Driven Engineering and Software Development (MODELSWARD 2022)*, pages 395–404, 2022.

[15] Zhewei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yujie Xu, Eric Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12) :1–38, 2023.

[16] Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. Prentice Hall, 2000.

[17] Barbara Kitchenham and Stuart Charters. Guidelines for performing systematic literature reviews in software engineering. *EBSE Technical Report*, 2(1) :1–57, 2007.

[18] Takwa Kochbati. Bridging the gap between natural language system requirements and architecture design models. Technical report, Université Paris-Saclay, CEA, Institut LIST, 2021. Disponible via HAL.

[19] Takwa Kochbati. *Combler le fossé entre les exigences des systèmes de langage naturel et les modèles de conception d'architecture*. PhD thesis, Université Paris-Saclay, 2021. Thèse de doctorat, soumise le 2 novembre 2021. Identifiant HAL : tel-03411393 [2, 3].

[20] Praveen Kumar Koppanati. Leveraging ai-powered chatbots for real-time test monitoring and reporting. *Institute of Technology, Illinois*, September 2024. Disponible via ResearchGate.

[21] LangChain Team. Langchain documentation. https://docs.langchain.com, 2023. Consulté en juin 2025.

[22] Craig Larman. *Agile and Iterative Development : A Manager's Guide*. Addison-Wesley Professional, 2003.

[23] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33 :9459–9474, 2020.

[24] J. Mayo. *Programmation du framework Microsoft BOT : une approche multi-plateforme pour la création de chatbots*. Microsoft Press, 2017.

[25] Michael McTear. *Conversational AI : Dialogue Systems, Conversational Agents, and Chatbots*. Springer, Cham, 2020.

[26] Object Management Group (OMG). Mda guide version 1.0.1. Technical report, OMG, 2003.

[27] OpenAI. Fine-tuning gpt-3.5 and gpt-4 models. https://platform.openai.com/docs/guides/fine-tuning, 2024. Consulté en juin 2025.

[28] Sara Pérez-Soler, Esther Guerra, and Juan de Lara. Développement de chatbots pilotés par modèle. In *Ingénierie Dirigée par les Modèles (IDM 2023)*, Autonomous University of Madrid, Spain, 2023. Disponible en ligne : https://idm2023.example.org [à adapter selon source réelle].

[29] Ronald A. Radice and Michael Harding. *Software Engineering Techniques : A Report on a Conference Sponsored by the NATO Science Committee*. NATO Science Committee, 1985. Discussion on the V-Model and software lifecycle.

[30] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140) :1–67, 2020.

[31] S. Raj, K. Raj, and Karkal. *Création de chatbots avec Python*. Apress, 2019.

[32] Rasa Technologies. Rasa studio documentation. https://rasa.com/docs/studio, 2023. Consulté en juin 2025.

[33] Winston W. Royce. Managing the development of large software systems. In *Proceedings of IEEE WESCON*, pages 1–9. IEEE, 1970.

[34] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha. Étude systématique de l'ingénierie des prompts dans les grands modèles de langage : techniques et applications. *Prépublication arXiv*, 2024. arXiv :2402.07927.

[35] Jean-Paul Sansonnet, David Leray, and Jean-Claude Martin. Architecture d'un cadre d'assistance générique aux agents conversationnels. In J. et al. Gratch, editor, *Intelligent Virtual Agents (IVA 2006), Lecture Notes in Artificial Intelligence*, volume 4133, pages 145–156. Springer-Verlag Berlin Heidelberg, 2006.

[36] Douglas C Schmidt. Model-driven engineering. *Computer*, 39(2) :25–31, 2006.

[37] W. Shalaby, A. Arantes, T. Gonzalez-Diaz, and C. Gupta. Création de chatbots à partir de bases de connaissances à grande échelle et spécifiques à un domaine : défis et opportunités. In *Conférence internationale IEEE 2020 sur le pronostic et la gestion de la santé (ICPHM)*, page 18. IEEE, jun 2020.

[38] Zéphyrin Soh. Cours de génie logiciel. https://hal.science/hal-01092795/document, 2008. Support pédagogique, Université de Yaoundé I, Année 2008-2009.

[39] H. Strobelt, A. Webson, V. Sanh, B. Hoover, J. Beyer, H. Pfister, and A. M. Rush. Ingénierie interactive et visuelle des invites pour l'adaptation de tâches ad hoc avec de grands modèles linguistiques. *Transactions IEEE sur la visualisation et l'infographie*, 29(1) :1146–1156, 2022.

[40] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[41] Aaron White, Yujia Zhang, Yuxin Lu, Jordan Boyd-Graber, et al. Prompt engineering : from scientific theory to engineering practice. *arXiv preprint arXiv :2306.05685*, 2023.

[42] Wikipédia contributors. Atelier de génie logiciel — wikipédia, 2024. Page consultée le 11 juin 2025.

[43] Shuo Zhou, Yuxuan Shen, Shu Wang, Qi Li, and Jindong Tang. A survey of large language models. *arXiv preprint arXiv :2303.18223*, 2023.